

CUDA Kernel 面试背题笔记

本书整理自 LeetCUDA 项目，系统汇编了 CUDA 面试中最高频的 GPU Kernel 实现，涵盖从 Warp/Block 原语、Elementwise、Softmax、GEMM、FlashAttention 等面试核心知识体系。

每个 Kernel 配备详细的中文面试要点注释 (WHY + HOW)，并以递进式优化路线 (naive → tiling → vectorize → tensor core → warp specialized) 展示 GPU 性能调优思维。

30 个 Kernel · 9 个 Phase · 全中文注释

项目主页: <https://github.com/xlite-dev/LeetCUDA>

更新日期: 2026 年 7 月 10 日

```

1 // =====
2 // notes-v2.cu — CUDA Kernel 面试刷题笔记
3 // =====
4 //
5 // 整理自 LeetCUDA 项目 (https://github.com/xlite-dev/LeetCUDA) , 涵盖:
6 //   - 面试高频 CUDA kernel 的完整实现 (~30 个 kernel)
7 //   - 每类 kernel 附带详细的面试要点注释 (WHY + HOW)
8 //   - 优化技术的递进式讲解 (naive → tiling → vectorize → tensor core → ws)
9 //   - BLAS 语义: N=col-major(Normal), T=row-major(Transposed)
10 //
11 // 10 个 Phase 覆盖:
12 //   Phase 0 — 面试框架速查 (GPU 架构 / Memory Hierarchy / Roofline / 优化清单)
13 //   Phase 1 — 基础原语: Warp Reduce / Block Reduce / Dot Product (含 broadcast 增强版)
14 //   Phase 2 — Elementwise: ReLU / Elementwise Add / Histogram (基础 + float4 向量化 + atomic)
15 //   Phase 3 — Softmax: naive → safe → online + RMS/Layer Norm
16 //   Phase 4 — RoPE: 旋转位置编码 (Llama 风格 theta=10000)
17 //   Phase 5 — Mat Transpose: 基础版 + BCF merge_write 最佳版 (Bank Conflict专题)
18 //   Phase 6 — GEMV: SGEMV K32/K128/K16 (warp-per-row)
19 //   Phase 7 — GEMM *: SGEMM → HGEMM → MMA m16n8k16(TN布局) → WGMMMA m64n128k16
20 //   Phase 8 — FlashAttention split_q (FA-2, 含 online softmax + P@V 寄存器复用)
21 //
22 // =====
23 // Phase 0: 面试框架速查 (纯注释, 面试开场必备的基础知识)
24 // =====
25
26 // ---- GPU 架构速查 ----
27 //
28 // SM (Streaming Multiprocessor) 内部结构:
29 //   - Warp Scheduler x4: 每 SM 4 个 warp scheduler, 每个每周期可发射 1 条指令
30 //   - Register File: 每 SM 65536 × 32-bit (4 bytes) = 256KB
31 //   - Shared Memory / L1: 可配置, 最大 shared memory ~228KB (Hopper)
32 //   - Tensor Cores: Hopper 每 SM 4 个; Blackwell 数量随型号/定义不同, 建议以官方 ISV guide 为准
33 //   - Warp = 32 threads: 最小调度单元, SIMT 执行模型
34 //
35 // Memory Hierarchy 带宽数量级 (H100 参考):
36 //   HBM3: ~3.35 TB/s (理论), 实际 ~2.5-3.0 TB/s
37 //   L2 Cache: ~12 TB/s (50MB, 跨 SM 共享)
38 //   L1/SMEM: ~19 TB/s (每 SM ~228KB)
39 //   Register: ~0 延迟, ~100+ TB/s 等效带宽
40 //
41 // 关键瓶颈判断:
42 //   Memory-bound: AI (Arithmetic Intensity) < 机器 FLOPS/带宽 比值
43 //   Compute-bound: AI 足够大, 受限于计算吞吐
44 //   Latency-bound: 线程不够多, 无法隐藏内存延迟
45 //
46 // Occupancy 公式:
47 //   occupancy = active_warps / max_warps_per_SM
48 //   受三类资源分别取下限: 每线程寄存器数 → threads/SM; 每 block shared memory
49 //   → blocks/SM; block 大小 → blocks/SM
50
51 // ---- 常见优化手段速查清单 ----
52 //
53 // 1. Coalesced Memory Access (合并访问)
54 //   - 同一 warp 的线程访问连续的 128B 对齐地址 → 1 次内存事务
55 //   - 否则产生多次事务 (最坏 32 次)
56 //
57 // 2. Tiling (分块)
58 //   - 将数据从 HBM 分块加载到 shared memory 复用, 减少 HBM 访问
59 //   - GEMM: Block Tile (BM×BN) + K Tile (BK)
60 //
61 // 3. Vectorized Memory Access (向量化)
62 //   - 使用 float4/half2 等向量类型, 减少 load/store 指令数

```

```

63 // - float4 = 128-bit, 单条指令加载 16 bytes
64 //
65 // 4. Thread Tile (寄存器分块)
66 // - 每个线程计算多个输出元素 (TM×TN), 提高计算密度
67 // - 减少线程总数, 降低同步开销
68 //
69 // 5. Bank Conflict Avoidance
70 // - Shared memory 有 32 banks × 4 bytes
71 // - 同 warp 多线程访问同一 bank 的不同地址 → bank conflict → 串行化
72 // - 解决方案: PAD (在每行末尾加 1 个元素打破对齐)
73 //
74 // 6. Pipeline / Double Buffering (流水线)
75 // - cp.async 异步拷贝下一批数据时同时做当前批的计算
76 // - Stage 数 = 2/3/4, 权衡 shared memory 占用和延迟隐藏
77 //
78 // 7. Tensor Core (MMA / WGMMa)
79 // - MMA m16n8k16 (Ampere): warp 级指令, 单 warp 完成 16×8×16 的矩阵乘
80 // - WGMMa m64n128k16 (Hopper): warpgroup 级指令 (128 threads), 异步执行
81 //
82 // 8. Warp Specialization (Hopper+)
83 // - Producer warpgroup 做 TMA 数据搬运, Consumer warpgroup 做计算
84 // - 通过 cuda::barrier 同步, 完全解耦数据搬运和计算
85 //
86 // 9. TMA (Tensor Memory Accelerator, Hopper+)
87 // - 硬件 DMA 引擎, 支持 1D~5D 寻址, 低寄存器开销
88 // - 配合 cp.async.bulk 实现异步数据搬运
89 //
90 // ---- Roofline 分析公式 ----
91 //
92 // AI (Arithmetic Intensity) = FLOPs / Bytes_transferred
93 //
94 // GEMM (M=N=K=4096):
95 // FLOPs = 2 × M × N × K = 2 × 40963 ≈ 137 GFLOPs
96 // Bytes = (M×K + K×N + M×N) × sizeof(float) ≈ 200 MB
97 // AI ≈ 137G / 200M ≈ 685 FLOPs/Byte → compute-bound (远超 H100 ridge point:
98 // FP16 TC ≈ 295:1, FP32 ≈ 20:1)
99 //
100 // GEMV (M=4096, K=4096):
101 // FLOPs = 2 × M × K = 2 × 40962 ≈ 33 MFLOPs
102 // Bytes = (M×K + K + M) × sizeof(float) ≈ 67 MB
103 // AI ≈ 33M / 67M ≈ 0.5 FLOPs/Byte → severely memory-bound
104 //
105 // Softmax (N=4096): AI ≈ (5×N) / (2×N×4) = 5/8 ≈ 0.625 FLOPs/Byte → memory-bound
106 //
107 // =====
108 // Phase 1: 头文件 + 宏定义 + 基础原语 (Warp Reduce / Block Reduce)
109 // =====
110 // 面试要点:
111 // - warp_reduce: 用 __shfl_xor_sync 做蝶形归约, O(logN) 步, 无需 shared
112 //   memory
113 // - block_reduce: 两级归约 (warp → shared memory → warp0 broadcast),
114 //   注意最后必须 broadcast 回所有线程 (__shfl_sync), 否则只有 warp0 知道结果
115 // - 为什么不用 __shfl_down_sync? xor 模式所有线程做相同工作量, 更均衡
116 //
117 #include <algorithm>
118 #include <cstring>
119 #include <cuda_fp16.h>
120 #include <cuda_runtime.h>
121 #include <float.h>
122 #include <stdio.h>
123 #include <stdlib.h>
124 #include <math.h>
125 #include <cublas_v2.h>

```

```

126 #include <cuda.h>
127 #include <cuda/barrier>
128
129 #define WARP_SIZE 32
130 #define INT4(value) (reinterpret_cast<int4*>(&(value)))[0])
131 #define FLOAT4(value) (reinterpret_cast<float4*>(&(value)))[0])
132 #define HALF2(value) (reinterpret_cast<half2*>(&(value)))[0])
133
134 // =====
135 // Phase 1a: Warp Reduce (warp 内归约, 纯寄存器操作, 无需 shared memory)
136 // =====
137
138 // Warp Reduce Sum — generic (used by both FP32 and FP16 contexts)
139 // 使用 __shfl_xor_sync 做蝶形归约 (butterfly reduction)
140 // 复杂度  $O(\log N)$ ,  $N=32$  时仅需 5 步
141 // 模板参数: T=数据类型, kWarpSize=segment width (默认 32)
142 // kWarpSize 会作为 __shfl_xor_sync 的第 4 个实参 width, 限制 shuffle 在同一 segment 内
143 // 当 kWarpSize < 32 (如 FA 中 kWarpSize=4) 时, 只有同 segment 的 lane 参与通信
144 // 蝶形归约示意 (以 warpSize=8 为例, 实际 warpSize=32 有 5 次迭代):
145 //
146 // 初始: 每个 lane 持有自己的值 v0..v7
147 // lane:  0    1    2    3    4    5    6    7
148 // val:  v0   v1   v2   v3   v4   v5   v6   v7
149 //
150 // mask=4 (第1次迭代, lane i 与 lane i^4 交换并累加):
151 //
152 //      ┌──────────┐
153 // 对:   (0,4) (1,5) (2,6) (3,7)
154 //
155 // lane:  0    1    2    3    4    5    6    7
156 // val:  v0+v4 v1+v5 v2+v6 v3+v7 v4+v0 v5+v1 v6+v2 v7+v3
157 //
158 // mask=2 (第2次迭代, lane i 与 lane i^2 交换并累加):
159 //
160 //      ┌───┐ ┌───┐ ┌───┐ ┌───┐
161 // 对:   (0,2) (1,3) (4,6) (5,7)
162 //      ─── 前4个一组 ───      ─── 后4个一组 ───
163 //
164 // lane:  0    1    2    3    4    5    6    7 (每lane持有前一轮2个值的和再加本轮配对)
165 // val:   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$ 
166 //      = v0+v2+v4+v6 ... (逐步归约)
167 //
168 // mask=1 (第3次迭代, lane i 与 lane i^1 交换并累加):
169 //
170 //      ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐
171 // 对:   (0,1) (2,3) (4,5) (6,7)
172 //
173 // lane:  0    1    2    3    4    5    6    7
174 // val:   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$  ← 所有 lane 拥有全归约结果!
175 //
176 // mask=0: 循环终止, 归约完成。
177 //
178 // 关键性质:
179 // - XOR 配对是对称的: lane i 的配对对象是 lane  $i^{mask}$ , 而  $(i^{mask})^{mask} = i$ 
180 // - 每轮每个 lane 只和恰好 1 个其他 lane 通信 (一对一, 无冲突)
181 // - 每轮信息传递距离减半:  $16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$  (距离减半, 信息翻倍)
182 // -  $O(\log_2 N)$  步完成, 无需 shared memory, 纯寄存器操作
183 // - __shfl_xor_sync 第四个参数 kWarpSize 限制 segment width:
184 //   当 kWarpSize=4 时, 只有同 segment(4个一组)内的 lane 参与 shuffle
185
186 template <const int kWarpSize = WARP_SIZE, typename T = float>
187 __device__ __forceinline__ T warp_reduce_sum(T val) {
188 #pragma unroll
189     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
190         val += __shfl_xor_sync(0xffffffff, val, mask, kWarpSize);
191     }
192     return val;
193 }

```

```

189 }
190
191 // Warp Reduce Max — generic
192 template <const int kWarpSize = WARP_SIZE, typename T = float>
193 __device__ __forceinline__ T warp_reduce_max(T val) {
194 #pragma unroll
195     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
196         val = max(val, __shfl_xor_sync(0xffffffff, val, mask, kWarpSize));
197     }
198     return val;
199 }
200
201 // =====
202 // Phase 1b: Block Reduce (block 内归约, 两级: warp → shared memory → warp reduce)
203 // =====
204
205 // Block Reduce Sum — FP32 (增强版, 带 broadcast)
206 // 两级归约流程:
207 // 1. 每个 warp 内做 warp_reduce_sum → 得到每 warp 的一个值
208 // 2. warp leader (lane=0) 写入 shared memory
209 // 3. syncthreads 后, lane 0~NUM_WARPS-1 读取 shared memory
210 // 4. 所有 warp 内再做一次 warp_reduce<NUM_WARPS> → 得到最终结果
211 // 5. __shfl_sync broadcast 到所有线程 (关键! 否则每个warp只有lane<NUM_WARPS 知道结果)
212 template <const int NUM_THREADS = 256>
213 __device__ float block_reduce_sum(float val) {
214     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
215     int warp = threadIdx.x / WARP_SIZE;
216     int lane = threadIdx.x % WARP_SIZE;
217     __shared__ float shared[NUM_WARPS];
218
219     float value = warp_reduce_sum<WARP_SIZE>(val);
220     if (lane == 0)
221         shared[warp] = value;
222     __syncthreads();
223     value = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
224     value = warp_reduce_sum<NUM_WARPS>(value);
225     // 关键: broadcast 结果到所有线程, 后续用 result
226     // 做除法等操作时所有线程都能拿到
227     value = __shfl_sync(0xffffffff, value, 0, 32);
228     return value;
229 }
230
231 // Block Reduce Max — FP32 (增强版, 带 broadcast)
232 template <const int NUM_THREADS = 256>
233 __device__ float block_reduce_max(float val) {
234     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
235     int warp = threadIdx.x / WARP_SIZE;
236     int lane = threadIdx.x % WARP_SIZE;
237     __shared__ float shared[NUM_WARPS];
238
239     float value = warp_reduce_max<WARP_SIZE>(val);
240     if (lane == 0)
241         shared[warp] = value;
242     __syncthreads();
243     value = (lane < NUM_WARPS) ? shared[lane] : -FLT_MAX;
244     value = warp_reduce_max<NUM_WARPS>(value);
245     value = __shfl_sync(0xffffffff, value, 0, 32);
246     return value;
247 }
248
249 // ---- Block Reduce Sum All: y = sum(a[0..N-1]) ----
250 // 多 block 各自做 warp→smem→warp0 reduce, 然后 atomicAdd 到全局 y
251 // 跨 block 求和的常见模式, 适合 N 较大时使用;

```

```

252 // Grid: ((N + 255) / 256, 1, 1)
253 // Block: (256, 1, 1)
254 // source: LeetCUDA/kernels/reduce/block_all_reduce.cu
255 template <const int NUM_THREADS = 256>
256 __global__ void block_reduce_all(float *a, float *y, int N) {
257     int tid = threadIdx.x;
258     int idx = blockIdx.x * NUM_THREADS + tid;
259     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
260     __shared__ float shared[NUM_WARPS];
261
262     float val = (idx < N) ? a[idx] : 0.0f;
263     int warp = tid / WARP_SIZE;
264     int lane = tid % WARP_SIZE;
265
266     val = warp_reduce_sum<WARP_SIZE>(val);
267     if (lane == 0)
268         shared[warp] = val;
269     __syncthreads();
270
271     val = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
272     if (warp == 0)
273         val = warp_reduce_sum<NUM_WARPS>(val);
274     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果，避免重复累加
275         atomicAdd(y, val);
276 }
277
278 // ---- Dot Product: y = sum(a[i] * b[i]) ----
279 // 核心模式: elementwise 乘法 → block reduce → atomicAdd 全局累加
280 // Grid: ((N + 255) / 256, 1, 1)
281 // Block: (256, 1, 1)
282 // source: LeetCUDA/kernels/dot-product/dot_product.cu
283 template <const int NUM_THREADS = 256>
284 __global__ void dot(float *a, float *b, float *y, int N) {
285     int tid = threadIdx.x;
286     int idx = blockIdx.x * NUM_THREADS + tid;
287     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
288     __shared__ float shared[NUM_WARPS];
289
290     float prod = (idx < N) ? a[idx] * b[idx] : 0.0f;
291     int warp = tid / WARP_SIZE;
292     int lane = tid % WARP_SIZE;
293
294     prod = warp_reduce_sum<WARP_SIZE>(prod);
295     if (lane == 0)
296         shared[warp] = prod;
297     __syncthreads();
298
299     prod = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
300     if (warp == 0) // 只需要 warp 0 的线程继续 reduce 即可
301         prod = warp_reduce_sum<NUM_WARPS>(prod);
302     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果，避免重复累加
303         atomicAdd(y, prod);
304 }
305
306 // Dot Product + float4
307 // Grid: ((N + 255) / 256, 1, 1), 每个block处理256元素，float4向量化后每个线程处理4元素
308 // Block: (64, 1, 1), 256/4=64
309 // 注意: 该版本默认输入地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
310 // source: LeetCUDA/kernels/dot-product/dot_product.cu
311 template <const int NUM_THREADS = 256 / 4>
312 __global__ void dot_vec4(float *a, float *b, float *y, int N) {
313     int tid = threadIdx.x;
314     int idx = (blockIdx.x * NUM_THREADS + tid) * 4;

```

```

315 constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
316 __shared__ float shared[NUM_WARPS];
317
318 float4 reg_a = FLOAT4(a[idx]);
319 float4 reg_b = FLOAT4(b[idx]);
320 float prod = (idx < N) ? (reg_a.x * reg_b.x + reg_a.y * reg_b.y +
321                          reg_a.z * reg_b.z + reg_a.w * reg_b.w)
322                  : 0.0f;
323 int warp = tid / WARP_SIZE;
324 int lane = tid % WARP_SIZE;
325
326 prod = warp_reduce_sum<WARP_SIZE>(prod);
327 if (lane == 0)
328     shared[warp] = prod;
329 __syncthreads();
330
331 prod = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
332 if (warp == 0)
333     prod = warp_reduce_sum<NUM_WARPS>(prod);
334 if (tid == 0)
335     atomicAdd(y, prod);
336 }
337
338
339 // =====
340 // Phase 2: Elementwise Ops (逐元素操作, 演示 coalesced access + vectorize)
341 // =====
342 // 面试要点:
343 //   - 逐元素操作是最简单的 kernel, 核心考点是 memory coalescing
344 //   - float4 向量化可将内存事务数减为 1/4, 大幅提升 bandwidth utilization
345 //   - grid/block 维度设计: grid(N/threads), block(threads), 一维即可
346
347 // ---- ReLU: y = max(0, x) ----
348 // Grid: ((N + 255) / 256, 1, 1)
349 // Block: (256, 1, 1)
350 // source: LeetCUDA/kernels/relu/relu.cu
351 __global__ void relu(float *x, float *y, int N) {
352     int idx = blockIdx.x * blockDim.x + threadIdx.x;
353     if (idx < N)
354         y[idx] = fmaxf(0.0f, x[idx]);
355 }
356
357 // ReLU + float4 向量化: 每个线程处理 4 个元素, 减少 75% 的 load/store 指令
358 // block(64)*4(float4)=256 元素/block, 与基础版吞吐相同
359 // Grid: ((N + 255) / 256, 1, 1)
360 // Block: (64, 1, 1)
361 // 注意: 该版本默认地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
362 // source: LeetCUDA/kernels/relu/relu.cu
363 __global__ void relu_vec4(float *x, float *y, int N) {
364     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
365     if (idx < N) {
366         float4 reg_x = FLOAT4(x[idx]); // 单条 128-bit load
367         float4 reg_y;
368         reg_y.x = fmaxf(0.0f, reg_x.x);
369         reg_y.y = fmaxf(0.0f, reg_x.y);
370         reg_y.z = fmaxf(0.0f, reg_x.z);
371         reg_y.w = fmaxf(0.0f, reg_x.w);
372         FLOAT4(y[idx]) = reg_y; // 单条 128-bit store
373     }
374 }
375
376 // ---- Elementwise Add: c = a + b ----
377 // Grid: ((N + 255) / 256, 1, 1)

```

```

378 // Block: (256, 1, 1)
379 // source: LeetCUDA/kernels/elementwise/elementwise.cu
380 __global__ void elementwise_add(float *a, float *b, float *c, int N) {
381     int idx = blockIdx.x * blockDim.x + threadIdx.x;
382     if (idx < N)
383         c[idx] = a[idx] + b[idx];
384 }
385
386 // Elementwise Add + float4 向量化
387 // Grid: ((N + 255) / 256, 1, 1)
388 // Block: (64, 1, 1), block(64)×4(float4)=256 元素/block
389 // 注意: 主路径要求 4 元素对齐; 尾部不足 4 个元素时回退到标量处理
390 // source: LeetCUDA/kernels/elementwise/elementwise.cu
391 __global__ void elementwise_add_vec4(float *a, float *b, float *c, int N) {
392     int idx = 4 * (blockIdx.x * blockDim.x + threadIdx.x);
393     if ((idx + 3) < N) {
394         float4 reg_a = FLOAT4(a[idx]);
395         float4 reg_b = FLOAT4(b[idx]);
396         float4 reg_c;
397         reg_c.x = reg_a.x + reg_b.x;
398         reg_c.y = reg_a.y + reg_b.y;
399         reg_c.z = reg_a.z + reg_b.z;
400         reg_c.w = reg_a.w + reg_b.w;
401         FLOAT4(c[idx]) = reg_c;
402     } else if (idx < N) {
403         for (int i = 0; (idx + i) < N; i++) {
404             c[idx + i] = a[idx + i] + b[idx + i];
405         }
406     }
407 }
408
409 // ---- Histogram: y[a[i]]++ ----
410 // 演示 atomicAdd 的用法: 多个线程可能同时更新同一个 bin
411 // Grid: ((N + 255) / 256, 1, 1)
412 // Block: (256, 1, 1)
413 // source: LeetCUDA/kernels/histogram/histogram.cu
414 __global__ void histogram(int *a, int *y, int N) {
415     int idx = blockIdx.x * blockDim.x + threadIdx.x;
416     if (idx < N)
417         atomicAdd(&(y[a[idx]]), 1);
418 }
419
420 // ---- Merge Attn States: 合并 Split-KV Attention 的部分结果 ----
421 // 实现 FlashInfer 论文 (arXiv 2501.01005) Section 2.2 的 attention 合并逻辑。
422 //
423 // 面试要点:
424 // - 应用场景: Split-KV attention 将序列沿 K 维分段计算 attention,
425 //   每段产出部分结果 (O_i, LSE_i), 本 kernel 将两段按指数权重加权合并。
426 // - 数学公式 (5 步推导):
427 //   Step 1 — max 归一化 (数值稳定, 与 safe softmax 同理):
428 //       L_max = max(LSE_1, LSE_2)
429 //   Step 2 — 还原指数权重 (un-normalized):
430 //       w_1 = exp(LSE_1 - L_max), w_2 = exp(LSE_2 - L_max)
431 //   Step 3 — 归一化为混合比例:
432 //       alpha = w_1 / (w_1 + w_2), beta = w_2 / (w_1 + w_2)
433 //       满足 alpha + beta = 1
434 //   Step 4 — 逐元素加权合并:
435 //       O = alpha * O_1 + beta * O_2
436 // - LSE 布局 [num_heads, num_tokens] (与 FlashAttention 惯例一致,
437 //   lse[head_idx][token_idx])
438 // - Output 布局 [num_tokens, num_heads, head_size] (token 维在最外,
439 //   展平为 [T0H0, T0H1, ..., T1H0, ...])
440 // - inf LSE → -inf: 空 attention 段 (causal mask 等导致全部 score

```



```

441 // 为 -inf) 的 LSE 可能为 +inf, 替换后  $\exp(-\text{inf} - L_{\text{max}}) = 0$ ,
442 // 该段权重退化为 0
443 // - 128-bit 向量化: uint4 = 4 × float, 每线程处理 4 个输出元素,
444 // pack load → FMA → pack store 一条流水线
445 // - AI ≈ 3 FLOP / 20 bytes ≈ 0.15 FLOPS/Byte → severely memory-bound
446 //
447 // 线程到数据的 3D 映射 (以 num_tokens=2, num_heads=2, head_size=8 为例):
448 // pack_size = 16 / sizeof(float) = 4 (每线程 4 个 float = 128-bit)
449 // threads_per_head = head_size / 4 = 2
450 // total_threads = num_tokens * num_heads * threads_per_head = 8
451 //
452 // global_idx:      0      1      2      3      4      5      6      7
453 // token_head_idx:  0      0      1      1      2      2      3      3
454 // (第几个 (token,head) 对, 行优先)
455 // pack_idx:        0      1      0      1      0      1      0      1
456 // (该 (token,head) 对内第几个 pack)
457 // token_idx:       0      0      0      0      1      1      1      1
458 // (token_head_idx / num_heads, token 变化慢)
459 // head_idx:        0      0      1      1      0      0      1      1
460 // (token_head_idx % num_heads, head 变化快)
461 // pack_offset:     0      4      0      4      0      4      0      4
462 // (pack_idx * 4, 该 pack 在 head_size 中的起始元素偏移)
463 // head_offset:     0      0      8      8      16     16     24     24
464 // (token_idx * num_heads * head_size + head_idx * head_size,
465 // 该 (token,head) 对在 output 展平数组中的起始偏移)
466 //
467 // Grid: ((total_threads + 127) / 128, 1, 1)
468 // Block: (128, 1, 1)
469 // source: LeetCUDA/kernels/openai-triton/merge-attn-states/cuda_merge_attn_states.cu
470 __global__ void merge_attn_states(
471     float *output,           // [num_tokens, num_heads, head_size]
472     const float *prefix_output, // [num_tokens, num_heads, head_size]
473     const float *prefix_lse,   // [num_heads, num_tokens]
474     const float *suffix_output, // [num_tokens, num_heads, head_size]
475     const float *suffix_lse,   // [num_heads, num_tokens]
476     int num_tokens, int num_heads, int head_size) {
477
478     constexpr int NUM_THREADS = 128;
479     constexpr int PACK_SIZE = 16 / sizeof(float); // 4 floats = 128-bit
480     using pack_t = uint4;
481
482     // 每个 (token, head) 对需要 threads_per_head 个线程覆盖 head_size 个元素
483     const int threads_per_head = head_size / PACK_SIZE;
484     // 实际有效总线程数, 超过这个数的线程会被忽略, block是按照NUM_THREADS=128
485     // 来分配的, 那么最后一个block的线程数可能会超过实际需要的线程数
486     const int total_threads = num_tokens * num_heads * threads_per_head;
487
488     const int global_idx = blockIdx.x * NUM_THREADS + threadIdx.x;
489     if (global_idx >= total_threads)
490         return;
491
492     // global_idx → (token_head_idx, pack_idx) → (token_idx, head_idx, pack_offset)
493     // token_head_idx: 第几个 (token, head) 对, 行优先展平
494     const int token_head_idx = global_idx / threads_per_head;
495     // pack_idx: 该 (token, head) 对内第几个 pack, 0 ~ threads_per_head-1
496     const int pack_idx = global_idx % threads_per_head;
497
498     // token_head_idx 分解为 (token_idx, head_idx)
499     const int token_idx = token_head_idx / num_heads; // token 变化慢 (外维)
500     const int head_idx = token_head_idx % num_heads; // head 变化快 (内维)
501
502     // pack_offset: 该 pack 覆盖的元素在 head_size 中的起始偏移 (0, 4, 8, ...)
503     const int pack_offset = pack_idx * PACK_SIZE;

```

```

504 // head_offset: 该 (token, head) 对在 output 展平一维数组中的起始偏移
505 const int head_offset = token_idx * num_heads * head_size + head_idx * head_size;
506
507 // 定位到当前 (token, head) 的输出段起始
508 const float *prefix_head = prefix_output + head_offset;
509 const float *suffix_head = suffix_output + head_offset;
510 float *output_head = output + head_offset;
511
512 // LSE 布局 [num_heads, num_tokens]: lse[head_idx][token_idx]
513 float p_lse = prefix_lse[head_idx * num_tokens + token_idx];
514 float s_lse = suffix_lse[head_idx * num_tokens + token_idx];
515
516 // inf → -inf: 空 attention 段 LSE 可能为 +inf, 替换后权重退化为 0
517 p_lse = isinf(p_lse) ? -INFINITY : p_lse;
518 s_lse = isinf(s_lse) ? -INFINITY : s_lse;
519
520 // Step 1: max 归一化 (与 safe softmax 同理, 防 exp 溢出)
521 const float max_lse = fmaxf(p_lse, s_lse);
522 p_lse -= max_lse;
523 s_lse -= max_lse;
524
525 // Step 2-3: 指数还原 → 混合比例 alpha, beta
526 const float p_se = expf(p_lse);
527 const float s_se = expf(s_lse);
528 const float out_se = p_se + s_se;
529 const float p_scale = p_se / out_se; // alpha = w_1 / (w_1 + w_2)
530 const float s_scale = s_se / out_se; // beta = w_2 / (w_1 + w_2)
531
532 // Step 4: 逐元素加权合并 0 = alpha * 0_1 + beta * 0_2
533 // 128-bit 向量化: uint4 load → per-element FMA → uint4 store
534 if (pack_offset < head_size) {
535     pack_t p_pack =
536         reinterpret_cast<const pack_t *>(prefix_head)[pack_idx];
537     pack_t s_pack =
538         reinterpret_cast<const pack_t *>(suffix_head)[pack_idx];
539     pack_t o_pack;
540
541 #pragma unroll
542     for (int i = 0; i < PACK_SIZE; ++i) {
543         const float p_v = reinterpret_cast<const float *>(&p_pack)[i];
544         const float s_v = reinterpret_cast<const float *>(&s_pack)[i];
545         const float o_v = p_v * p_scale + (s_v * s_scale); // FMA
546         reinterpret_cast<float *>(&o_pack)[i] = o_v;
547     }
548
549     reinterpret_cast<pack_t *>(output_head)[pack_idx] = o_pack;
550 }
551 }
552
553 // =====
554 // Phase 3: Reduce 类 Ops — Softmax / RMS Norm / Layer Norm
555 // =====
556 // 面试要点:
557 //   - Softmax 三种实现递进: naive (溢出) → safe (2-pass, max 减法) →
558 //     online (1-pass, 增量更新)
559 //   - Online Softmax 是 FlashAttention 的数学基础
560 //   - RMS Norm vs Layer Norm: RMS 只需 1 次 reduce, Layer Norm 需要 2 次 (mean
561 //     + variance)
562 //   - Per-token 设计: 一个 block 处理一个 token, 无需跨 block 同步
563
564 // =====
565 // Phase 3a: Softmax — 三级递进 (面试核心考点)
566 // =====

```

```

567 // 面试常问: 「Softmax 有哪些实现方式? 各有什么优缺点?」
568 // 回答线索: naive(溢出) → safe → online
569
570 // ---- Level 1: 基础 Softmax (per-token, 无 max 减法, 数值不稳定) ----
571 // grid(S*h/h, h), block(h), 一个 block 处理一个 token
572 // 问题: x 值很大时 exp(x) 溢出为 inf
573 template <const int NUM_THREADS = 256>
574 // Grid: (S, 1, 1), S=batch*seq_len, DISPATCH_SOFTMAX_F32_PER_TOKEN_KERNEL
575 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024, 一个 block 处理一个 token
576 // source: LeetCUDA/kernels/softmax/softmax.cu
577 __global__ void softmax_per_token(float *x, float *y, int N) {
578     const int tid = threadIdx.x;
579     const int idx = blockIdx.x * blockDim.x + tid;
580
581     float exp_val = (idx < N) ? expf(x[idx]) : 0.0f;
582     float exp_sum = block_reduce_sum<NUM_THREADS>(exp_val);
583     if (idx < N)
584         y[idx] = exp_val / exp_sum;
585 }
586
587 // ---- Level 2: Safe Softmax (2-pass: 先 max 再 exp, 数值稳定) ----
588 // 面试重点: 为什么 Softmax 需要 Safe?
589 //   - expf 溢出阈值约 88.7: exp(88)≈1.65e38 (接近 float32 上限 3.4e38), exp(89)≈4.5e38 已溢出为 inf
590 //   - 减去 max 后: exp(x - max) ≤ exp(0) = 1.0, 永不超过 1
591 //   - 数学等价性: softmax(x) = softmax(x - c) 对任意常数 c 成立
592 //   - 代价: 2 次 block reduce (先 max, 再 sum), 但仍 O(N/B) 高效
593 // Grid: (S, 1, 1)
594 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
595 // source: LeetCUDA/kernels/softmax/softmax.cu
596 template <const int NUM_THREADS = 256>
597 __global__ void safe_softmax_per_token(float *x, float *y, int N) {
598     const int tid = threadIdx.x;
599     const int idx = blockIdx.x * blockDim.x + tid;
600
601     // Pass 1: block reduce max — 找最大值
602     float val = (idx < N) ? x[idx] : -FLT_MAX;
603     float max_val = block_reduce_max<NUM_THREADS>(val);
604
605     // Pass 2: exp(x - max) → block reduce sum
606     float exp_val = (idx < N) ? expf(x[idx] - max_val) : 0.0f;
607     float exp_sum = block_reduce_sum<NUM_THREADS>(exp_val);
608
609     if (idx < N)
610         y[idx] = exp_val / exp_sum;
611 }
612
613 // ---- Level 3: Online Safe Softmax (FlashAttention 的数学基础) ----
614 // 面试重点:
615 // 两种使用场景 (公式不同, 但结果等价):
616 //   1) 单元素增量更新 (online update, 处理新元素 x_i):
617 //       m_new = max(m_old, x_i)
618 //       d_new = d_old * exp(m_old - m_new) + exp(x_i - m_new)
619 //   2) 二元合并 (binary merge, 合并两个部分累加器, warp/block reduce 使用):
620 //       m = max(m1, m2) safe softmax用的max值
621 //       d = d1*exp(m1-m) + d2*exp(m2-m) softmax用的分母值
622 //       当 m1≥m2 时退化为 d1 + d2*exp(m2-m1) (d_bigger + d_smaller*exp(m_smaller - m_bigger))
623 // 算法来源: "Online normalizer calculation for softmax" (arXiv:1805.02867)
624 // Warp Reduce for Online Softmax — binary merge of two partial (m,d) accumulators.
625 //
626 // 不同于单元素增量更新公式 (m_new = max(m_old, x_i); d_new = d_old*exp(m_old-m_new) + exp(x_i-m_new)),
627 // warp reduce 中每次合并的是两个**已各自归一化到各自 max 的部分累加器**:
628 //   (m1,d1): max 和 Σexp(x_j - m1), 覆盖集合 S1
629 //   (m2,d2): max 和 Σexp(x_k - m2), 覆盖集合 S2

```

```

630 //
631 // 合并公式 (对称, 不依赖"新/旧"概念):
632 //   m = max(m1, m2)
633 //   d = d1*exp(m1-m) + d2*exp(m2-m)   ← m1≥m2 时退化为 d1 + d2*exp(m2-m1)
634 //
635 // 该操作满足结合律 → XOR butterfly 任意归约顺序均保证全局正确结果。
636 struct __align__(8) MD {
637     float m; // running max
638     float d; // running denominator (sum of exp(x - max))
639 };
640
641 template <const int kWarpSize = WARP_SIZE>
642 __device__ __forceinline__ MD warp_reduce_md(MD md1) {
643     #pragma unroll
644     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
645         MD md2;
646         md2.m = __shfl_xor_sync(0xffffffff, md1.m, mask, kWarpSize);
647         md2.d = __shfl_xor_sync(0xffffffff, md1.d, mask, kWarpSize);
648
649         MD b_m = (md1.m > md2.m) ? md1 : md2; // max
650         MD s_m = (md1.m > md2.m) ? md2 : md1;
651
652         // b_m.d 无需 rescale (其基准 max 已是全局 max),
653         // s_m.d 需 rescale: exp(s_m.m - b_m.m)
654         md1.d = b_m.d + s_m.d * __expf(s_m.m - b_m.m);
655         md1.m = b_m.m;
656     }
657     return md1;
658 }
659
660 // 注意: 这里默认一个 block 处理一个 token; 边界线程的 d=0 只参与归约, 不会写回 y
661 // Grid: (S, 1, 1)
662 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
663 // source: LeetCUDA/kernels/softmax/softmax.cu
664 template <const int NUM_THREADS = 256>
665 __global__ void online_safe_softmax_per_token(const float *x, float *y, int N) {
666     int tid = threadIdx.x;
667     int idx = blockIdx.x * NUM_THREADS + threadIdx.x;
668     const int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
669     __shared__ MD shared[NUM_WARPS];
670
671     float val = (idx < N) ? x[idx] : -FLT_MAX;
672     int warp = tid / WARP_SIZE;
673     int lane = tid % WARP_SIZE;
674
675     // 初始化: 每个线程持有一个 (max, denom) 对
676     MD md;
677     md.m = idx < N ? val : -FLT_MAX;
678     md.d = idx < N ? 1.0f : 0.0f;
679
680     // 第一级规约: warp_reduce_md 在归约中自动更新 m 和 d
681     md = warp_reduce_md<WARP_SIZE>(md);
682
683     if (lane == 0)
684         shared[warp] = md;
685     __syncthreads();
686
687     // 第二级归约: 每个 warp 结果再做一次 warp_reduce_md (复用 block_reduce 模式)
688     md = lane < NUM_WARPS ? shared[lane] : MD{-FLT_MAX, 0.0f};
689     md = warp_reduce_md<WARP_SIZE>(md); // 用WARP_SIZE确保每个lane都能拿到最终结果
690
691     // 用全局 max 和 denom 做最终 softmax
692     float d_inv = __fdividef(1.0f, md.d);

```

```

693 // 边界线程即使看到 d=0 的填充值, 也不会走到写回路径
694 if (idx < N) {
695     y[idx] = __expf(val - md.m) * d_inv;
696 }
697 }
698
699 // =====
700 // Phase 3b: RMS Normalization (1-pass reduce)
701 // =====
702 // 面试要点:
703 //   - RMS Norm:  $y = (x / \text{rms}(x)) * g$ ,  $1/\text{rms}(x) = \text{rsqrt}(\text{mean}(x^2))$ 
704 //   - 只需 1 次 block reduce (sum of squares), 比 Layer Norm 少 1 次同步
705 //   - Llama 系列使用 RMS Norm
706 //   - grid(N, K/K), block(K): 一行一个 block
707 // Grid: (N, 1, 1), N=batch*seq_len, 每行一个 block
708 // Block: (128, 1, 1), NUM_THREADS=K=128 (K>128 时调整模板参数)
709 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
710 template <const int NUM_THREADS = 128>
711 __global__ void rms_norm(float *x, float *y, float g, int N, int K) {
712     int tid = threadIdx.x;
713     int idx = blockIdx.x * blockDim.x + threadIdx.x;
714     const float epsilon = 1e-5f;
715
716     __shared__ float s_variance;
717     float value = (idx < N * K) ? x[idx] : 0.0f;
718     float variance = value * value;
719     variance = block_reduce_sum<NUM_THREADS>(variance);
720     if (tid == 0)
721         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/rms(x)
722     __syncthreads();
723     if (idx < N * K)
724         y[idx] = (value * s_variance) * g;
725 }
726
727 // RMS Norm + float4
728 // Grid: (N, 1, 1)
729 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
730 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
731 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
732 template <const int NUM_THREADS = 128 / 4>
733 __global__ void rms_norm_vec4(float *x, float *y, float g, int N, int K) {
734     int tid = threadIdx.x;
735     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
736     const float epsilon = 1e-5f;
737
738     __shared__ float s_variance;
739     float4 reg_x = FLOAT4(x[idx]);
740     float4 variance = (idx < N * K) ? (reg_x.x * reg_x.x + reg_x.y * reg_x.y +
741                                     reg_x.z * reg_x.z + reg_x.w * reg_x.w)
742                                     : 0.0f;
743     variance = block_reduce_sum<NUM_THREADS>(variance);
744     if (tid == 0)
745         s_variance = rsqrtf(variance / (float)K + epsilon);
746     __syncthreads();
747     float4 reg_y;
748     reg_y.x = reg_x.x * s_variance * g;
749     reg_y.y = reg_x.y * s_variance * g;
750     reg_y.z = reg_x.z * s_variance * g;
751     reg_y.w = reg_x.w * s_variance * g;
752     if (idx < N * K)
753         FLOAT4(y[idx]) = reg_y;
754 }
755

```

```

756 // =====
757 // Phase 3c: Layer Normalization (2-pass reduce)
758 // =====
759 // 面试要点:
760 //   - Layer Norm:  $y = ((x - \text{mean}) / \text{std}) * g + b$ ,  $\text{std} = \sqrt{\text{variance}}$ ,  $\text{variance} = \text{mean}((x - \text{mean})^2)$ 
761 //   - 需要 2 次 block reduce: 先 mean (sum/K), 再 variance (sum((x-mean)2)/K)
762 //   - 两次 __syncthreads 必须到位, 否则 s_mean 未对所有线程可见就计算 variance
763 // Grid: (N, 1, 1), 一行一个 block
764 // Block: (128, 1, 1), NUM_THREADS=K=128
765 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
766 template <const int NUM_THREADS = 128>
767 __global__ void layer_norm(float *x, float *y, float g, float b, int N, int K) {
768     int tid = threadIdx.x;
769     int idx = blockIdx.x * blockDim.x + threadIdx.x;
770     const float epsilon = 1e-5f;
771
772     __shared__ float s_mean;
773     __shared__ float s_variance;
774     float value = (idx < N * K) ? x[idx] : 0.0f;
775
776     // Pass 1: compute mean
777     float sum = block_reduce_sum<NUM_THREADS>(value);
778     if (tid == 0)
779         s_mean = sum / (float)K;
780     __syncthreads(); // 必须等待 s_mean 对所有线程可见
781
782     // Pass 2: compute variance = (x - mean)2
783     float variance = (value - s_mean) * (value - s_mean);
784     variance = block_reduce_sum<NUM_THREADS>(variance);
785     if (tid == 0)
786         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/std
787     __syncthreads(); // 必须等待 s_variance 对所有线程可见
788
789     if (idx < N * K)
790         y[idx] = ((value - s_mean) * s_variance) * g + b;
791 }
792
793 // Layer Norm + float4
794 // Grid: (N, 1, 1)
795 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
796 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
797 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
798 template <const int NUM_THREADS = 128 / 4>
799 __global__ void layer_norm_vec4(float *x, float *y, float g, float b, int N, int K) {
800     int tid = threadIdx.x;
801     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
802     const float epsilon = 1e-5f;
803
804     __shared__ float s_mean;
805     __shared__ float s_variance;
806     float4 reg_x = FLOAT4(x[idx]);
807     float value = (idx < N * K) ? (reg_x.x + reg_x.y + reg_x.z + reg_x.w) : 0.0f;
808
809     float sum = block_reduce_sum<NUM_THREADS>(value);
810     if (tid == 0)
811         s_mean = sum / (float)K;
812     __syncthreads();
813
814     float4 reg_x_hat;
815     reg_x_hat.x = reg_x.x - s_mean;
816     reg_x_hat.y = reg_x.y - s_mean;
817     reg_x_hat.z = reg_x.z - s_mean;
818     reg_x_hat.w = reg_x.w - s_mean;

```

```

819 float variance = reg_x_hat.x * reg_x_hat.x + reg_x_hat.y * reg_x_hat.y +
820         reg_x_hat.z * reg_x_hat.z + reg_x_hat.w * reg_x_hat.w;
821 variance = block_reduce_sum<NUM_THREADS>(variance);
822 if (tid == 0)
823     s_variance = rsqrtf(variance / (float)K + epsilon);
824 __syncthreads();
825
826 float4 reg_y;
827 reg_y.x = reg_x_hat.x * s_variance * g + b;
828 reg_y.y = reg_x_hat.y * s_variance * g + b;
829 reg_y.z = reg_x_hat.z * s_variance * g + b;
830 reg_y.w = reg_x_hat.w * s_variance * g + b;
831 if (idx < N * K)
832     FLOAT4(y[idx]) = reg_y;
833 }
834
835 // =====
836 // Phase 4: RoPE — 旋转位置编码 (Rotary Position Embedding)
837 // =====
838 // 面试要点:
839 //   - RoPE 数学公式: 对每对相邻维度做 2D 旋转
840 //     [x1']   [cos(θ)  -sin(θ)] [x1]
841 //     [x2'] = [sin(θ)   cos(θ)] [x2]
842 //   - θ_i = 1 / (theta^(2i/d)), theta=10000.0f (Llama 风格)
843 //   - token_pos = idx / N: token 在序列中的位置
844 //   - token_idx = idx % N: token 内的维度索引
845 //   - 输入 [seq_len, hidden_size], 输出同形状
846
847 // Grid: ((seq_len * N + 255) / 256, 1, 1)
848 // Block: (256, 1, 1)
849 // source: LeetCUDA/kernels/rope/rope.cu
850 __global__ void rope(float *x, float *out, int seq_len, int N) {
851     int idx = blockIdx.x * blockDim.x + threadIdx.x;
852     float x1 = x[idx * 2];
853     float x2 = x[idx * 2 + 1];
854     int token_pos = idx / N; // 序列位置
855     int token_idx = idx % N; // 维度索引
856
857     // 频率计算: θ_i = 1 / (10000^(2i/d))
858     float exp_v = 1.0f / powf(10000.0f, 2 * token_idx / (N * 2.0f));
859     float sin_v = sinf(token_pos * exp_v);
860     float cos_v = cosf(token_pos * exp_v);
861
862     // 2D 旋转
863     float out1 = x1 * cos_v - x2 * sin_v;
864     float out2 = x1 * sin_v + x2 * cos_v;
865     out[idx * 2] = out1;
866     out[idx * 2 + 1] = out2;
867 }
868
869 // =====
870 // Phase 5: Mat Transpose — 矩阵转置 (Bank Conflict 专题)
871 // =====
872 // 面试要点 (Bank Conflict 专题):
873 //   - Shared memory 有 32 个 bank, 每个 bank 4 bytes (32-bit)
874 //   - 同一 warp 的多个线程访问同一 bank 的不同地址 → bank conflict
875 //   - n-way bank conflict: n 个线程冲突 → 访问串行化为 n 次
876 //   - 解决方案: PAD (在每行末尾加 1 个元素, 打破地址对齐)
877 //
878 // 转置的四步演进:
879 //   naive: 非合并写入 (列优先写) → 每个 warp 产生 32 次内存事务
880 //   shared: 写入 smem (行优先) → 从 smem 读取 (列优先) → 合并写入 gmem
881 //   BCF:   smem 布局 [WARP_SIZE_S*4][WARP_SIZE_S+PAD] = [64][17], PAD=1 加在第二维消除 bank conflict

```



```

882 // merge_write: 进一步将 4 次 separate store 合并为 1 次 float4 store
883 // 注: 本文件仅实现 Level 1(naive) 与 Level 4(BCF+merge_write), Level 2/3 省略
884
885 // 每个线程处理 1 个元素, block(16,16)
886 // 读: x 按行优先访问, warp 的 32 线程访问 32 个连续地址 → 合并读取 ✓
887 // 写: y 按列优先写入, 32 线程跨 row 行分散 → 非合并写入 x (32 次内存事务)
888 // Grid: ((col + 15) / 16, (row + 15) / 16, 1), 每线程 1 元素
889 // Block: (16, 16, 1)
890 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
891 __global__ void mat_transpose(float *x, float *y, const int row, const int col) {
892     const int c = blockIdx.x * blockDim.x + threadIdx.x; // col in x
893     const int r = blockIdx.y * blockDim.y + threadIdx.y; // row in x
894     if (r < row && c < col) {
895         // x[r][c] → y[c][r]; x stride=col, y (transposed) stride=row
896         y[c * row + r] = x[r * col + c];
897     }
898 }
899
900 // Grid: ((col + 15) / 16, (row + 63) / 64, 1), 每线程 4 元素(float4)
901 // Block: (16, 16, 1)
902 // 注意: 该版本默认按 float4 打包写回; 最适合 row 能按 4 对齐的场景
903 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
904 __global__ void mat_transpose_padded(
905     float *x, float *y, const int row, const int col) {
906     const int tx = threadIdx.x;
907     const int ty = threadIdx.y;
908
909     constexpr int TILE = 16;
910     constexpr int PAD = 1;
911     // Bank conflict fix: 每行多 1 个元素, 打破 32-bank 对齐
912     __shared__ float tile[TILE * 4][TILE + PAD]; // 64x16
913
914     // x 空间坐标; 每线程覆盖 4 行 (actual row = x_r * 4)
915     const int x_c = blockIdx.x * TILE + tx;
916     const int x_r = blockIdx.y * TILE + ty;
917
918     if (x_r * 4 < row && x_c < col) {
919         // Step 1: 4 rows × 1 col → smem (row-major);
920 #pragma unroll
921         for (int i = 0; i < 4; ++i) {
922             tile[ty * 4 + i][tx] = x[(x_r * 4 + i) * col + x_c];
923         }
924         __syncthreads();
925
926         // Step 2: Transposed read from smem
927         float4 reg_trans;
928         reg_trans.x = tile[tx * 4 + 0][ty];
929         reg_trans.y = tile[tx * 4 + 1][ty];
930         reg_trans.z = tile[tx * 4 + 2][ty];
931         reg_trans.w = tile[tx * 4 + 3][ty];
932
933         // y 空间坐标: y_r = x 的列, y_c = x 的行
934         const int y_r = blockIdx.x * TILE + ty; // = x col
935         const int y_c = (blockIdx.y * TILE + tx) * 4; // = x row
936
937         // Coalesced write: y[y_r][y_c .. y_c+3]
938         FLOAT4(y[y_r * row + y_c]) = reg_trans;
939     }
940 }
941
942 // =====
943 // Phase 6: GEMV — 矩阵向量乘 (M or N = 1, 纯 memory-bound 算子, warp-per-row 策略)
944 // =====

```



```

945 // 面试要点:
946 //   - GEMV 是典型的 memory-bound 算子:  $AI \approx O(1)$ , 瓶颈在内存带宽
947 //   - 核心策略: warp-per-row (每个 warp 负责矩阵的一行)
948 //   - 不同 K 值对应不同分块策略:
949 //       K=32 倍数: 一个 warp 的 32 线程恰好覆盖 K 维
950 //       K=128 倍数: 每个线程用 float4 处理 4 元素, warp 覆盖 128
951 //       K=16 < 32: 一个 warp 用不满, 用 ROW_PER_WARP=2 让每个 warp 处理 2 行
952 //
953 // a: MxK, x: Kx1, y: Mx1, 计算:  $y = a * x$ ; N = 1
954
955 // ---- SGEMV K32: 基础 warp-per-row ----
956 // 设计: block(32, 4), blockDim.x=WARP_SIZE=32 (K 需为 32 倍数时一轮覆盖, 否则内层循环 NUM_WARPS 次)
957 // grid(M/4), 每个 warp 负责一行
958 // K 为 32 的倍数时, warp 的 32 个线程恰好覆盖 K 维
959 // Grid: ((M + 3) / 4, 1, 1), 每 block 处理 4 行
960 // Block: (32, 4, 1), 每行 1 warp
961 // 注意: 该版本最适合 K 按 32 对齐; 当 K 更小时通常切到 K16 这类专用分支
962 // source: LeetCUDA/kernels/sgemv/sgemv.cu
963 __global__ void sgemv_k32(float *a, float *x, float *y, int M, int K) {
964     int tx = threadIdx.x; // 0~31
965     int ty = threadIdx.y; // 0~3
966     int lane = tx % WARP_SIZE; // 0~31
967     int m = blockIdx.x * blockDim.y + ty; // 全局行号
968     if (m < M) {
969         float sum = 0.0f;
970         // 沿 K 维的迭代数 = ceil(K/32), 每个 warp 要累加完整的K, 那么
971         // 每个thread就要负责累加NUM_ITERS个元素, NUM_ITERS = ceil(K/32)
972         const int NUM_ITERS = (K + WARP_SIZE - 1) / WARP_SIZE;
973 #pragma unroll
974         for (int w = 0; w < NUM_ITERS; ++w) {
975             // 假设K是32的整倍数, m * K 本行的起始地址, x: Kx1
976             int k = w * WARP_SIZE + lane;
977             sum += a[m * K + k] * x[k];
978         }
979         sum = warp_reduce_sum<WARP_SIZE>(sum);
980         // 每个 warp 处理一行, lane 0 写回结果
981         if (lane == 0)
982             y[m] = sum;
983     }
984 }
985
986 // ---- SGEMV K128: float4 向量化 ----
987 // 每个线程处理 4 个元素(float4), 一个 warp 覆盖 128 个元素
988 // Grid: ((M + 3) / 4, 1, 1)
989 // Block: (32, 4, 1)
990 // 注意: 该版本最适合 K 按 128 对齐, 且 x/a 的地址满足 float4 对齐
991 // source: LeetCUDA/kernels/sgemv/sgemv.cu
992 __global__ void sgemv_k128(float *a, float *x, float *y, int M, int K) {
993     int tx = threadIdx.x; // 0~31
994     int ty = threadIdx.y; // 0~3
995     int lane = tx % WARP_SIZE; // 0~31
996     int m = blockDim.y * blockIdx.x + ty;
997
998     if (m < M) {
999         float sum = 0.0f;
1000         // 沿 K 维的迭代数 = ceil(K/128), 每个 warp 每轮用 float4 覆盖 128 个 K 元素
1001         const int NUM_ITERS = ((K + WARP_SIZE - 1) / WARP_SIZE) + 4 - 1) / 4;
1002 #pragma unroll
1003         for (int w = 0; w < NUM_ITERS; ++w) {
1004             int k = (w * WARP_SIZE + lane) * 4;
1005             float4 reg_x = FLOAT4(x[k]);
1006             float4 reg_a = FLOAT4(a[m * K + k]);
1007             sum += (reg_a.x * reg_x.x + reg_a.y * reg_x.y + reg_a.z * reg_x.z +

```

```

1008         reg_a.w * reg_x.w);
1009     }
1010     sum = warp_reduce_sum<WARP_SIZE>(sum);
1011     if (lane == 0)
1012         y[m] = sum;
1013 }
1014 }
1015
1016 // ---- SGEMV K16: K < WarpSize, ROW_PER_WARP=2 ----
1017 // 面试亮点: K=16 < 32, 一个 warp 可以处理多行
1018 // ROW_PER_WARP=2, K_WARP_SIZE=16, 前 16 个 lane 处理 row0, 后 16 个 lane 处理 row1
1019 // Grid: ((M + 7) / 8, 1, 1), NUM_ROWS=8
1020 // Block: (32, 4, 1)
1021 // 注意: 这一版是面向 K=16 的专用写法; ROW_PER_WARP=2 时一个 warp 同时处理 2 行
1022 // source: LeetCUDA/kernels/sgemv/sgemv.cu
1023 template <const int ROW_PER_WARP = 2>
1024 __global__ void sgemv_k16(float *A, float *x, float *y, int M, int K) {
1025     constexpr int K_WARP_SIZE = (WARP_SIZE + ROW_PER_WARP - 1) / ROW_PER_WARP; // 16
1026     int tx = threadIdx.x;
1027     int ty = threadIdx.y;
1028     int lane = tx % WARP_SIZE;
1029     int k = lane % K_WARP_SIZE; // 0~15
1030     int m = (blockDim.y * blockIdx.x + ty) * ROW_PER_WARP + lane / K_WARP_SIZE;
1031     if (m < M) {
1032         float sum = A[m * K + k] * x[k];
1033         // 按照K_WARP_SIZE=16, 分2组各自做 warp reduce sum, k==0的lane写回结果
1034         sum = warp_reduce_sum<K_WARP_SIZE>(sum);
1035         // 注意: 判断条件是 k == 0, 不是 lane == 0!
1036         if (k == 0)
1037             y[m] = sum;
1038     }
1039 }
1040
1041 // =====
1042 // Phase 7: GEMM — 矩阵矩阵乘 (GPU 最重要的算子, 面试核心考点)
1043 // =====
1044 // 面试要点 (GEMM 优化五层金字塔):
1045 //   Level 1 — Tiling (分块 + shared memory): 将数据从 HBM 搬到 SMEM 复用
1046 //   Level 2 — Thread Tile (寄存器分块): 每个线程计算 TM×TN
1047 //   个元素, 提高计算密度 Level 3 — Vectorize (向量化访存): float4/half2, 减少
1048 //   load/store 指令数 Level 4 — Tensor Core (MMA
1049 //   m16n8k16): 硬件矩阵乘单元, warp 级指令 Level 5 — Warp Specialization +
1050 //   TMA (WGMMA m64n128k16): Hopper 异步执行
1051 //
1052 // 计算密度递进:
1053 //   Level 1: AI ≈ B_K / (2×sizeof) ≈ 32/8 = 4 → 仍是 memory-bound
1054 //   Level 2: AI ≈ TM×TN×B_K / (2×sizeof) ≈ 8×8×8/8 = 64 → compute-bound
1055 //   Level 4: Tensor Core 提供硬件加速的 256 FMA/cycle/warp → 大幅提升吞吐
1056 //
1057 // =====
1058 // Phase 7a: SGEMM (非 Tensor Core 路径)
1059 // =====
1060
1061 // ---- Level 1: SGEMM — Block Tile 32×32 + K Tile 32 ----
1062 // 最基础的 tiling 实现, 演示 shared memory 的核心用法
1063 // C = A × B, C[M, N] = A[M, K] × B[K, N]
1064 // BM=BN=32, BK=32, block(32, 32), 一个线程计算 c 的一个元素
1065 // Grid: ((N + 31) / 32, (M + 31) / 32, 1)
1066 // Block: (32, 32, 1), 1024 线程
1067 // source: LeetCUDA/kernels/sgemm/sgemm.cu
1068 __global__ void sgemm(float *a, float *b, float *c, int M, int N, int K) {
1069     constexpr int BM = 32; // vec 版: 32×4 = 128
1070     constexpr int BN = 32; // vec 版: 32×4 = 128

```

```

1071 constexpr int BK = 32;
1072 __shared__ float s_a[BM][BK], s_b[BK][BN]; // 32x32x4=4KB smem, float = 4 bytes
1073
1074 int bx = blockIdx.x;
1075 int by = blockIdx.y;
1076 int tx = threadIdx.x;
1077 int tid = threadIdx.y * blockDim.x + tx;
1078
1079 // 线程到 smem 的映射: 32x32 线程, 每个线程加载 a 和 b 各 1 个元素
1080 int load_smem_a_m = tid / 32; // row 0~31 由 32 线程加载;
1081 int load_smem_a_k = tid % 32; // col 0~31 由 32 线程加载;
1082 int load_smem_b_k = tid / 32; // row 0~31 由 32 线程加载;
1083 int load_smem_b_n = tid % 32; // col 0~31 由 32 线程加载;
1084 int load_gmem_a_m = by * BM + load_smem_a_m; // gmem row;
1085 int load_gmem_b_n = bx * BN + load_smem_b_n; // gmem col;
1086
1087 float sum = 0.f; // 遍历完整的K, slice K;
1088 for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1089     int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
1090     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1091     s_a[load_smem_a_m][load_smem_a_k] = a[load_gmem_a_addr];
1092     int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
1093     int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1094     s_b[load_smem_b_k][load_smem_b_n] = b[load_gmem_b_addr];
1095     __syncthreads(); // 确保整个 smem tile 加载完毕
1096
1097 #pragma unroll
1098     for (int k = 0; k < BK; ++k) {
1099         int comp_smem_a_m = load_smem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续)
1100         int comp_smem_b_n = load_smem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1101         sum += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
1102     }
1103     __syncthreads(); // 确保 smem 不会在下一轮加载时被覆盖
1104 }
1105 int store_gmem_c_m = load_gmem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续) [128x128]
1106 int store_gmem_c_n = load_gmem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1107 int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
1108 c[store_gmem_c_addr] = sum; // C [M, N] = A[M, K] x B[K, N]
1109 }
1110
1111 // ---- Level 1+: SGEMM Vec4 — Block Tile 128x128 + K Tile 32 + Thread Tile 4x4 ----
1112 // 在 Level 1 基础上引入两层优化:
1113 // 1) float4 向量化加载: A/B 各用 1 条 128-bit load 取代 4 条 32-bit load
1114 // 2) Thread Tile 4x4: 每线程计算 16 个 C 元素, 提升计算/访存比 (AI 从
1115 // BK/2≈16 提升到 TM*TN*BK/2≈256), 减少线程总数带来的同步开销
1116 // C = A x B, C[M, N] = A[M, K] x B[K, N], A/B 均 row-major
1117 // BM=BN=128, BK=32, block(32, 32)=1024 线程, 每线程负责 4x4=16 个 C 元素
1118 // 1024 x 16 = 16384 = 128 x 128 ✓
1119 //
1120 // 线程到 4x4 tile 的映射 (与加载映射解耦, 独立计算更清晰):
1121 // m_tile = tid / 32 (0~31), 每 tile 4 行 → 行 [m_tile*4, m_tile*4+3], 覆盖 0~127
1122 // n_tile = tid % 32 (0~31), 每 tile 4 列 → 列 [n_tile*4, n_tile*4+3], 覆盖 0~127
1123 //
1124 // 加载映射 (每线程 4 个元素, float4):
1125 // A[128][32]: a_m = tid/8 (8 线程/行), a_k = (tid%8)*4 (4 列/线程) → 8x4=32 列 ✓
1126 // B[32][128]: b_k = tid/32 (32 线程/行), b_n = (tid%32)*4 (4 列/线程) → 32x4=128 列 ✓
1127 // row-major 下 A[m][k..k+3] 与 B[k][n..n+3] 均连续 → float4 load 合法
1128 //
1129 // △ Bank Conflict 提示 (面试加分点):
1130 // s_b[32][128] 上 warp 内 32 线程按 stride=4 访问 (tid%32 决定列 0,4,8,...,124)
1131 // → 每 4 个线程落同一 bank 不同地址 → 4-way bank conflict。生产代码可用
1132 // s_b[BK][BN+1] PAD 打散, 这里保持最简布局便于讲解。
1133 //

```

```

1134 // Grid: ((N + 127) / 128, (M + 127) / 128, 1)
1135 // Block: (32, 32, 1), 1024 线程
1136 // 假设: M/N 为 128 的倍数, K 为 32 的倍数 (与 Level 1 naive 版一致的边界约定)
1137 // source: LeetCUDA/kernels/sgemm/sgemm.cu (vec4 variant)
1138 __global__ void sgemm_vec4(float *a, float *b, float *c, int M, int N, int K) {
1139     constexpr int BM = 128;
1140     constexpr int BN = 128;
1141     constexpr int BK = 32;
1142     __shared__ float s_a[BM][BK]; // 128*32*4 = 16KB, float = 4 bytes
1143     __shared__ float s_b[BK][BN]; // 32*128*4 = 16KB
1144
1145     int bx = blockIdx.x;
1146     int by = blockIdx.y;
1147     int tx = threadIdx.x;
1148     int tid = threadIdx.y * blockDim.x + tx; // 0~1023
1149
1150     // 加载 A: 每线程加载 s_a[a_m][a_k..a_k+3] 共 4 个元素
1151     int load_smem_a_m = tid / 8; // 0~127, 8 线程/行
1152     int load_smem_a_k = (tid % 8) * 4; // 0,4,...,28
1153     // 加载 B: 每线程加载 s_b[b_k][b_n..b_n+3] 共 4 个元素
1154     int load_smem_b_k = tid / 32; // 0~31, 32 线程/行
1155     int load_smem_b_n = (tid % 32) * 4; // 0,4,...,124
1156
1157     int load_gmem_a_m = by * BM + load_smem_a_m;
1158     int load_gmem_b_n = bx * BN + load_smem_b_n;
1159
1160     // 4x4 Thread Tile 基址 (独立于加载映射), 这里compute索引的计算逻辑要和
1161     // load索引的计算逻辑分开, load/compute是可以独立索引的, 理解这点很重要。
1162     // 目标C Tile为[BM,BN]=[128x128], 有32x32线程, 则每个线程处理4x4 tile
1163     // 那么, 就可以不重不漏地覆盖[32x4,32x4]=[128x128]的大小
1164     int comp_smem_a_m_base = (tid / 32) * 4; // 0,4,8,...,124
1165     int comp_smem_b_n_base = (tid % 32) * 4; // 0,4,8,...,124
1166
1167     float sum[4][4] = {0.f};
1168     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1169         int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
1170         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1171         FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr]); // s_a [BM,BK]
1172         int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
1173         int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1174         FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]); // s_b [BK,BN]
1175         __syncthreads();
1176
1177     #pragma unroll
1178         for (int k = 0; k < BK; ++k) {
1179             // 每次迭代加载 4 个 A 元素 + 4 个 B 元素, 再做 4x4=16 次 FMA
1180             float a_vals[4] = {s_a[comp_smem_a_m_base + 0][k],
1181                                s_a[comp_smem_a_m_base + 1][k],
1182                                s_a[comp_smem_a_m_base + 2][k],
1183                                s_a[comp_smem_a_m_base + 3][k]};
1184             float b_vals[4] = {s_b[k][comp_smem_b_n_base + 0],
1185                                s_b[k][comp_smem_b_n_base + 1],
1186                                s_b[k][comp_smem_b_n_base + 2],
1187                                s_b[k][comp_smem_b_n_base + 3]};
1188
1189             #pragma unroll
1190             for (int i = 0; i < 4; ++i) {
1191                 #pragma unroll
1192                 for (int j = 0; j < 4; ++j) {
1193                     sum[i][j] += a_vals[i] * b_vals[j];
1194                 }
1195             }
1196         }
1197     }
1198     __syncthreads();

```

```

1197 }
1198
1199 // 存储 4x4: 每行 4 个元素连续 → 可用 float4 store (要求 N 为 4 的倍数以保证对齐)
1200 int store_gmem_c_m = by * BM + comp_smem_a_m_base;
1201 int store_gmem_c_n = bx * BN + comp_smem_b_n_base;
1202 #pragma unroll
1203 for (int i = 0; i < 4; ++i) {
1204     int store_gmem_c_addr = (store_gmem_c_m + i) * N + store_gmem_c_n;
1205     float4 reg_c;
1206     reg_c.x = sum[i][0];
1207     reg_c.y = sum[i][1];
1208     reg_c.z = sum[i][2];
1209     reg_c.w = sum[i][3];
1210     FLOAT4(c[store_gmem_c_addr]) = reg_c;
1211 }
1212 }
1213
1214 // =====
1215 // Phase 7b: HGEMM — Tensor Core 路径 (MMA m16n8k16 + WGMMMA m64n128k16)
1216 // =====
1217 // 面试要点:
1218 // - MMA (Matrix Multiply-Accumulate): Ampere+ 的 Tensor Core 指令
1219 // - m16n8k16 含义: M=16, N=8, K=16 的矩阵乘, 结果 [16x8] = [16x16]x[16x8]
1220 // - ldmatrix: 从 shared memory 加载 16x16 的矩阵片段到寄存器 (4 条 32-bit
1221 // 寄存器)
1222 // - Multistage Pipeline: s2/s3/s4 个 stage, 用 cp.async 异步加载下一批数据
1223 // - Block Swizzle: 在 grid 维度做 swizzle, 改善 L2 cache locality
1224 //
1225 // ★ TN 布局详解 (面试高频考点):
1226 // TN 命名约定: 来自 BLAS 的 op(A) × op(B) 语义
1227 // BLAS 源自 Fortran, 默认列优先 (column-major) 存储
1228 // N = Normal (列优先, BLAS 原生格式)
1229 // T = Transposed (行优先, 相对 BLAS 来说是"转置过的")
1230 // 第一个字母 → A 的 op, 第二个字母 → B 的 op
1231 // 所以 TN 表示: A 是行优先 (相对 BLAS=Transposed), B 是列优先 (相对
1232 // BLAS=Normal) 即: C = op(A) × op(B) = A^T × B? 不对! 在 row-major
1233 // 视角下: TN = A row-major [MxK], B^T row-major [NxK] (等价于 B col-major [KxN]) 在 cuBLAS
1234 // 调用中: cublasGemmEx(..., CUBLAS_OP_T, CUBLAS_OP_N, ...)
1235 // T on A: BLAS 把 row-major 的 A 视为 A^T, 传 T 表示"转置回去"
1236 // N on B: B 已经是 BLAS 原生的 col-major, 无需转置
1237 //
1238 // 记忆口诀: TN = A行(T) B列(N), 第一字母 A 第二字母 B
1239 // T = row-major (行优先, 对 BLAS 来说是 transposed)
1240 // N = col-major (列优先, BLAS native = normal)
1241 //
1242 // LeetCUDA _nn 布局对比 (A/B 均 row-major 自然存储): C[MxN] = A[MxK] × B[KxN]
1243 // - A: row-major [M, K], B: row-major [K, N]
1244 // - 按 N=col-major/T=row-major 约定, 二者 BLAS 视角均为 T → cuBLAS 等效 (T,T)
1245 // - 问题: ldmatrix 默认加载 col-major, B 是 row-major 需要 .trans
1246 //
1247 // TN 布局: C[MxN] = A[MxK] × B[KxN], B 以 B^T=[NxK] row-major 存储 (A 行优先, B 列优先)
1248 // - A [MxK]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1249 // - B^T [NxK]: row-major → 全局索引 B[n*K+k] 即访问原 B 元素 (k,n) (△ 内维连续的是 K)
1250 // - 优势: B^T 已是 row-major, ldmatrix 无需 .trans, 天然匹配 MMA row.col
1251 // MMA 指令: mma.sync.aligned.m16n8k16.row.col
1252 // - row.col: A 输入 row-major, B 输入 col-major → 与 TN 布局天然匹配
1253 //
1254 // WGMMMA (Warp Group MMA, Hopper+):
1255 // - warpgroup 级指令 (128 threads = 4 warps), 异步执行
1256 // - m64n128k16: 一次处理 64x128x16 的矩阵乘 (总 tile 量 131072, 是 MMA m16n8k16 的 64 倍)
1257 // - Warp Specialization: Producer(128 threads) 做 TMA 搬运, Consumer(128
1258 // threads) 做计算
1259 // - TMA: 硬件 DMA, ~零寄存器开销, 支持 1D~5D 寻址

```

```

1260
1261 // =====
1262 // Phase 7b-1: MMA PTX 宏定义
1263 // =====
1264
1265 // ---- gmem → smem: cp.async ----
1266 // cp.async.commit_group / wait_group / wait_all 语义 (PTX ISA §9.7.9.25.3) :
1267 // - commit_group: 将此前所有未提交的 cp.async 归入一个新的 async-group (per-thread) 。
1268 // - wait_group N: 阻塞直到最多 N 个 async-group 尚未完成 (即 pending ≤ N) 。
1269 //   N=0 → 等所有 group 完成。与 wgmma.wait_group 语义一致。
1270 // - wait_all: 等价于 commit_group + wait_group 0。
1271 #define CP_ASYNC_COMMIT_GROUP() asm volatile("cp.async.commit_group;\n" ::)
1272 #define CP_ASYNC_WAIT_ALL() asm volatile("cp.async.wait_all;\n" ::)
1273 #define CP_ASYNC_WAIT_GROUP(n) \
1274     asm volatile("cp.async.wait_group %0;\n" :: "n"(n))
1275
1276 // 注意: cg 只支持 16 bytes, ca 支持 4/8/16 bytes
1277 #define CP_ASYNC_CG(dst, src, bytes) \
1278     asm volatile( \
1279         "cp.async.cg.shared.global.L2::128B [%0], [%1], %2;\n" :: "r"(dst), \
1280         "l"(src), "n"(bytes))
1281
1282 // ---- ldmatrix: smem → register (Tensor Core 专用) ----
1283 // ldmatrix.sync.aligned.xN.m8n8.shared.b16
1284 // 每次加载 8x8 的 half 矩阵片段到 1/2/4 条 32-bit 寄存器
1285 // aligned: 要求 128-bit 对齐, trans: 转置加载
1286 #define LDMATRIX_X4(R0, R1, R2, R3, addr) \
1287     asm volatile( \
1288         "ldmatrix.sync.aligned.x4.m8n8.shared.b16 {%0, %1, %2, %3}, [%4];\n" \
1289         : "=r"(R0), "=r"(R1), "=r"(R2), "=r"(R3) \
1290         : "r"(addr))
1291
1292 #define LDMATRIX_X2(R0, R1, addr) \
1293     asm volatile("ldmatrix.sync.aligned.x2.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1294         : "=r"(R0), "=r"(R1) \
1295         : "r"(addr))
1296
1297 // ldmatrix.x2.trans: 转置加载 (用于 NN 布局中需要 col-major B 矩阵的场景)
1298 // FA 中 V[Bc,d] 为 row-major, 但 P@V 的 MMA 需要 col-major 的 B → 使用 trans
1299 #define LDMATRIX_X2_T(R0, R1, addr) \
1300     asm volatile( \
1301         "ldmatrix.sync.aligned.x2.trans.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1302         : "=r"(R0), "=r"(R1) \
1303         : "r"(addr))
1304
1305 // ---- mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 ----
1306 // m16n8k16: M=16, N=8, K=16 (Ampere Tensor Core 的基本 tile)
1307 // row.col: A 是 row-major, B 是 col-major
1308 // f16.f16.f16.f16: A/B 是 f16, C/D 是 f16 (f32 累加版本用 f32.f16.f16.f32)
1309 // 2 个输出寄存器 (RD0, RD1), 4 个 A 寄存器 + 2 个 B 寄存器
1310 // C 矩阵大小 16x8=128 元素 = 128 个 half; 32 线程分担, 每线程 4 half = 2 个 uint32
1311 #define HMMA16816(RD0, RD1, RA0, RA1, RA2, RA3, RB0, RB1, RC0, RC1) \
1312     asm volatile( \
1313         "mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 {%0, %1}, {%2, %3, " \
1314         "%4, %5}, {%6, %7}, {%8, %9};\n" \
1315         : "=r"(RD0), "=r"(RD1) \
1316         : "r"(RA0), "r"(RA1), "r"(RA2), "r"(RA3), "r"(RB0), "r"(RB1), "r"(RC0), \
1317         "r"(RC1))
1318
1319 // ---- MMA 辅助函数 ----
1320 // div_ceil: 整数除法向上取整
1321 #define HOST_DEVICE_INLINE __device__ __host__ inline
1322 HOST_DEVICE_INLINE int div_ceil(int a, int b) {

```



```

1323     return (a % b != 0) ? (a / b + 1) : (a / b);
1324 }
1325
1326 // =====
1327 // Phase 7b-2: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 (统一循环版)
1328 // =====
1329 // 面试重点 — Tile Hierarchy:
1330 //   MMA Atom:          m16n8k16 (1 条 MMA 指令处理的最小 tile)
1331 //   MMA Tile (more warps): 2x4=8 个 MMA atom → [2x16, 8x4]=[32,32]
1332 //   VAL Tile (more values): 4x4=16 expand → [32x4, 32x4]=[128,128]
1333 //   实际: MMA_TILE_M=2, MMA_TILE_N=4, VAL_TILE_M=4, VAL_TILE_N=4
1334 //           → BM=16x2x4=128, BN=8x4x4=128, Warps=2x4=8, Threads=8x32=256
1335 //
1336 // ★ TN 布局 (T=A 行优先, N=B 列优先):
1337 //   - A[M][K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1338 //   - B[K][N]: col-major (等价于 B^T[N][K] row-major) → 全局索引 B[n*K+k]
1339 //   - ldmatrix A: 用 x4 (非转置), A row-major 原生匹配
1340 //   - ldmatrix B: 用 x2 (非转置), B^T row-major 逐行加载 = B 的列, 天然匹配 MMA row.col
1341 //   - MMA 指令: mma.sync.aligned.m16n8k16.row.col → 天然匹配 TN 布局
1342 //
1343 // ★ 统一循环设计 (k 从 0 开始, 消除尾端重复代码):
1344 //   - k 从 0 开始: 每次迭代 k 直接对应 tile k, sel = k % K_STAGE (零偏移)
1345 //   - 加载语义为"预取未来": 迭代 k 加载 tile (k+K_STAGE-1) 供后续使用
1346 //   - cp.async 条件化: 仅当 k+K_STAGE-1 < NUM_K_TILES 时加载
1347 //   - WAIT_GROUP 自适应: 满载期用 K_STAGE-2, 尾部排空用 0
1348 //
1349 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1350 // Block: (256, 1, 1), 8 warps
1351 // source: LeetCUDA/kernels/hgemm/mma/basic/hgemm_mma_stage_tn.cu
1352 // =====
1353 template <const int MMA_M = 16, const int MMA_N = 8, const int MMA_K = 16,
1354           const int MMA_TILE_M = 2, const int MMA_TILE_N = 4,
1355           const int VAL_TILE_M = 4, const int VAL_TILE_N = 4,
1356           const int K_STAGE = 3, const bool BLOCK_SWIZZLE = false>
1357 __global__ void __launch_bounds__(256)
1358   hgemm_mma_stages_tn(half *A, half *B, half *C, int M, int N, int K) {
1359   // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1360   const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
1361   const int by = blockIdx.y;
1362   constexpr int BM = MMA_M * MMA_TILE_M * VAL_TILE_M; // 16*2*4=128
1363   constexpr int BN = MMA_N * MMA_TILE_N * VAL_TILE_N; // 8*4*4=128
1364   constexpr int BK = MMA_K; // 16
1365
1366   // Dynamic shared memory: K_STAGE 个 stage 的 A 和 B
1367   // TN 布局: s_a[BM][BK]=[128][16] (A row-major), s_b[BN][BK]=[128][16] (B^T
1368   // row-major, 即 B col-major [KxN] 在 smem 中按 B^T[NxK] 存储)
1369   // 原始实现会按配置决定是否给 A/B 的 K 维加 PAD; 尤其 B 在 TN 布局下常见会额外
1370   // 加 B_PAD 来打散 bank 映射, 避免按列访问时出现明显 bank conflict。这里先保留最简 PAD=0 版本。
1371   extern __shared__ half smem[];
1372   half *s_a = smem;
1373   half *s_b = smem + K_STAGE * BM * BK; // A 和 B 连续存放
1374   constexpr int s_a_stage_offset = BM * BK; // 128*16
1375   constexpr int s_b_stage_offset = BN * BK; // 128*16 △ BN(128)xBK(16) = B^T row-major
1376   const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1377   const int warp_id = tid / WARP_SIZE; // 0~7
1378   const int lane_id = tid % WARP_SIZE; // 0~31
1379   // warp_m变化快(0->0,1->1), warp_n变化慢([0,1]->0,[2,3]->1,...), 因此,
1380   // 这种2x4的MMA(Warp) layout是按照col major的顺序来排列MMA0~MMA7的
1381   const int warp_m = warp_id % 2; // 0,1 (M 方向 2 个 warp)
1382   const int warp_n = warp_id / 2; // 0,1,2,3 (N 方向 4 个 warp)
1383
1384   // 线程到 global memory 的映射 (用于加载 A 和 B) 共 256 个线程
1385   // TN 布局关键: A[m*K+k] 是 row-major, B^T[n*K+k] 是 row-major (内维连续的是 K)

```

```

1386 int load_smem_a_m = tid / 2; // 0~127
1387 int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1388 int load_smem_b_n = tid / 2; // 0~127 → B^T 的 N 方向 (row-major 的行)
1389 int load_smem_b_k = (tid % 2 == 0) ? 0 : 8; // 0, 8 → B^T 的 K 方向 (row-major 的列)
1390 int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号 = M 方向的 tile 起始 + 线程偏移
1391 int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号 = N 方向的 tile 起始 + 线程偏移
1392 if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1393     return;
1394
1395 // 8个Warps(MMAs)的排布是M方向2个, N方向4个, 则MMA Atom一次性能处理的tile大小是[2*16, 4*8]=[32, 32].
1396 // CUDA中每个block能放的线程数是有上限的 (warp数量有限), 一般为4或者8个warps。为了能处理更大的C tile,
1397 // 需要把MMA Atom的tile再M方向和N方向各自重复4次, 得到[128, 128]的C block tile, 这就是Value Tile的作用。
1398 // MMA Tile对应到cutlass cute中的TiledMMA的概念, Value Tile对应到cutlass cute中的PermuteMNK的概念。
1399 uint32_t RC[VAL_TILE_M][VAL_TILE_N][2] = {0}; // 初始化为 0
1400
1401 // CVTA: 一次转换 smem 基地址, 避免每次 cp.async 都做转换
1402 uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1403 uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1404
1405 // 预加载前 (K_STAGE-1) 个 stage
1406 // TN 布局: A 的 gmem 索引用 m*K+k(row-major), B^T 用 n*K+k(row-major, 即 B col-major [K×N])
1407 #pragma unroll
1408 for (int k = 0; k < (K_STAGE - 1); ++k) {
1409     int load_gmem_a_k = k * BK + load_smem_a_k;
1410     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: [m][k]
1411     uint32_t load_smem_a_ptr = (smem_a_base_ptr +
1412         (k * s_a_stage_offset + load_smem_a_m * BK + load_smem_a_k) * sizeof(half)
1413     );
1414     CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1415
1416     int load_gmem_b_k = k * BK + load_smem_b_k;
1417     // B^T: [n][k] row-major (即 B[k][n] col-major) △
1418     int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1419     uint32_t load_smem_b_ptr = (smem_b_base_ptr +
1420         (k * s_b_stage_offset + load_smem_b_n * BK + load_smem_b_k) * sizeof(half)
1421     );
1422     CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1423
1424     CP_ASYNC_COMMIT_GROUP();
1425 }
1426
1427 CP_ASYNC_WAIT_GROUP(K_STAGE - 2); // 允许有 K_STAGE-2 个group未完成
1428 __syncthreads();
1429
1430 // 统一循环: k 从 0 开始, 每次迭代负责 tile k (加载 + 计算合并为单循环)
1431 const int NUM_K_TILES = div_ceil(K, BK);
1432 #pragma unroll
1433 for (int k = 0; k < NUM_K_TILES; ++k) {
1434     int smem_sel = k % K_STAGE; // 计算 tile k 的 stage
1435     int smem_sel_next = (k + K_STAGE - 1) % K_STAGE; // 预加载目标 stage
1436
1437     // 条件加载: 预加载 tile (k+K_STAGE-1) 供将来使用。因为在prefetch loop中已经
1438     // loaded (K_STAGE-1) 个 stage, 因此这里是从 tile (k+K_STAGE-1) 开始预加载
1439     // TN 布局: A 的 gmem 地址用 m*K+k (row-major), B^T 的 gmem 地址用 n*K+k
1440     // (row-major, 内维连续的是 K)
1441     if (k + K_STAGE - 1 < NUM_K_TILES) {
1442         int load_gmem_a_k = (k + K_STAGE - 1) * BK + load_smem_a_k;
1443         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: row-major [m][k]
1444         int load_gmem_b_k = (k + K_STAGE - 1) * BK + load_smem_b_k;
1445         int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k; // B^T: row-major [n][k]
1446
1447         uint32_t load_smem_a_ptr =
1448             (smem_a_base_ptr + (smem_sel_next * s_a_stage_offset +

```



```

1449         load_smem_a_m * BK + load_smem_a_k) *
1450         sizeof(half));
1451 CP_ASYNC.CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1452
1453 uint32_t load_smem_b_ptr =
1454     (smem_b_base_ptr + (smem_sel_next * s_b_stage_offset +
1455         load_smem_b_n * BK + load_smem_b_k) *
1456         sizeof(half));
1457 CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1458 CP_ASYNC.COMMIT_GROUP();
1459 }
1460
1461 // ldmatrix: 从 smem_sel 加载 A 和 B 到寄存器
1462 // TN 布局关键: A 用 x4 (非转置), 因为 A 是 row-major; B 用 x2 (非转置), smem 中
1463 // B^T 为 row-major, 逐行加载即得 B 的列, 天然匹配 col-major B
1464 uint32_t RA[VAL_TILE_M][4]; // [4][4], M方向4次repeat, A regs = 4 uint32_t per MMA
1465 uint32_t RB[VAL_TILE_N][2]; // [4][2], N方向4次repeat, B regs = 2 uint32_t per MMA
1466
1467 // ldmatrix.x4: 加载 A 的 m16k16 片段 (row-major A, 非转置)
1468 #pragma unroll
1469 for (int i = 0; i < VAL_TILE_M; ++i) {
1470     // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1471     // 其中 warp_m {0,1}; warp_m_0 offsets {0,16,32,48}; warp_m_1 offsets {64,80,96,112}
1472     int warp_smem_a_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;
1473     // {0,16,32,...,112} + {0~15} = {0~127}, 按照col-major的顺序访问A的4个8x8 matrix (16x16)
1474     // ldmatrix.{...}.x4.{...} 需要warp内32个线程都参与, 每个线程提供一个有效的, 不重叠的addr
1475     int lane_smem_a_m = warp_smem_a_m + lane_id % 16; // t{0...15}=0~15, t{16...31}=0~15
1476     int lane_smem_a_k = (lane_id / 16) * 8; // 0, 8, t{0...15}=0, t{16...31}=8
1477     uint32_t lane_smem_a_ptr =
1478         (smem_a_base_ptr +
1479         (smem_sel * s_a_stage_offset + lane_smem_a_m * BK + lane_smem_a_k) *
1480         sizeof(half));
1481     LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
1482 }
1483
1484 // ldmatrix.x2: 加载 B 的 k16n8 片段 (非转置)
1485 // 为什么不用 .trans? 因为 smem 中存的是 B^T row-major [N][K],
1486 // ldmatrix 逐行加载 B^T 的行 = B 的列, 天然给出 col-major B fragment → 直接匹配 MMA row.col
1487 #pragma unroll
1488 for (int j = 0; j < VAL_TILE_N; ++j) {
1489     // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1490     // warp_n_0 offsets {0,8,16,24}; warp_n_1 offsets {32,40,48,56}
1491     // warp_n_2 offsets {64,72,80,88}; warp_n_3 offsets {96,104,112,120}
1492     int warp_smem_b_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
1493     // {0,8,...,120} + {0~7} = {0~127}, 按照row-major的顺序访问B^T的2个8x8 matrix (8x16)
1494     // ldmatrix.{...}.x2.{...} 需要warp内前16个线程参与, 后16个线程传的addr会被忽略
1495     int lane_smem_b_n = warp_smem_b_n + lane_id % 8; // t{0...7}=0~7, t{8...15}=0~7
1496     int lane_smem_b_k = ((lane_id / 8) % 2) * 8; // t{0...7}=0, t{8...15}=8
1497     uint32_t lane_smem_b_ptr =
1498         (smem_b_base_ptr +
1499         (smem_sel * s_b_stage_offset + lane_smem_b_n * BK + lane_smem_b_k) *
1500         sizeof(half));
1501     LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1502 }
1503
1504 // MMA compute: 每个Warp(MMA) 在M方向重复VAL_TILE_M(4)次, 在N方向重复VAL_TILE_N(4)次
1505 #pragma unroll
1506 for (int i = 0; i < VAL_TILE_M; ++i) {
1507     #pragma unroll
1508     for (int j = 0; j < VAL_TILE_N; ++j) {
1509         HMMA16816(RC[i][j][0], RC[i][j][1], // C fragment
1510             RA[i][0], RA[i][1], RA[i][2], RA[i][3], // A fragment
1511             RB[j][0], RB[j][1], // B fragment

```

```

1512         RC[i][j][0], RC[i][j][1]);
1513     }
1514 }
1515
1516 // 自适应等待: 流水线满载期用 K_STAGE-2, 尾部排空用 0
1517 if (k + K_STAGE - 1 < NUM_K_TILES) {
1518     CP_ASYNC_WAIT_GROUP(K_STAGE - 2);
1519 } else { // 对于尾部的 k, 等待所有剩余的 cp.async 完成
1520     CP_ASYNC_WAIT_GROUP(0);
1521 }
1522 __syncthreads();
1523 }
1524
1525 // Epilogue: 寄存器 → global memory (通过 warp shuffle + 128-bit store)
1526 {
1527     for (int i = 0; i < VAL_TILE_M; ++i) {
1528         // RC[VAL_TILE_M][VAL_TILE_N][2] 中 RC[...][...][2] 中保存了2个 uint32
1529         // 寄存器, 每个uint32 寄存器表示两个临近的fp16值: {c0,c1}, 然后RC[...][...][0]
1530         // 和RC[...][...][1]代表的是按照col-major排布的2个8x8子矩阵上 (不同物理行, 跨8x8)
1531         // 同一个位置上的元素, 也就是, 实际代表了2个不同的行的元素, 因此要分开RC0, RC1;
1532         // RC0表示第一行, 8个half, 可以用4个uint32寄存器来装; 同理, RC1表示第二行。
1533         uint32_t RC0[VAL_TILE_N][4]; // 32 bits x 4 = 128 bits = 8 half
1534         uint32_t RC1[VAL_TILE_N][4]; // 32 bits x 4 = 128 bits = 8 half
1535         // =====
1536         // MMA m16n8k16 C fragment — a single 16x8 matrix (registers per warp):
1537         // Thread t holds 4 half values → RC[0] for rows 0-7, RC[1] for rows 8-15.
1538         // Mapping: row = t/4, col-pair = t%4 (each uint32 = 2 half values).
1539         //
1540         //      cols: c0-1   c2-3   c4-5   c6-7
1541         //  r0:  T0{c0,c1}  T1{c2,c3} T2{c4,c5} T3{c6,c7}  |
1542         //  r1:  T4{c0,c1}  T5{c2,c3} T6{c4,c5} T7{c6,c7}  |
1543         //  r2:  T8{c0,c1}  T9{c2,c3} T10{c4,c5} T11{c6,c7} | RC[0]
1544         //  ...
1545         //  r7:  T28{c0,c1} T29{c2,c3} T30{c4,c5} T31{c6,c7} |
1546         //  r8:  T0{c0,c1}  T1{c2,c3} T2{c4,c5} T3{c6,c7}  |
1547         //  r9:  T4{c0,c1}  T5{c2,c3} T6{c4,c5} T7{c6,c7}  |
1548         // r10: T8{c0,c1}  T9{c2,c3} T10{c4,c5} T11{c6,c7} | RC[1]
1549         //  ...
1550         // r15: T28{c0,c1} T29{c2,c3} T30{c4,c5} T31{c6,c7} |
1551         //
1552         // Within a 4-lane group (e.g. T0-T3 for row 0):
1553         // lane+0 holds {c0,c1}, lane+1 holds {c2,c3},
1554         // lane+2 holds {c4,c5}, lane+3 holds {c6,c7}.
1555         // shfl 从 lane+1/+2/+3 各收 2 个 half 到 lane+0, 4 次 shuffle 凑齐
1556         // 一行 8 个 half = {c0,c1,c2,c3,c4,c5,c6,c7}, 再由 lane%4==0 128-bit store.
1557         // =====
1558 #pragma unroll
1559         for (int j = 0; j < VAL_TILE_N; ++j) {
1560             RC0[j][0] = RC[i][j][0];
1561             RC1[j][0] = RC[i][j][1];
1562             RC0[j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
1563             RC0[j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
1564             RC0[j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
1565             RC1[j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
1566             RC1[j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
1567             RC1[j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
1568         }
1569         // 每 4 个 lane 中只有 lane 0 做 128-bit store
1570         // lane_id / 4 → 行号映射 (每 4 个连续 lane 负责上8x8矩阵和下8x8矩阵各一行):
1571         //
1572         // | lane_id | lane_id/4 | RC[0] 的行 | RC[1] 的行 |
1573         // |-----|-----|-----|-----|
1574         // | 0~3   | 0       | row 0     | row 8     |

```

```

1575 // | 4~7 | 1 | row 1 | row 9 |
1576 // | 8~11 | 2 | row 2 | row 10 |
1577 // | 12~15 | 3 | row 3 | row 11 |
1578 // | 16~19 | 4 | row 4 | row 12 |
1579 // | 20~23 | 5 | row 5 | row 13 |
1580 // | 24~27 | 6 | row 6 | row 14 |
1581 // | 28~31 | 7 | row 7 | row 15 |
1582 // |-----|
1583 if (lane_id % 4 == 0) {
1584     // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1585     int store_warp_smem_c_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;
1586     int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
1587 #pragma unroll
1588     for (int j = 0; j < VAL_TILE_N; ++j) {
1589         // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1590         int store_warp_smem_c_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
1591         int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
1592         int store_gmem_c_addr_0 = store_lane_gmem_c_m * N + store_lane_gmem_c_n;
1593         int store_gmem_c_addr_1 = (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
1594         // 128-bit store: 一次写入 8 个 half
1595         *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
1596             *reinterpret_cast<float4*>(&RC0[j][0]);
1597         *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
1598             *reinterpret_cast<float4*>(&RC1[j][0]);
1599     }
1600 }
1601 }
1602 }
1603 }
1604
1605 // =====
1606 // Phase 7b-3: HGEMM MMA Swizzle — m16n8k16 + multistage pipeline + TN 布局
1607 // + XOR swizzle 消除 smem bank conflict (统一循环版)
1608 // =====
1609 // 面试要点 (Swizzle 原理 — smem Bank Conflict 消除) :
1610 // - 问题: ldmatrix 以 8x8 片段 (m8n8) 从 smem 加载 16x16 的矩阵。同一 warp 中
1611 //   不同 lane 访问的地址在 bank 上产生冲突 → 串行化 (most common: 2/4-way) 。
1612 // - XOR swizzle 方案: 对列地址做 `((j>>3) ^ (i>>2)) % 2 << 3` 的 XOR 置换,
1613 //   将连续 4 行的 bank 映射打散。每 4 行为一组的 pattern 交替:
1614 //   rows 0~3: 列 0→地址偏移 0, 列 8→地址偏移 8
1615 //   rows 4~7: 列 0→地址偏移 8, 列 8→地址偏移 0 (XOR 翻转)
1616 //   rows 8~11: repeat rows 0~3 pattern
1617 //   rows 12~15: repeat rows 4~7 pattern
1618 // - 效果: 原来 n-way 的 bank conflict 降低到 1-way (fully conflict-free) 。
1619 // - 优势: 无需 smem PAD (不浪费空间), 原理上完全消除特定 pattern 的 bank conflict。
1620 // - 局限性: 要求 kColStride ≤ 16 (即 BK ≤ 16), kStep ∈ {4,8}。
1621 //   对 MMA m16n8k16 的 BK=16 而言刚好满足。
1622 //
1623 // 参考: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1624 // 参考: https://zhuanlan.zhihu.com/p/4746910252
1625
1626 // ---- Swizzle 辅助函数 ----
1627
1628 // swizzle_permuted_j: 对 smem 列索引 j 做 XOR 置换, 消除 bank conflict。
1629 // i: row index; j: col index.
1630 // e.g kColStride = 16, kStep = 8 -> load 8 half as 128 bits memory issue.
1631 // 公式: ((j / kStep) ^ (i / 4)) % (kColStride / kStep) * kStep
1632 // 用位运算展开 (kStep=8) : (((j >> 3) ^ (i >> 2)) % (kColStride >> 3)) << 3
1633 // 限制: kColStride ≤ 16 (BK ≤ 16), kStep ∈ {4, 8}, kColStride % kStep == 0
1634 // source: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1635 template <const int kColStride = 16, const int kStep = 8>
1636 static __device__ __forceinline__ int swizzle_permuted_j(int i, int j) {
1637     // for col_stride > 16, we have to permute it using col major ZigZag order.

```

```

1638 // e.g, A smem logical layout [Br,d]=[Br,64] -> store layout [4][Br][16].
1639 static_assert(kColStride <= 16, "kColStride must <= 16");
1640 // swizzle: ((int(j / kStep) ^ int(i / 4)) % int(kColStride / kStep)) * kStep;
1641 static_assert(kStep == 4 || kStep == 8, "kStep must be 8 or 4.");
1642 static_assert(kColStride % kStep == 0,
1643             "kColStride must be multiple of kStep.");
1644 if constexpr (kStep == 8) {
1645     return (((j >> 3) ^ (i >> 2)) % (kColStride >> 3)) << 3;
1646 } else {
1647     static_assert(kStep == 4);
1648     return (((j >> 2) ^ (i >> 2)) % (kColStride >> 2)) << 2;
1649 }
1650 }
1651
1652 // swizzle_permuted_A_j: A 矩阵专用封装 (kMmaAtomK=16, kStep=8)。
1653 // 16 行 (一个 MMA atom 的 M 维) 内的 swizzle pattern:
1654 // -----
1655 // -col 0~16, step 8--
1656 // -----
1657 // | row 0 | (0, 8) |
1658 // | row 1 | (0, 8) |
1659 // | row 2 | (0, 8) |
1660 // | row 3 | (0, 8) |
1661 // -----
1662 // | row 4 | (8, 0) |
1663 // | row 5 | (8, 0) |
1664 // | row 6 | (8, 0) |
1665 // | row 7 | (8, 0) |
1666 // -----
1667 // | row 8 | (0, 8) |
1668 // | row 9 | (0, 8) |
1669 // | row 10 | (0, 8) |
1670 // | row 11 | (0, 8) |
1671 // -----
1672 // | row 12 | (8, 0) |
1673 // | row 13 | (8, 0) |
1674 // | row 14 | (8, 0) |
1675 // | row 15 | (8, 0) |
1676 // -----
1677 // source: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1678 template <const int kMmaAtomK = 16>
1679 static __device__ __forceinline__ int swizzle_permuted_A_j(int i, int j) {
1680     return swizzle_permuted_j<kMmaAtomK, 8>(i, j);
1681 }
1682
1683 // swizzle_permuted_B_j: B 矩阵专用封装 (与 A 相同的 pattern)。
1684 // B^T smem 布局 [BN][BK]=[128][16] 在 BK 维做 swizzle,
1685 // pattern 与 A 完全相同 (kMmaAtomK=16, kStep=8)。
1686 // source: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1687 template <const int kMmaAtomK = 16>
1688 static __device__ __forceinline__ int swizzle_permuted_B_j(int i, int j) {
1689     return swizzle_permuted_j<kMmaAtomK, 8>(i, j);
1690 }
1691
1692 // =====
1693 // Phase 7b-3: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 + XOR swizzle
1694 // =====
1695 // 与 Phase 7b-2 (hgemm_mma_stages_tn) 的核心区别:
1696 // - 所有 smem 地址计算加 swizzle_permuted_A_j/swizzle_permuted_B_j, 消除 bank conflict
1697 // - VAL_TILE_M/N → VAL_TILE_M/N (与源 kernel 命名一致)
1698 // - 其余 tile hierarchy、pipeline、epilogue 与统一循环版完全一致
1699 //
1700 // Tile Hierarchy:

```

```

1701 // MMA Atom: m16n8k16 (1 条 MMA 指令处理的最小 tile)
1702 // MMA Tile (more warps): 2x4=8 个 MMA atom → [2x16, 8x4]=[32,32]
1703 // WARP Tile (more values): 4x4=16 expand → [32x4, 32x4]=[128,128]
1704 // 实际: MMA_TILE_M=2, MMA_TILE_N=4, VAL_TILE_M=4, VAL_TILE_N=4
1705 // → BM=16x2x4=128, BN=8x4x4=128, Warps=2x4=8, Threads=8x32=256
1706 //
1707 // TN 布局: C[MxN] = A[MxK] × BT[NxK]
1708 // - A[M][K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1709 // - BT[N][K]: row-major → 全局索引 B[n*K+k]
1710 // - ldmatrix A: x4 (非转置), A row-major 原生匹配
1711 // - ldmatrix B: x2 (非转置), BT row-major 逐行加载 = B 的列
1712 // - MMA 指令: mma.sync.aligned.m16n8k16.row.col → 天然匹配 TN 布局
1713 //
1714 // 统一循环设计 (同 hgemmmma_stages_tn) :
1715 // - k 从 0 开始: 每次迭代 k 直接对应 tile k, sel = k % K_STAGE (零偏移)
1716 // - 加载语义为"预取未来": 迭代 k 加载 tile (k+K_STAGE-1) 供后续使用
1717 // - cp.async 条件化: 仅当 k+K_STAGE-1 < NUM_K_TILES 时加载
1718 // - WAIT_GROUP 自适应: 满载期用 K_STAGE-2, 尾部排空用 0
1719 //
1720 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1721 // Block: (256, 1, 1), 8 warps
1722 // source: LeetCUDA/kernels/hgemmmma/swizzle/hgemmmma_stage_tn_swizzle.cu
1723 template <const int MMA_M = 16, const int MMA_N = 8, const int MMA_K = 16,
1724           const int MMA_TILE_M = 2, const int MMA_TILE_N = 4,
1725           const int VAL_TILE_M = 4, const int VAL_TILE_N = 4,
1726           const int K_STAGE = 3,
1727           const bool BLOCK_SWIZZLE = false>
1728 __global__ void __launch_bounds__(256)
1729 hgemmmma_stages_tn_swizzle(half *A, half *B, half *C, int M, int N, int K) {
1730 // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1731 const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
1732 const int by = blockIdx.y;
1733 constexpr int BM = MMA_M * MMA_TILE_M * VAL_TILE_M; // 16*2*4=128
1734 constexpr int BN = MMA_N * MMA_TILE_N * VAL_TILE_N; // 8*4*4=128
1735 constexpr int BK = MMA_K; // 16
1736
1737 // Dynamic shared memory: K_STAGE 个 stage 的 A 和 B (与 hgemmmma_stages_tn 相同)
1738 extern __shared__ half smem[];
1739 half *s_a = smem;
1740 half *s_b = smem + K_STAGE * BM * BK;
1741 constexpr int s_a_stage_offset = BM * BK; // 128*16
1742 constexpr int s_b_stage_offset = BN * BK; // 128*16 △ BT row-major
1743 const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1744 const int warp_id = tid / WARP_SIZE; // 0~7
1745 const int lane_id = tid % WARP_SIZE; // 0~31
1746 const int warp_m = warp_id % 2; // 0,1 (M 方向 2 个 warp)
1747 const int warp_n = warp_id / 2; // 0,1,2,3 (N 方向 4 个 warp)
1748
1749 // 线程到 global memory 的映射 (用于加载 A 和 B)
1750 int load_smem_a_m = tid / 2; // 0~127
1751 int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1752 int load_smem_b_n = tid / 2; // 0~127 → BT 的 N 方向 (row-major 的行)
1753 int load_smem_b_k = (tid % 2 == 0) ? 0 : 8; // 0, 8 → BT 的 K 方向 (row-major 的列)
1754 int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号
1755 int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号
1756 if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1757     return;
1758
1759 // 8个Warps(MMAs)的排布是M方向2个, N方向4个, 则MMA Atom一次性能处理的tile大小是[2*16, 4*8]=[32,32].
1760 // CUDA中每个block能放的线程数是有上限的 (warp数量有限), 一般为4或者8个warps。为了能处理更大的C tile,
1761 // 需要把MMA Atom的tile再M方向和N方向各自重复4次, 得到[128,128]的C block tile, 这就是Value Tile的作用。
1762 // MMA Tile对应到cutlass cute中的TiledMMA的概念, Value Tile对应到cutlass cute中的PermuteMNK的概念。
1763 uint32_t RC[VAL_TILE_M][VAL_TILE_N][2] = {0}; // 初始化为 0

```

```

1764
1765 // CVTA: 一次转换 smem 基地址, 避免每次 cp.async 都做转换
1766 uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1767 uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1768
1769 // 预加载前 (K_STAGE-1) 个 stage
1770 // * swizzle 区别: cp.async 的目标 smem 地址加 swizzle_permuted_A_j/swizzle_permuted_B_j
1771 #pragma unroll
1772 for (int k = 0; k < (K_STAGE - 1); ++k) {
1773     int load_gmem_a_k = k * BK + load_smem_a_k;
1774     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1775     int load_gmem_b_k = k * BK + load_smem_b_k;
1776     int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1777
1778     uint32_t load_smem_a_ptr =
1779         (smem_a_base_ptr +
1780          (k * s_a_stage_offset + load_smem_a_m * BK +
1781           swizzle_permuted_A_j<MMA_K>(load_smem_a_m, load_smem_a_k)) *
1782           sizeof(half));
1783     CP_ASYNC.CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1784
1785     uint32_t load_smem_b_ptr =
1786         (smem_b_base_ptr +
1787          (k * s_b_stage_offset + load_smem_b_n * BK +
1788           swizzle_permuted_B_j<MMA_K>(load_smem_b_n, load_smem_b_k)) *
1789           sizeof(half));
1790     CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1791
1792     CP_ASYNC.COMMIT_GROUP();
1793 }
1794
1795 CP_ASYNC.WAIT_GROUP(K_STAGE - 2); // 允许有 K_STAGE-2 个group未完成
1796 __syncthreads();
1797
1798 // 统一循环: k 从 0 开始, 每次迭代负责 tile k (加载 + 计算合并为单循环)
1799 const int NUM_K_TILES = div_ceil(K, BK);
1800 #pragma unroll
1801 for (int k = 0; k < NUM_K_TILES; ++k) {
1802     int smem_sel = k % K_STAGE; // 计算 tile k 的 stage
1803     int smem_sel_next = (k + K_STAGE - 1) % K_STAGE; // 预加载目标 stage
1804
1805     // 条件加载: 预加载 tile (k+K_STAGE-1) 供将来使用
1806     // swizzle 区别: smem 地址加 swizzle_permuted_*_j
1807     if (k + K_STAGE - 1 < NUM_K_TILES) {
1808         int load_gmem_a_k = (k + K_STAGE - 1) * BK + load_smem_a_k;
1809         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1810         int load_gmem_b_k = (k + K_STAGE - 1) * BK + load_smem_b_k;
1811         int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1812
1813         uint32_t load_smem_a_ptr =
1814             (smem_a_base_ptr +
1815              (smem_sel_next * s_a_stage_offset + load_smem_a_m * BK +
1816               swizzle_permuted_A_j<MMA_K>(load_smem_a_m, load_smem_a_k)) *
1817               sizeof(half));
1818         CP_ASYNC.CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1819
1820         uint32_t load_smem_b_ptr =
1821             (smem_b_base_ptr +
1822              (smem_sel_next * s_b_stage_offset + load_smem_b_n * BK +
1823               swizzle_permuted_B_j<MMA_K>(load_smem_b_n, load_smem_b_k)) *
1824               sizeof(half));
1825         CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1826         CP_ASYNC.COMMIT_GROUP();

```

```

1827     }
1828
1829     // ldmatrix: 从 smem_sel 加载 A 和 B 到寄存器
1830     // * swizzle 区别: ldmatrix 的源 smem 地址也加 swizzle_permuted_*_j
1831     uint32_t RA[VAL_TILE_M][4];
1832     uint32_t RB[VAL_TILE_N][2];
1833
1834     // ldmatrix.x4: 加载 A 的 m16k16 片段 (row-major A, 非转置)
1835     #pragma unroll
1836     for (int i = 0; i < VAL_TILE_M; ++i) {
1837         int warp_smem_a_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;
1838         int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
1839         int lane_smem_a_k = (lane_id / 16) * 8; // 0, 8
1840         uint32_t lane_smem_a_ptr =
1841             (smem_a_base_ptr +
1842              (smem_sel * s_a_stage_offset + lane_smem_a_m * BK +
1843               swizzle_permuted_A_j < MMA_K > (lane_smem_a_m, lane_smem_a_k)) *
1844               sizeof(half));
1845         LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
1846     }
1847
1848     // ldmatrix.x2: 加载 B 的 k16n8 片段 (非转置)
1849     #pragma unroll
1850     for (int j = 0; j < VAL_TILE_N; ++j) {
1851         int warp_smem_b_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
1852         int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
1853         int lane_smem_b_k = ((lane_id / 8) % 2) * 8; // 0, 8
1854         uint32_t lane_smem_b_ptr =
1855             (smem_b_base_ptr +
1856              (smem_sel * s_b_stage_offset + lane_smem_b_n * BK +
1857               swizzle_permuted_B_j < MMA_K > (lane_smem_b_n, lane_smem_b_k)) *
1858               sizeof(half));
1859         LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1860     }
1861
1862     // MMA compute: 每个Warp(MMA) 在M方向重复VAL_TILE_M(4)次, 在N方向重复VAL_TILE_N(4)次
1863     #pragma unroll
1864     for (int i = 0; i < VAL_TILE_M; ++i) {
1865         #pragma unroll
1866         for (int j = 0; j < VAL_TILE_N; ++j) {
1867             MMA16816(RC[i][j][0], RC[i][j][1], // C fragment
1868                     RA[i][0], RA[i][1], RA[i][2], RA[i][3], // A fragment
1869                     RB[j][0], RB[j][1], // B fragment
1870                     RC[i][j][0], RC[i][j][1]);
1871         }
1872     }
1873
1874     // 自适应等待: 流水线满载期用 K_STAGE-2, 尾部排空用 0
1875     if (k + K_STAGE - 1 < NUM_K_TILES) {
1876         CP_ASYNC_WAIT_GROUP(K_STAGE - 2);
1877     } else {
1878         CP_ASYNC_WAIT_GROUP(0);
1879     }
1880     __syncthreads();
1881 }
1882
1883 // Epilogue: 寄存器 → global memory (通过 warp shuffle + 128-bit store)
1884 // 与 hgemm_mma_stages_tn 完全一致的 epilogue 模式
1885 {
1886     for (int i = 0; i < VAL_TILE_M; ++i) {
1887         uint32_t RC0[VAL_TILE_N][4];
1888         uint32_t RC1[VAL_TILE_N][4];
1889     }
1890     #pragma unroll

```



```

1890     for (int j = 0; j < VAL_TILE_N; ++j) {
1891         RC0[j][0] = RC[i][j][0];
1892         RC1[j][0] = RC[i][j][1];
1893         RC0[j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
1894         RC0[j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
1895         RC0[j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
1896         RC1[j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
1897         RC1[j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
1898         RC1[j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
1899     }
1900     if (lane_id % 4 == 0) {
1901         int store_warp_smem_c_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;
1902         int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
1903 #pragma unroll
1904         for (int j = 0; j < VAL_TILE_N; ++j) {
1905             int store_warp_smem_c_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
1906             int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
1907             int store_gmem_c_addr_0 = store_lane_gmem_c_m * N + store_lane_gmem_c_n;
1908             int store_gmem_c_addr_1 = (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
1909             *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
1910                 *reinterpret_cast<float4*>(&RC0[j][0]);
1911             *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
1912                 *reinterpret_cast<float4*>(&RC1[j][0]);
1913         }
1914     }
1915 }
1916 }
1917 }
1918
1919 // =====
1920 // Phase 7c: HGEMM CuTe — CUTLASS CuTe DSL 实现 (SM80+, Tensor Core 全自动调度)
1921 // =====
1922 // 面试要点 (CuTe vs 手写 MMA PTX) :
1923 //   - CuTe (CUTLASS Templates) 是 NVIDIA CUTLASS 3.x 的核心 DSL, 提供编译器
1924 //     Tensor 抽象, 自动推导 MMA、Copy、Swizzle 的线程-数据映射
1925 //   - 与手写 MMA PTX (Phase 7b) 的对比:
1926 //     - 手写 MMA: 手动计算 smem 地址、手动 ldmatrix/mma PTX、手动 swizzle、手动 epilogue shuffle
1927 //     - CuTe: 声明 TiledMMA / TiledCopy / SmemLayout, 编译器自动推导线程-数据映射,
1928 //       cute::copy / cute::gemm 自动展开为高效指令序列
1929 //   - 核心抽象 ("CuTe 五要素") :
1930 //     1. Tensor: 全局/共享/寄存器数据的逻辑视图 = (ptr, shape, stride)
1931 //     2. TiledMMA: 描述 MMA 指令 + warp/warpgroup 排布 + value tile 的 compile-time type
1932 //     3. TiledCopy: 描述数据搬运 (G→S, S→R, R→S, S→G) 的线程-元素映射
1933 //     4. Layout + Swizzle: 描述 smem 的数据排布和 bank conflict 消除
1934 //     5. Pipeline: cp.async + multistage, 由 cute::copy 自动管理
1935 //
1936 // CuTe kernel 的参数推导链 (launch wrapper 负责实例化, kernel 只接收实例化后的类型) :
1937 //   MMA Atom (SM80_16x8x16_F16F16F16F16_TN)
1938 //     → TiledMMA (EURepeat=MxNxK, ValTile=MxNxK)
1939 //     → TiledCopy (G2S/S2R/R2S/S2G, 每种有 ThrLayout + ValLayout)
1940 //     → SmemLayout (Atom + Swizzle + tile_to_shape + Stage)
1941 //
1942 // 调参口诀 (面试速记) :
1943 //   BM/BN/BK 越大 → smem 越大 → occupancy 越低, 但 K 循环更少 → 指令开销更低
1944 //   Swizzle<B,M,S>: B=bank 维度 bit, M=M维 bit, S=stride bit → Swizzle<3,3,3> = 512 values = 1024B
1945 //   kStage: 2=最低延迟, 3/4=更好隐藏延迟 → 权衡 smem 占用
1946 //   EURepeat: 增加 warp 数 → 更多并行但每个 warp 做更少 MMA → 寄存器压力降低
1947 //
1948 // ★ 与手写 MMA Swizzle (Phase 7b-3) 的本质区别:
1949 //   - 手写 swizzle: 在 smem 地址计算时手动 XOR 列索引
1950 //   - CuTe Swizzle: SmemLayout 声明式指定 swizzle pattern, cute::copy 自动应用
1951 //   - CuTe 优势: 类型安全, 编译器可做更激进的优化, 布局变更只需改类型定义
1952 //

```

```

1953 // ★ 本实现的 Tile 配置:
1954 //   BM=128, BN=256, BK=32, kStage=2
1955 //   MMA: SM80_16x8x16_F16F16F16F16_TN, EURepeat=2x2x1, ValTile=32x32x16
1956 //   Smem Swizzle<3,3,3>: 512-value XOR permutation, 消除 ldmatrix bank conflict
1957 //   Threads: size(MMA{}) = 128 (4 warps × 2x2 EU repeat)
1958 //   Smem: ~49KB (Stage=2, A[128×32] + B[256×32] × half)
1959 //
1960 // 参考: LeetCUDA/kernels/hgemm/cutlass/hgemm_mma_stage_tn_cute.cu
1961 // 参考: https://zhuanlan.zhihu.com/p/671419093 (CuTe Swizzle 详解)
1962 // =====
1963
1964 #if defined(NOTES_V2_ENABLE_CUTE)
1965
1966 #include <cute/tensor.hpp>
1967
1968 // =====
1969 // Phase 7d-1: CuTe HGEMM Kernel — TN 布局 + Smem Swizzle + Multistage Pipeline
1970 // =====
1971 // TN 布局:  $C[M \times N] = A[M \times K] \times B^T[N \times K]$ 
1972 //   -  $A[M][K]$  row-major,  $B^T[N][K]$  row-major (等价  $B$  col-major  $[K \times N]$ )
1973 //   - CuTe 中通过 make_tensor(make_gmem_ptr(ptr), shape, stride) 描述
1974 //
1975 // Pipeline 流程 (每个 stage 对应一次完整的 G→S→R→MMA→R→S→G 链条):
1976 //   1. PREFETCH: 预加载 kStage-1 个 tile (G→S via cp.async)
1977 //   2. 主循环 over K tiles:
1978 //       a. S→R: cute::copy(s2r_tiled_copy, sA, rA) ← 自动展开为 ldmatrix
1979 //       b. MMA: cute::gemm(tiled_mma, rD, rA, rB, rD) ← 自动展开为 mma.sync
1980 //       c. G→S (next tile): cute::copy(g2s_tiled_copy, gA, sA) ← cp.async
1981 //       d. Pipeline wait + smem stage advance
1982 //   3. Epilogue: R→S (via UniversalCopy) → S→G (128-bit store)
1983 //
1984 // 面试对比 — 手写 MMA vs CuTe 代码量:
1985 //   手写: ldmatrix PTX 地址计算 + swizzle XOR + mma PTX + epilogue shuffle (~200行)
1986 //   CuTe: partition + copy + gemm (~80行), 其余由 launch wrapper 的类型推导完成
1987 template <typename T, int BM, int BN, int BK, int kStage, typename TiledMMA,
1988           typename G2SCopyA, typename G2SCopyB, typename SmemLayoutA,
1989           typename SmemLayoutB, typename SmemLayoutC, typename S2RCopyAtomA,
1990           typename S2RCopyAtomB, typename R2SCopyAtomC, typename S2GCopyAtomC,
1991           typename S2GCopyC>
1992 __global__ void hgemm_mma_stages_tn_cute(T *Aptr, T *Bptr, T *Dptr, int m,
1993                                           int n, int k) {
1994     using namespace cute;
1995
1996     // 动态 shared memory: A tile + B tile (C epilogue 复用 A tile 的空间)
1997     extern __shared__ T shm_data[];
1998     T *Ashm = shm_data;
1999     T *Bshm = shm_data + cute::cosize(SmemLayoutA{});
2000
2001     int idx = threadIdx.x;
2002     int ix = blockIdx.x; // N 方向 block 索引 (本实现默认不开启 BlockSwizzle)
2003     int iy = blockIdx.y; // M 方向 block 索引
2004     if (iy * BM >= m || ix * BN >= n) return;
2005
2006     // CuTe Tensor 抽象: (ptr, shape, stride) 三元组描述数据布局
2007     // make_stride(leading_dim, Int<1>{}) → row-major: 同行相邻元素步长=1, 跨行步长=leading_dim
2008     Tensor A = make_tensor(make_gmem_ptr(Aptr), make_shape(m, k),
2009                             make_stride(k, Int<1>{}));
2010     Tensor B = make_tensor(make_gmem_ptr(Bptr), make_shape(n, k),
2011                             make_stride(k, Int<1>{}));
2012     Tensor D = make_tensor(make_gmem_ptr(Dptr), make_shape(m, n),
2013                             make_stride(n, Int<1>{}));
2014
2015     // local_tile: 从全局 Tensor 切出当前 CTA 负责的 tile

```

```

2016 // make_coord(iy, _): M 维固定为 iy, K 维自动按 BK 分 tile (_ = 通配符)
2017 Tensor gA = local_tile(A, make_tile(Int<BM>{}, Int<BK>{}), make_coord(iy, _));
2018 Tensor gB = local_tile(B, make_tile(Int<BN>{}, Int<BK>{}), make_coord(ix, _));
2019 Tensor gD = local_tile(D, make_tile(Int<BM>{}, Int<BN>{}), make_coord(iy, ix));
2020
2021 // Shared memory Tensor: 按 SmemLayout 解读 smem 区域
2022 auto sA = make_tensor(make_smem_ptr(Ashm), SmemLayoutA{}); // (BM, BK, kStage)
2023 auto sB = make_tensor(make_smem_ptr(Bshm), SmemLayoutB{}); // (BN, BK, kStage)
2024
2025 // TiledMMA partition: 将 MMA 的 A/B/C tile 映射到各线程的寄存器 fragment
2026 TiledMMA tiled_mma;
2027 auto thr_mma = tiled_mma.get_slice(threadIdx.x);
2028 auto tCrA = thr_mma.partition_fragment_A(gA(_, _, 0)); // (MMA, MMA_M, MMA_K)
2029 auto tCrB = thr_mma.partition_fragment_B(gB(_, _, 0)); // (MMA, MMA_N, MMA_K)
2030 auto tCrD = thr_mma.partition_fragment_C(gD); // (MMA, MMA_M, MMA_N)
2031 cclear(tCrD); // 累加器清零
2032
2033 // G2S TiledCopy: 描述 global → shared memory 的数据搬运 (使用 cp.async 128-bit)
2034 G2SCopyA g2s_tiled_copy_a;
2035 auto g2s_thr_copy_a = g2s_tiled_copy_a.get_slice(idx);
2036 auto tAgA_copy = g2s_thr_copy_a.partition_S(gA); // (CPY, CPY_M, CPY_K, k_tile)
2037 auto tAsA_copy = g2s_thr_copy_a.partition_D(sA); // (CPY, CPY_M, CPY_K, kStage)
2038
2039 G2SCopyB g2s_tiled_copy_b;
2040 auto g2s_thr_copy_b = g2s_tiled_copy_b.get_slice(idx);
2041 auto tBgB_copy = g2s_thr_copy_b.partition_S(gB);
2042 auto tBsB_copy = g2s_thr_copy_b.partition_D(sB);
2043
2044 // S2R TiledCopy: 描述 shared → register 的数据搬运 (使用 ldmatrix)
2045 // make_tiled_copy_A/B: 根据 TiledMMA 自动推导 S2R copy 的线程-数据映射
2046 auto s2r_tiled_copy_a = make_tiled_copy_A(S2RCopyAtomA{}, tiled_mma);
2047 auto s2r_thr_copy_a = s2r_tiled_copy_a.get_slice(idx);
2048 auto tAsA = s2r_thr_copy_a.partition_S(sA); // (CPY, CPY_M, CPY_K, kStage)
2049 auto tCrA_view = s2r_thr_copy_a.retile_D(tCrA); // (CPY, CPY_M, CPY_K) — 与 rA 寄存器布局对齐
2050
2051 auto s2r_tiled_copy_b = make_tiled_copy_B(S2RCopyAtomB{}, tiled_mma);
2052 auto s2r_thr_copy_b = s2r_tiled_copy_b.get_slice(idx);
2053 auto tBsB = s2r_thr_copy_b.partition_S(sB);
2054 auto tCrB_view = s2r_thr_copy_b.retile_D(tCrB);
2055
2056 // PREFETCH: 预加载前 (kStage - 1) 个 K tile, 填满流水线
2057 int itile_to_read = 0, ismem_read = 0, ismem_write = 0;
2058 #pragma unroll
2059 for (int istage = 0; istage < kStage - 1; ++istage) {
2060     cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, istage),
2061               tAsA_copy(_, _, _, istage));
2062     cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, istage),
2063               tBsB_copy(_, _, _, istage));
2064     cp_async_fence();
2065     ++itile_to_read;
2066     ++ismem_write;
2067 }
2068 cp_async_wait<kStage - 2>();
2069 __syncthreads();
2070
2071 // 主循环: over K tiles, 每个 K tile 内再分 MMA_K 步迭代
2072 int ntile = k / BK; // K 方向总 tile 数
2073 int ik = 0;
2074 // 首轮 S→R 加载 (在进入循环前先加载第一个 ik slice)
2075 cute::copy(s2r_tiled_copy_a, tAsA(_, _, ik, ismem_read), tCrA_view(_, _, ik));
2076 cute::copy(s2r_tiled_copy_b, tBsB(_, _, ik, ismem_read), tCrB_view(_, _, ik));
2077
2078 #pragma unroll 1

```

```

2079 for (int itile = 0; itile < ntile; ++itile) {
2080     int nk = size<2>(tCrA); // 每个 K tile 内的 MMA_K 步数
2081
2082     #pragma unroll
2083     for (int ik = 0; ik < nk; ++ik) {
2084         int ik_next = (ik + 1) % nk;
2085
2086         if (ik == nk - 1) {
2087             // 等待下一批 smem 数据就绪 (pipeline 同步)
2088             cp_async_wait<kStage - 2>();
2089             __syncthreads();
2090             ismem_read = (ismem_read + 1) % kStage;
2091         }
2092
2093         // S→R: 加载下一组 A/B fragment 到寄存器 (用 ik_next 预取)
2094         cute::copy(s2r_tiled_copy_a, tAsA(_, _, ik_next, ismem_read),
2095                   tCrA_view(_, _, ik_next));
2096         cute::copy(s2r_tiled_copy_b, tBsB(_, _, ik_next, ismem_read),
2097                   tCrB_view(_, _, ik_next));
2098
2099         if (ik == 0) {
2100             // G→S: 提交下一个 K tile 的异步拷贝 (流水线加载)
2101             if (itile_to_read < ntile) {
2102                 cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, itile_to_read),
2103                           tAsA_copy(_, _, _, ismem_write));
2104                 cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, itile_to_read),
2105                           tBsB_copy(_, _, _, ismem_write));
2106                 ++itile_to_read;
2107                 ismem_write = (ismem_write + 1) % kStage;
2108             }
2109             cp_async_fence();
2110         }
2111
2112         // MMA: 发射 Tensor Core 指令 (自动展开为 mma.sync.aligned.m16n8k16)
2113         cute::gemm(tiled_mma, tCrD, tCrA(_, _, ik), tCrB(_, _, ik), tCrD);
2114     }
2115 }
2116
2117 // Epilogue: D寄存器 → shared memory → global memory
2118 // 使用 A tile 的最后一个 stage 作为 C 的 scratchpad (节省 smem)
2119 auto sC = make_tensor(sA(_, _, ismem_read).data(), SmemLayoutC{});
2120
2121 // R2S TiledCopy: 寄存器 → shared memory (使用 UniversalCopy 做类型转换)
2122 auto r2s_tiled_copy_c = make_tiled_copy_C(R2SCopyAtomC{}, tiled_mma);
2123 auto r2s_thr_copy_c = r2s_tiled_copy_c.get_slice(idx);
2124 auto tCrC_r2s = r2s_thr_copy_c.retiled_S(tCrD); // (CPY, CPY_M, CPY_N)
2125 auto tCsC_r2s = r2s_thr_copy_c.partition_D(sC); // (CPY, _1, _1, pipe)
2126
2127 // S2G TiledCopy: shared memory → global memory (128-bit store)
2128 S2GCopyC s2g_tiled_copy_c;
2129 auto s2g_thr_copy_c = s2g_tiled_copy_c.get_thread_slice(idx);
2130 auto tCsC_s2g = s2g_thr_copy_c.partition_S(sC); // (CPY, _1, _1, pipe)
2131 auto tCgC_s2g = s2g_thr_copy_c.partition_D(gD); // (CPY, CPY_M, CPY_N)
2132
2133 // group_modes: 将多维 Tensor 的某些 mode 合并, 简化遍历
2134 auto tCgC_s2gx = group_modes<1, 3>(tCgC_s2g);
2135 auto tCrC_r2sx = group_modes<1, 3>(tCrC_r2s);
2136
2137 int step = size<3>(tCsC_r2s); // pipeline depth of C smem
2138 #pragma unroll
2139 for (int i = 0; i < size<1>(tCrC_r2sx); i += step) {
2140     // R→S: 寄存器写回 shared memory
2141     #pragma unroll

```

```

2142     for (int j = 0; j < step; ++j) {
2143         auto t = make_tensor_like<T>(tCrC_r2sx(_, i + j));
2144         cute::copy(tCrC_r2sx(_, i + j), t);
2145         cute::copy(r2s_tiled_copy_c, t, tCsC_r2s(_, 0, 0, j));
2146     }
2147     __syncthreads();
2148
2149     // S→G: shared memory 写回 global memory
2150 #pragma unroll
2151     for (int j = 0; j < step; ++j) {
2152         cute::copy(s2g_tiled_copy_c, tCsC_s2g(_, 0, 0, j), tCgC_s2gx(_, i + j));
2153     }
2154     __syncthreads();
2155 }
2156 }
2157
2158 // =====
2159 // Phase 7c-2: CuTe HGEMM Launch Wrapper — 类型实例化 + Grid/Block 配置
2160 // =====
2161 // CuTe 的核心设计理念: "类型即配置" (Type-level configuration)
2162 // 所有的 MMA atom、TiledCopy、SmemLayout、Swizzle 都是 **编译期类型**,
2163 // kernel 接收这些类型的实例 (通常为空 struct), 在编译期完成全部映射推导。
2164 //
2165 // 面试常问: 「CuTe 为什么比手写 PTX 简洁? 」
2166 // 答: 手写需要:
2167 //   1) 手动计算每线程的 smem 地址偏移
2168 //   2) 手动计算 ldmatrix 的 lane 映射
2169 //   3) 手动插入 swizzle XOR 到每个地址计算点
2170 //   4) 手动做 epilogue 的 warp shuffle 编排
2171 // CuTe 的 partition + copy + gemm 通过 TiledCopy/TiledMMA 的类型信息
2172 // 在编译期自动完成上述全部推导, 生成与手写等价的 PTX 指令序列。
2173 //
2174 // 模板参数推导链 (面试速记):
2175 //   SM80_16x8x16_F16F16F16F16_TN ← MMA 指令 atom
2176 //   → MMA_Atom<MMA_Traits<mma_op>>
2177 //   → make_tiled_mma(mma_atom, EURepeat{2,2,1}, ValTile{32,32,16})
2178 //   → TiledMMA (128 threads, 4 warps × 2×2 slices)
2179 //
2180 //   SM80_CP_ASYNC_CACHEGLOBAL<uint128_t> ← G→S copy atom
2181 //   → Copy_Atom<Copy_Traits<g2s_copy_op>, T>
2182 //   → make_tiled_copy(copy_atom, ThrLayout{32,4}, ValLayout{1,8})
2183 //   → G2SCopy: 32×4=128 threads, each copies 1×8=8 halves = 128 bits
2184 //
2185 //   Swizzle<3,3,3> + Atom{8,BK} → composition → tile_to_shape{BM,BK,kStage}
2186 //   → SmemLayoutA/B: 带 XOR swizzle 的 smem 排布
2187 //
2188 //   SM75_U32x4_LDSTM_N ← S→R copy atom (ldmatrix 的 CuTe 封装)
2189 //   → Copy_Atom<Copy_Traits<s2r_copy_op>, T>
2190 //
2191 //   UniversalCopy<uint128_t> ← S→G copy atom (128-bit store)
2192 //   → make_tiled_copy(copy_atom, ThrLayout{32,4}, ValLayout{1,8})
2193 //
2194 // ★ Tile 尺寸速查:
2195 //   BM=128: M 方向 block tile (16 × 2 EU × 4 Val = 128)
2196 //   BN=256: N 方向 block tile (8 × 2 EU × 16 Val... etc. 实际 8 × 2 × 16 = 256)
2197 //   BK=32: K 方向 block tile (= kMmaPK = 1 × 1 × 16 = 16? 不对, BK=32 由 SmemLayout 控制)
2198 //   kStage=2: 双缓冲 pipeline
2199 // =====
2200 template <typename T, const int Stages = 2>
2201 void launch_hgemm_mma_stages_tn_cute(T *a, T *b, T *c, int M, int N, int K) {
2202     using namespace cute;
2203
2204     auto BM = Int<128>{};

```

```

2205 auto BN = Int<256>{};
2206 auto BK = Int<32>{};
2207 auto KStage = Int<Stages>{};
2208 auto kSmemLayoutCBatch = Int<4>{}; // C smem pipeline depth
2209
2210 // SmemLayout: Swizzle<3,3,3> 做 512-value (1024-byte) XOR 置换消除 bank conflict
2211 // composition: 将 swizzle pattern 应用到 8×BK 的 base layout
2212 // tile_to_shape: 将 atom layout 扩展到完整的 BM×BK×kStage
2213 using SmemLayoutAtom = decltype(composition(
2214     Swizzle<3, 3, 3>{},
2215     make_layout(make_shape(Int<8>{}, Int<BK>{}),
2216         make_stride(Int<BK>{}, Int<1>{}))));
2217 using SmemLayoutA = decltype(tile_to_shape(
2218     SmemLayoutAtom{}, make_shape(Int<BM>{}, Int<BK>{}, Int<KStage>{})));
2219 using SmemLayoutB = decltype(tile_to_shape(
2220     SmemLayoutAtom{}, make_shape(Int<BN>{}, Int<BK>{}, Int<KStage>{})));
2221
2222 // MMA: SM80_16x8x16_F16F16F16F16_TN
2223 // TN = A row-major, B col-major (与 Phase 7b 手写 MMA 完全相同的布局约定)
2224 using mma_op = SM80_16x8x16_F16F16F16F16_TN;
2225 using mma_traits = MMA_Traits<mma_op>;
2226 using mma_atom = MMA_Atom<mma_traits>;
2227 using mma_atom_shape = mma_traits::Shape_MNK; // (Int<16>, Int<8>, Int<16>)
2228
2229 static constexpr int kMmaEURepeatM = 2;
2230 static constexpr int kMmaEURepeatN = 2;
2231 static constexpr int kMmaEURepeatK = 1;
2232 static constexpr int kMmaPM = 1 * kMmaEURepeatM * get<0>(mma_atom_shape{}); // 32
2233 static constexpr int kMmaPN = 2 * kMmaEURepeatN * get<1>(mma_atom_shape{}); // 32
2234 static constexpr int kMmaPK = 1 * kMmaEURepeatK * get<2>(mma_atom_shape{}); // 16
2235
2236 using MMA_EU_RepeatT = decltype(make_layout(make_shape(
2237     Int<kMmaEURepeatM>{}, Int<kMmaEURepeatN>{}, Int<kMmaEURepeatK>{})));
2238 using MMA_P_T = Tile<Int<kMmaPM>, Int<kMmaPN>, Int<kMmaPK>>;
2239 using MMA = decltype(make_tiled_mma(mma_atom{}, MMA_EU_RepeatT{}, MMA_P_T{}));
2240
2241 // G2S Copy: cp.async 128-bit, 32×4 threads each loading 1×8 halves
2242 using g2s_copy_op = SM80_CP_ASYNC_CACHEGLOBAL<cute::uint128_t>;
2243 using g2s_copy_traits = Copy_Traits<g2s_copy_op>;
2244 using g2s_copy_atom = Copy_Atom<g2s_copy_traits, T>;
2245 using G2SCopyA = decltype(make_tiled_copy(
2246     g2s_copy_atom{},
2247     make_layout(make_shape(Int<32>{}, Int<4>{}),
2248         make_stride(Int<4>{}, Int<1>{})),
2249     make_layout(make_shape(Int<1>{}, Int<8>{}))));
2250 using G2SCopyB = G2SCopyA;
2251
2252 // S2R Copy: ldmatrix (SM75_U32x4_LDSM_N), 由 make_tiled_copy_A/B 自动与 TiledMMA 对齐
2253 using s2r_copy_op = SM75_U32x4_LDSM_N;
2254 using s2r_copy_traits = Copy_Traits<s2r_copy_op>;
2255 using s2r_copy_atom = Copy_Atom<s2r_copy_traits, T>;
2256 using S2RCopyAtomA = s2r_copy_atom;
2257 using S2RCopyAtomB = s2r_copy_atom;
2258
2259 // Epilogue smem layout for C: Swizzle<3,3,3> on 32×32 atom, 4 pipeline stages
2260 using SmemLayoutAtomC = decltype(composition(
2261     Swizzle<3, 3, 3>{},
2262     make_layout(make_shape(Int<kMmaPM>{}, Int<kMmaPN>{}),
2263         make_stride(Int<kMmaPN>{}, Int<1>{}))));
2264 using SmemLayoutC = decltype(tile_to_shape(
2265     SmemLayoutAtomC{},
2266     make_shape(Int<kMmaPM>{}, Int<kMmaPN>{}, Int<kSmemLayoutCBatch>{})));
2267

```



```

2268 // R2S Copy: 寄存器 → shared memory (epilogue 阶段, UniversalCopy<int> 做逐 int 拷贝)
2269 using R2SCopyAtomC = Copy_Atom<UniversalCopy<int>, T>;
2270
2271 // S2G Copy: shared memory → global (128-bit store)
2272 using S2GCopyAtomC = Copy_Atom<UniversalCopy<cute::uint128_t>, T>;
2273 using S2GCopyC = decltype(make_tiled_copy(
2274     S2GCopyAtomC{},
2275     make_layout(make_shape(Int<32>{}, Int<4>{}),
2276         make_stride(Int<4>{}, Int<1>{})),
2277     make_layout(make_shape(Int<1>{}, Int<8>{}))));
2278
2279 // Grid/Block 配置
2280 int BX = (N + BN - 1) / BN;
2281 int BY = (M + BM - 1) / BM;
2282 dim3 block(size(MMA{})); // 128 threads
2283 dim3 grid(BX, BY);
2284
2285 // Smem 大小: A tile + B tile (C epilogue 复用 A tile 空间)
2286 static constexpr int shm_size_AB =
2287     cute::cosize(SmemLayoutA{}) + cute::cosize(SmemLayoutB{});
2288 static constexpr int shm_size_C = cute::cosize(SmemLayoutC{});
2289 // C smem ≤ A 的一个 pipe, 可以安全复用
2290 static_assert(size<0>(SmemLayoutA{}) * size<1>(SmemLayoutA{}) >= size(SmemLayoutC{}),
2291     "C shared memory must fit within one A pipe");
2292 static constexpr int kShmSize =
2293     cute::max(shm_size_AB, shm_size_C) * sizeof(T);
2294
2295 cudaFuncSetAttribute(
2296     hgemm_mma_stages_tn_cute<T, BM, BN, BK, KStage, MMA, G2SCopyA, G2SCopyB,
2297         SmemLayoutA, SmemLayoutB, SmemLayoutC,
2298         S2RCopyAtomA, S2RCopyAtomB, R2SCopyAtomC,
2299         S2GCopyAtomC, S2GCopyC>,
2300     cudaFuncAttributeMaxDynamicSharedMemorySize, kShmSize);
2301
2302 hgemm_mma_stages_tn_cute<T, BM, BN, BK, KStage, MMA, G2SCopyA, G2SCopyB,
2303     SmemLayoutA, SmemLayoutB, SmemLayoutC, S2RCopyAtomA,
2304     S2RCopyAtomB, R2SCopyAtomC, S2GCopyAtomC, S2GCopyC>
2305 <<<grid, block, kShmSize>>>(a, b, c, M, N, K);
2306 }
2307
2308 #endif /* NOTES_V2_ENABLE_CUTE */
2309
2310 // =====
2311 // Phase 7d: HGEMM WGMMMA — m64n128k16 + TMA + Warp Specialization (Hopper)
2312 // =====
2313 // 面试要点 (WGMMMA vs MMA 对比):
2314 // - MMA: warp 级 (32 threads), 同步执行
2315 // - WGMMMA: warpgroup 级 (128 threads = 4 warps), 异步执行 (fire-and-forget)
2316 // - m64n128k16: M=64, N=128, K=16 → 一次处理 64×128×16=131K 个乘加 (MMA m16n8k16的 4×16=64倍)
2317 // - TMA (Tensor Memory Accelerator): 硬件 DMA, 2D 寻址, 零寄存器开销
2318 // - Warp Specialization: Producer 做 TMA, Consumer 做 WGMMMA, 通过 barrier 同步
2319 // - 128B swizzle: shared memory 的 128B swizzle 模式, 避免 bank conflict
2320
2321 // ---- WGMMMA 辅助函数 ----
2322 // wgmma.commit_group / wait_group 语义 (PTX ISA §9.7.15.7):
2323 // - commit_group: 将此前所有未提交的 wgmma.mma_async 归入一个新的 wgmma-group
2324 //   (per-warpgroup, 故需 .sync.aligned 确保所有线程同步执行)。
2325 // - wait_group N: 阻塞直到最多 N 个 wgmma-group 尚未完成 (pending ≤ N)。
2326 //   N=0 → 等所有 group 完成。★ 与 cp.async.wait_group 语义一致 (PTX ISA §9.7.9.25.3.3),
2327 //   都是 "wait until only N or fewer groups are pending"。
2328 #define WGMMMA_FENCE() asm volatile("wgmma.fence.sync.aligned;\n" ::: "memory")
2329 #define WGMMMA_COMMIT_GROUP() \
2330     asm volatile("wgmma.commit_group.sync.aligned;\n" ::: "memory")

```



```

2331 #define WGMMMA_WAIT_GROUP(n) \
2332     asm volatile("wgmma.wait_group.sync.aligned %0;\n" :: "n"(n) : "memory")
2333
2334 // SMEM_DESC_ENCODE: 将 byte 偏移编码为 WGMMMA descriptor 位域中的 14-bit 值。
2335 // 编码规则 (PTX ISA §9.7.15.5.1.2.2) :
2336 //     encoded = (x & 0x3FFFF) >> 4
2337 // 其中 0x3FFFF = 2^18 - 1 = 262143, 只保留低 18 bit。
2338 // >>4 是因为 shared memory 地址/偏移必须 16B 对齐 (2^4), 低 4 bit 恒为 0,
2339 // 可省略以扩大可表示范围。
2340 // 解码: decoded = encoded << 4 (回到原始 byte 值)。
2341 #define SMEM_DESC_ENCODE(x) (((uint64_t)(x)) & 0x3FFFF) >> 0x4
2342
2343 // make_smem_desc: 构造 WGMMMA (warpgroup MMA) 的 64-bit shared memory 矩阵描述符。
2344 // 硬件 WGMMMA (如 wgmma.mma_async.m64n128k16) 需要 descA/descB 两个 64-bit 描述符,
2345 // 来定位 shared memory 中的 A/B tile 并告知 swizzle 模式。
2346 //
2347 // 64-bit descriptor 位域布局 (参见 PTX ISA §9.7.15.5.1.2.2 & CUTLASS GmmaDescriptor) :
2348 // -----
2349 // | 位域           | 范围       | 大小 | 含义
2350 // |-----|-----|-----|-----
2351 // | start_address   | [0, 14)    | 14   | smem 基址编码
2352 // | (unused)        | [14, 16)   | 2    | -
2353 // | leading_byte_offset | [16, 30)   | 14   | 主要维度 (K) 的字节步长; swizzle 下硬件未使用 (assumed=1)
2354 // | (unused)        | [30, 32)   | 2    | -
2355 // | stride_byte_offset | [32, 46)   | 14   | 跨越维度 (M/N) 的字节步长: K-Major 下为 8-row stripe 间距
2356 // | (unused)        | [46, 49)   | 3    | -
2357 // | base_offset     | [49, 52)   | 3    | swizzle pattern 偏移
2358 // | (unused)        | [52, 62)   | 10   | -
2359 // | layout_type     | [62, 64)   | 2    | swizzle 模式
2360 // -----
2361 // layout_type: 0=None, 1=128B swizzle, 2=64B swizzle, 3=32B swizzle
2362 //
2363 // K-Major + 128B swizzle + half(16-bit) 的几何推导 (PTX ISA §9.7.15.5.1.2) :
2364 // - 128B swizzle atom 在 128-bit 单位下为 8x8
2365 // - 归一化到 half(2B) 后: atom 覆盖 8x(128/16)=64 个 K 方向元素 × 8 个 M 方向元素
2366 // - 即每个 atom = 64(K) × 8(M) 个 half = 64x8x2 = 1024 bytes
2367 // - 这是 stride_byte_offset=1024 的来源
2368 //
2369 // - leading_byte_offset 在 K-Major swizzle 下 unused (hardware assumes 1),
2370 //   此处填 16 仅为占位, 编码后为 1。
2371 //
2372 // - base_offset=0 (bits [49,52) 未显式置位), 表示 smem ptr 已 1024B 对齐。
2373 //   若未对齐需计算 (pattern_start_addr >> 7) & 0x7。
2374 //
2375 // 参考: CUTLASS GmmaDescriptor (cute/arch/mma_sm90_desc.hpp)
2376 // 注: descA/descB 共用此函数; 对 B 而言物理存储为 [BN, BK], 详见 WgmmaSMem 注释。
2377 __device__ inline uint64_t make_smem_desc(half *ptr) {
2378     // __cvta_generic_to_shared: 将通用地址空间中的指针转换为 shared memory 地址
2379     // (一个 32-bit 的 smem byte 偏移量, 相对于当前 CTA 的 shared memory 基址)。
2380     uint32_t addr = static_cast<uint32_t>(__cvta_generic_to_shared(ptr));
2381
2382     // 从全零开始: 所有位域初始化为 0。
2383     uint64_t desc = 0x0000000000000000;
2384
2385     // — bits [0, 14): start_address —
2386     // SMEM_DESC_ENCODE(addr) 将 addr 右移 4 位 (丢弃低 4 个 0 bit),
2387     // 只保留 14 位。硬件解码时左移 4 位恢复完整的 16B-aligned 地址。
2388     // 可寻址范围: 2^(14+4) = 2^18 = 256K 个 half = 512 KB 共享内存。
2389     desc |= SMEM_DESC_ENCODE(addr);
2390
2391     // — bits [16, 30): leading_byte_offset —
2392     // 原始值 16 (字节), 编码后为 (16 & 0x3FFFF) >> 4 = 1。
2393     // << 16 将编码值放到 bits [16, 30)。

```

```

2394 // K-Major + 128B swizzle 下该字段硬件未使用 (assumed to be 1, 参见 PTX ISA §9.7.15.5.1.2.1.1) ,
2395 // 此处填 16 仅为占位, 使编码值为 1 满足硬件预期。
2396 // 若为 MN-Major 或 INTERLEAVE 布局, LBO 有实际含义:
2397 // 对 INTERLEAVE: 同一 8×2 brick 内第一列到第二列的字节偏移。
2398 // 对 MN-Major swizzle: 偏移从首 (swizzle-byte-size/16) 行到下一组行。
2399 desc |= SMEM_DESC_ENCODE((uint64_t)16) << 16;
2400
2401 // — bits [32, 46): stride_byte_offset —
2402 // 原始值 1024 (字节), 编码后为 (1024 & 0x3FFFF) >> 4 = 64。
2403 // << 32 将编码值放到 bits [32, 46)。
2404 // K-Major 下定义为"从首 8 行到次 8 行的字节偏移" (PTX ISA §9.7.15.5.1.2.1.2) 。
2405 // 1024 的来源 (K-Major + 128B swizzle + half) :
2406 // 一个 128B swizzle atom 覆盖 64(K) × 8(M) 个 half。
2407 // 从 1 个 8-row stripe 到下一个 stripe 需要跨越 8 × 64 × 2 = 1024 bytes。
2408 // 若为 MN-Major: SBO 是从首列到下一列的偏移 (8 列一组) 。
2409 desc |= SMEM_DESC_ENCODE((uint64_t)1024) << 32;
2410
2411 // — bits [62, 64): layout_type (swizzle mode) —
2412 // 1llu << 62 = 二进制 01 在 bits [62, 64) = layout_type = 1。
2413 // 1 = 128B swizzle mode。
2414 // 其他取值: 0=None, 2=64B swizzle, 3=32B swizzle。
2415 // 128B swizzle 将 smem 中连续的行按 128B 粒度进行 XOR 重映射,
2416 // 确保同一 warp 内各线程访问不同 bank 的同一偏移时不会产生 bank conflict。
2417 desc |= 1llu << 62;
2418
2419 return desc;
2420 }
2421
2422 // ---- WGMMA PTX 指令宏 ----
2423 // wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16
2424 // m64n128k16: M=64, N=128, K=16。一次 WGMMA 计算 D[64×128] += A[64×16] × B[16×128]。
2425 // f16.f16.f16: A=f16, B=f16, D=f16 (累加器也是 f16 精度) 。
2426 // 每线程 32 个 uint32 输出: D=64×128=8192 half, warpgroup 128 线程分担,
2427 // 每线程 64 half = 32 uint32。
2428 // descA/descB: shared memory 描述符 (由 make_smem_desc 生成, 64-bit 编码) 。
2429 // ScaleD: 0=清零累加器再写入, 1=与累加器中的旧值累加 (K 维迭代必须用 1) 。
2430 // ScaleA/ScaleB: 1=正常符号, -1=翻转符号 (此处始终用 1) 。
2431 // TransA/TransB: 0=K-major (K 维连续), 1=MN-major (M/N 维连续) 。本实现始终用 0。
2432 // 参考: PTX ISA §9.7.15.4 (wgmma.mma_async)
2433 #define WGMMA_M64N128K16_F16F16F16(d, sA, sB, ScaleD, ScaleA, ScaleB, TransA, \
2434                                     TransB) \
2435 { \
2436     uint64_t desc_a = make_smem_desc(&(sA)[0]); \
2437     uint64_t desc_b = make_smem_desc(&(sB)[0]); \
2438     asm volatile( \
2439         "{\n" \
2440         "wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16 " \
2441         "{%0, %1, %2, %3, %4, %5, %6, %7, " \
2442         "%8, %9, %10, %11, %12, %13, %14, %15, " \
2443         "%16, %17, %18, %19, %20, %21, %22, %23, " \
2444         "%24, %25, %26, %27, %28, %29, %30, %31}, " \
2445         "%32, " \
2446         "%33, " \
2447         "%34, %35, %36, %37, %38;\n" \
2448         "}\n" \
2449         : "+r"((d)[0][0]), "+r"((d)[0][1]), "+r"((d)[0][2]), "+r"((d)[0][3]), \
2450           "+r"((d)[1][0]), "+r"((d)[1][1]), "+r"((d)[1][2]), "+r"((d)[1][3]), \
2451           "+r"((d)[2][0]), "+r"((d)[2][1]), "+r"((d)[2][2]), "+r"((d)[2][3]), \
2452           "+r"((d)[3][0]), "+r"((d)[3][1]), "+r"((d)[3][2]), "+r"((d)[3][3]), \
2453           "+r"((d)[4][0]), "+r"((d)[4][1]), "+r"((d)[4][2]), "+r"((d)[4][3]), \
2454           "+r"((d)[5][0]), "+r"((d)[5][1]), "+r"((d)[5][2]), "+r"((d)[5][3]), \
2455           "+r"((d)[6][0]), "+r"((d)[6][1]), "+r"((d)[6][2]), "+r"((d)[6][3]), \
2456           "+r"((d)[7][0]), "+r"((d)[7][1]), "+r"((d)[7][2]), "+r"((d)[7][3]) \

```

```

2457         : "l"(desc_a), "l"(desc_b), "n"(int32_t(Scaled)), \
2458         "n"(int32_t(ScaleA)), "n"(int32_t(ScaleB)), "n"(int32_t(TransA)), \
2459         "n"(int32_t(TransB))); \
2460     }
2461
2462 // ---- TMA Shared Memory Layout ----
2463 // Multi-stage pipeline: K_STAGE 个 stage, 每个 stage 存储 A[BM×BK] + B[BK×BN]
2464 //
2465 // 每个 stage 包含两块 smem:
2466 //   A tile: [BM, BK] row-major → BK×BM 个 half, 地址连续
2467 //   B tile: TMA 按 [BN, BK] 物理写入 (详见下方 ★), BN×BK 个 half
2468 //
2469 // ★ 关键区别 — WGMMA 的 B 物理存储 vs 逻辑视图:
2470 //
2471 //   上层数据: 源矩阵 BT [N, K] row-major (TN 布局, B 的列以行优先形式存储)。
2472 //   物理存储: TMA 将 BT 的 tile 写入 smem, 物理布局为 [BN, BK] row-major
2473 //               (BN=128 行, BK=64 列, 每行 64 个连续的 half)。
2474 //   TMA 的 smem_box_shape = (BK=64, BN=128) 定义了每个 tile 的 shape,
2475 //   TMA 按行写入, 行内 BK 个 half 连续 → 物理上就是 [BN][BK]。
2476 //   逻辑视图: WGMMA 指令 (wgmma.mma_async.m64n128k16, PTX ISA §9.7.15.4)
2477 //   通过 imm-trans-b=0 指定 B 为 K-major, 即逻辑上按 [BK, BN] 寻址。
2478 //   实际 [BN, BK] → 逻辑 [BK, BN] 的"转置"是通过 descB 中的 128B
2479 //   swizzle (layout_type=1) 在硬件地址重映射层完成的, 无需软件转置。
2480 //
2481 //   stride_byte_offset=1024 的来源 (K-major + 128B swizzle + f16, PTX ISA §9.7.15.5.1.2.1.2) :
2482 //   128B swizzle atom = 8(N) × 8(K) × 128-bit = 8 rows × 64 个 half。
2483 //   stride_byte_offset = 从当前 8-row stripe 到下一个 8-row stripe 的字节偏移
2484 //   = 8 rows × (BK=64) halves × 2 bytes = 1024。
2485 //   这个值恰好等于物理 [BN, BK] 布局中连续 8 行的跨度 ← 进一步证实物理存储是 [BN, BK]。
2486 //
2487 //   与 MMA(TN) 的对比:
2488 //   MMA (TN): smem 存 BT [N, K] row-major, ldmatrix 按列解出 col-major B fragment。
2489 //   WGMMA: smem 物理存 [BN, BK] (TMA 直写), WGMMA 通过 swizzle 硬件以 K-major [BK, BN] 逻辑视图读取。
2490 //   简言之: swizzle 桥接了 "物理 row-major [BN, BK]" 和 "逻辑 K-major [BK, BN]" 的差异。
2491 template <int BM, int BN, int BK, int QSIZE> struct WgmmaSMem {
2492     alignas(128) half A[BM * BK * QSIZE]; // A tile: row-major [BM, BK]
2493     // B tile 注记:
2494     //   - 物理存储: TMA 从 BT[N,K] row-major 搬入, 物理上是 [BN, BK] row-major (BN 行, BK 列)
2495     //   - 逻辑视图: WGMMA 通过 descB 的 128B swizzle (PTX ISA §9.7.15.5.1.2) 将地址重映射,
2496     //     以 K-major [BK, BN] 逻辑视图供 wgmma.mma_async (imm-trans-b=0, PTX ISA §9.7.15.4) 读取
2497     //   - B[BN * BK * QSIZE] 与 B[BK * BN * QSIZE] 数值等价 (BN×BK = BK×BN),
2498     //     但写 BN×BK 更能体现物理存储形态
2499     alignas(128) half B[BN * BK * QSIZE];
2500 };
2501
2502 // ---- WGMMA Kernel: Warp Specialization + TMA ----
2503 // 面试重点 — Warp Specialization (Hopper 最核心的编程模型变化):
2504 //
2505 // 背景: 传统 GEMM kernel 中, 所有线程同步地"加载→计算→存回",
2506 // 数据搬运和计算无法重叠。Hopper 的 TMA + WGMMA 支持将工作拆分为两个
2507 // warpgroup, 让数据搬运和矩阵乘完全异步执行:
2508 //
2509 //   WG0 (128 threads, Producer): 仅 thread 0 提交 TMA 2D 拷贝指令,
2510 //   将 A/B tile 从 HBM 搬到 shared memory。
2511 //   WG1 (128 threads, Consumer): 所有 128 个线程参与 WGMMA 矩阵乘。
2512 //
2513 // Producer 和 Consumer 通过 cuda::barrier (CTA 级别) 同步:
2514 //   - full[qidx]: Producer 发信号表示 stage qidx 的数据已就绪, 可被使用
2515 //   - empty[qidx]: Consumer 发信号表示 stage qidx 的使用完毕, 可以被覆盖
2516 //
2517 // 本节重点理解:
2518 //   1) 为什么 Producer 只需要 thread 0? TMA 是硬件 DMA 指令, 一次提交
2519 //     即可搬运整个 2D tile, 无需所有线程参与。

```

```

2520 // 2) barrier 的 arrive count = 129: 128 个 Consumer 线程 + 1 个 Producer 提交线程。
2521 // 3) 多 stage pipeline 使 Consumer 计算 stage qidx 的同时,
2522 //     Producer 可以搬运 stage (qidx+1), 隐藏 HBM→SMEM 延迟。
2523 //
2524 // Tile Hierarchy (与 MMA m16n8k16 kernel 对比) :
2525 //   WGMMMA Atom:      m64n128k16 (一次处理 64×128×16, 是 MMA 的 64 倍)
2526 //   K Tile:           BK=64, 每个 K tile 包含 BK/WGMMMA_K=4 个 WGMMMA atom
2527 //   M Tile:           BM=128, 每个 M tile 包含 BM/WGMMMA_M=2 个 WGMMMA atom
2528 //   N Tile:           BN=128, 单个 WGMMMA_N=128 即可覆盖, 无需在 N 方向分块
2529 //   Block Tile:       C[128,128] = A[128,64] × B[64,128]
2530 //   Threads:         256 = 2 warpgroups × 128 threads/warpgroup
2531 //   Producer(WG0):    128 threads (仅 thread 0 做 TMA 提交)
2532 //   Consumer(WG1):    128 threads = 4 warps (全部参与 WGMMMA 和写回)
2533 //
2534 // K_STAGE=3: 3 级流水线。Consumer 滞后 Producer 最多 2 步, 确保 HBM→SMEM
2535 // 的延迟被计算完全掩盖。
2536 //
2537 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
2538 //   - grid.z = S 个 swizzle 分区, 将连续 block 打散到不同 N 区域改善 L2 命中
2539 // Block: (256, 1, 1), 2 warpgroups
2540 // source: LeetCUDA/kernels/hgemm/wgmma/hgemm_wgmma_fp16acc_stages_tn.cu
2541 template <const int WGMMMA_M = 64, const int WGMMMA_N = 128,
2542           const int WGMMMA_K = 16, const int BM = 128, const int BN = 128,
2543           const int BK = 64, const int NUM_THREADS = 256, const int K_STAGE = 3,
2544           const bool BLOCK_SWIZZLE = false>
2545 __global__ void __launch_bounds__(NUM_THREADS)
2546   hgemm_wgmma_stages_tn(
2547     int M, int N, int K, half *C,
2548     const CTensorMap * __restrict__ tensorMapA,
2549     const CTensorMap * __restrict__ tensorMapB) {
2550   // 注意: tensorMapA/tensorMapB 需要由 host 侧按当前 tile 布局预先创建;
2551   // 对 row-major [M, K] 矩阵, TMA shape 参数写的是 (K, M) 而不是 (M, K),
2552   // 也就是TMA descriptor中把连续的维度写在最内层, 非连续的维度写在最外层。
2553   // 这是 TMA descriptor 最容易背错的地方之一。notes 这里只保留 kernel 主体,
2554   // 不展开宿主侧 create_tensor_map 细节。
2555
2556   // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
2557   // bx = blockIdx.z * gridDim.x + blockIdx.x, 将相邻 block 打散到不同 N 区域
2558   const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
2559   const int by = blockIdx.y;
2560   // num_consumers = (NUM_THREADS / 128) - 1 = 2 - 1 = 1 (1 个 consumer WG)
2561   // 这里的 -1 是因为 2 个 warpgroup 中 1 个是 producer, 剩余都是 consumer
2562   constexpr int num_consumers = (NUM_THREADS / 128) - 1; // 1 consumer WG
2563   // B_WG_M = BM / num_consumers: 每个 consumer warpgroup 负责的 M 行数
2564   // 当只有一个 consumer 时, 它负责全部 BM=128 行
2565   constexpr int B_WG_M = BM / num_consumers; // 128
2566
2567   // 边界检查: 确保当前 block 不超出 M/N 范围
2568   if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
2569     return;
2570
2571   // ---- Shared Memory 分配 ----
2572   // 动态 shared memory (由 host 侧通过 kernel launch 的 smem 参数指定大小)
2573   // __align__(128) 满足 TMA 和 WGMMMA 的 16B 对齐 + 128B swizzle 对齐要求
2574   extern __shared__ __align__(128) uint8_t smem_AB[];
2575   WgmmaSMem<BM, BN, BK, K_STAGE> &s =
2576     *reinterpret_cast<WgmmaSMem<BM, BN, BK, K_STAGE>*>(smem_AB);
2577   half *s_a = s.A;
2578   half *s_b = s.B;
2579
2580   // ---- cuda::barrier 初始化 ----
2581   //
2582   // ★ Barrier 机制速查 (arrive / wait 核心语义) :

```

```

2583 //
2584 // cuda::barrier 是一个 CTA 级同步原语，基于 **phase (奇偶交替)** 工作：
2585 //
2586 //   init(bar, N): 设置 arrive_count = N。
2587 //   含义：每 phase 需要 N 个线程各调用一次 arrive(), barrier 才翻转 phase。
2588 //
2589 //   arrive(): 线程声明"我已到达此 barrier 点"。
2590 //   - 非阻塞，立即返回一个 token (包含当前 phase 值)。
2591 //   - 不关心"谁"到达，只关心"到达次数"是否达到 arrive_count。
2592 //
2593 //   wait(token): 线程阻塞，直到 barrier 的当前 phase ≠ token 中的 phase。
2594 //   - 即：等待 phase 翻转。
2595 //   - Phase 翻转条件：(1) arrive 次数达到 arrive_count
2596 //                     (2) 若使用了 barrier_arrive_tx, 还需 TMA 字节全部写完
2597 //
2598 //   典型用法: b.wait(b.arrive())
2599 //   - arrive() 注册自己到达，拿到 token (当前 phase = P)
2600 //   - wait(token) 阻塞直到 phase 翻转 (P → P+1)
2601 //   - 如果自己是第 N 个到达者 → phase 立即翻转 → wait 立即返回 (不阻塞)
2602 //   - 如果自己不是最后一个 → wait 阻塞，直到第 N 个到达者触发翻转
2603 //
2604 // ★ 本 kernel 的 Pipeline 同步协议 (K_STAGE=3, arrive_count=129) :
2605 //
2606 //   Producer (1 thread)           Consumer (128 threads)
2607 //   ────────────                   ────────────
2608 //                                     C0: arrive(empty[*]) ×128 [init: 标记所有 stage 为空]
2609 //   ┌ for each k_tile: ─┐           ┌ for each k_tile: ─┐
2610 //   │ P1: arrive+wait(empty[q]) │   │ C1: arrive+wait(full[q]) │
2611 //   │   ↓ 等 stage q 被消费完   │   │   ↓ 等 TMA 数据就绪     │
2612 //   │ P2: TMA(A[q]) + TMA(B[q]) │   │ C2: WGMMMA 计算         │
2613 //   │   ↓ 异步拷贝             │   │ C3: wait WGMMMA 完成     │
2614 //   │ P3: arrive_tx(full[q])   │   │ C4: arrive(empty[q])    │
2615 //   │   ↑ 通知: stage q 数据就绪 │   │   ↑ 通知: stage q 已消费完
2616 //   └──────────────────┘           └──────────────────┘
2617 //
2618 //   关键不变式 (invariant) :
2619 //   - Producer 不会覆盖 Consumer 正在读的 stage
2620 //   - Consumer 不会读 Producer 还没写完的 stage
2621 //   - 通过 full/empty 两个 barrier 的 phase 交替来保证
2622 //
2623 //   每个 stage 有两个 barrier:
2624 //   full[qidx]: TMA 数据就绪信号。Producer 发 (arrive_tx), Consumer 等 (wait)。
2625 //   empty[qidx]: Stage 空闲信号。Consumer 发 (arrive), Producer 等 (wait)。
2626 //
2627 //   arrive_count = 128 (consumer) + 1 (producer) = 129:
2628 //   每 phase, 128 个 consumer 线程 + 1 个 producer 线程都要 arrive 一次。
2629 #pragma nv_diag_suppress static_var_with_dynamic_init
2630 __shared__ cuda::barrier<cuda::thread_scope_block> full[K_STAGE];
2631 __shared__ cuda::barrier<cuda::thread_scope_block> empty[K_STAGE];
2632 #pragma nv_diag_default static_var_with_dynamic_init
2633
2634 // K 方向总 tile 数。要求 K 能被 BK 整除，否则尾 tile 被丢弃。
2635 const int num_blocks_k = K / BK;
2636 const int wg_idx = threadIdx.x / 128; // 0=Producer, 1=Consumer
2637 const int tid = threadIdx.x % 128;    // 0~127 within warpgroup
2638
2639 // 初始化 barriers (仅 thread 0 执行)
2640 if (threadIdx.x == 0) {
2641     for (int i = 0; i < K_STAGE; ++i) {
2642         // arrive_count = 129: 128 consumer + 1 producer
2643         // init 是 CUDA barrier 的 hidden friend 函数 (定义在 cuda::barrier 类体内),
2644         // 只能通过 ADL (Argument-Dependent Lookup) 调用。因为第一个参数 &full[i] 的类型是
2645         // cuda::barrier<cuda::thread_scope_block>*, 编译器通过 ADL 在 cuda 命名空间中自动找到它。

```



```

2646     init(&full[i], num_consumers * 128 + 1); // 128 consumer + 1 producer
2647     init(&empty[i], num_consumers * 128 + 1); // same
2648 }
2649 // fence_proxy_async_shared_cta: 确保 barrier 在 smem 中的初始化
2650 // 对 async proxy (TMA/WGMMMA) 可见。
2651 cuda::device::experimental::fence_proxy_async_shared_cta();
2652 }
2653 __syncthreads();
2654
2655 // =====
2656 // ★ Warp Specialization 并发模型 — 为什么只有 if/else 没有 while?
2657 // =====
2658 //
2659 // 256 个线程在同一个 SM 上**并发执行**。if/else 只是分工，不是先后顺序：
2660 //   - WG0 (threadIdx.x 0~127): 全部走 if 分支，做 Producer。
2661 //   - WG1 (threadIdx.x 128~255): 全部走 else 分支，做 Consumer。
2662 //
2663 // SM 的 warp scheduler 会交替调度来自 WG0 和 WG1 的 warp，两者**同时**推进：
2664 //   Producer: for (k_iter = 0 .. num_blocks_k) { P1→P2→P3 } ← 自己的循环
2665 //   Consumer: for (k_iter = 0 .. num_blocks_k) { C1→C2→C3→C4 } ← 自己的循环
2666 //
2667 // 两个 warpgroups 各自独立迭代 K-tiles，通过 full[] / empty[] barrier 同步：
2668 //   - Producer 领先 Consumer 太多 → wait(empty[q]) 阻塞，等 Consumer 消费完
2669 //   - Consumer 领先 Producer 太多 → wait(full[q]) 阻塞，等 TMA 搬运完
2670 //
2671 // 这是生产者-消费者模型的 GPU 实现：不需要共享循环变量，barrier 的 phase
2672 // 交替天然保证了流水线串行化 (at most K_STAGE-1 steps ahead)。
2673 // =====
2674 // Producer Warpgroup (WG0, threadIdx.x 0~127)
2675 // 职责：提交 TMA 2D 拷贝，将 A/B tile 从 HBM 异步搬运到 SMEM。
2676 // 只有 tid==0 执行实际拷贝提交，其余 127 个线程空闲。
2677 // =====
2678 if (wg_idx == 0) {
2679     if (tid == 0) {
2680         // qidx: 当前操作的 stage 索引 (round-robin 0 -> 1 -> 2 -> 0 -> ...)
2681         int qidx = 0;
2682         for (int block_k_iter = 0; block_k_iter < num_blocks_k; ++block_k_iter, ++qidx) {
2683             if (qidx == K_STAGE)
2684                 qidx = 0;
2685
2686             // Step P1: 等待 stage qidx 变为"空" (可被覆盖写入)
2687             //
2688             // 模式 empty[qidx].wait(empty[qidx].arrive()):
2689             //   - arrive(): Producer (1 个线程) 在 empty barrier 上注册到达，
2690             //     获得当前 phase 的 token。如果 Consumer 已经在 C4 步骤积累了
2691             //     128 次 arrive (来自上一轮迭代或 C0 初始化)，则加上这次共 129 次
2692             //     → phase 立即翻转。
2693             //   - wait(token): 阻塞直到 barrier phase ≠ token 中的 phase。
2694             //     由于 arrive() 是第 129 次到达，phase 在 arrive 时已翻转，
2695             //     wait 看到 phase 已变 → 立即返回 (首轮不阻塞)。
2696             //
2697             // 语义：Producer 说"我准备好写 stage qidx 了，Consumer 用完没有？"
2698             //     如果 Consumer 还没用完 → phase 未翻转 → wait 阻塞等待。
2699             //     如果 Consumer 已用完 → phase 已翻转 → wait 立即返回。
2700             empty[qidx].wait(empty[qidx].arrive());
2701
2702             // Step P2: 提交 TMA 2D 拷贝指令
2703             // cp_async_bulk_tensor_2d_global_to_shared:
2704             //   参数1(dst): smem 目标地址 (当前 stage 的 A/B tile 起始位置)
2705             //   参数2(tensorMap): Host 预创建的 TMA descriptor (描述源矩阵的 shape/stride/dtype)
2706             //   参数3/4(coords): (k_offset, m_offset) 或 (k_offset, n_offset) 的全局坐标
2707             //   参数5(barrier): 拷贝完成后自动 arrive 到此 barrier
2708             //

```

```

2709 // TMA 是硬件 DMA 引擎：一次指令提交即可搬运整个 2D tile,
2710 // 无需线程逐元素搬运，零寄存器开销。
2711 // TMA 2D 加载 A tile: coords = (k_offset, m_offset)
2712 cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
2713     &s_a[qidx * BK * BM], tensorMapA, block_k_iter * BK, by * BM,
2714     full[qidx]);
2715
2716 // TMA 2D 加载 B tile: coords = (k_offset, n_offset)
2717 cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
2718     &s_b[qidx * BK * BN], tensorMapB, block_k_iter * BK, bx * BN,
2719     full[qidx]);
2720
2721 // Step P3: 通知 Consumer: stage qidx 的 TMA 数据已就绪
2722 //
2723 // barrier_arrive_tx(bar, arrive_count_update, byte_count):
2724 //   - 在 bar 上注册 1 次到达 (arrive_count_update=1)
2725 //   - 同时声明预期有 byte_count 字节将通过 async copy (TMA) 写入 smem
2726 //   - Phase 翻转条件:
2727 //       (a) 总 arrive 次数达到 129 (128 consumer + 1 producer)
2728 //       (b) 所有声明的 async 字节已写入 smem
2729 //   两个条件都满足后 phase 才翻转, Consumer 的 wait() 才返回。
2730 //   - 即: Consumer 不会在 TMA 数据完整到达前就开始读 smem。
2731 [[maybe_unused]] auto token = cuda::device::barrier_arrive_tx(
2732     full[qidx], 1, (BK * BN + BK * BM) * sizeof(half));
2733 }
2734 }
2735 }
2736 // =====
2737 // Consumer Warpgroup (WG1, threadIdx.x 128~255)
2738 // 职责: 等待 TMA 数据就绪 -> 发射 WGMMMA 做矩阵乘 -> 积累结果 -> 写回 C
2739 // 所有 128 个线程 (4 warps) 全部参与。
2740 // =====
2741 else {
2742     // Step C0: Consumer 初始化 — 标记所有 stage 为"空" (可被 Producer 写入)
2743     //
2744     // 所有 128 个 Consumer 线程对每个 stage 的 empty barrier 调用 arrive()。
2745     // 这是 Pipeline 的"预热"步骤——没有它, Producer 的 empty[qidx].wait()
2746     // 在第一轮会永远阻塞 (因为 Producer 的 1 次 arrive 不足以凑够 129)。
2747     //
2748     // 注意: 此时每个 empty[i] 只有 128 次 arrive, 未达 129, phase 不翻转。
2749     // Producer 后续的 empty[qidx].arrive() 作为第 129 次, 触发 phase 翻转。
2750     for (int i = 0; i < K_STAGE; ++i) {
2751         [[maybe_unused]] auto token = empty[i].arrive();
2752     }
2753
2754     // 累加器寄存器声明
2755     // d[B_WG_M / WGMMMA_M][WGMMMA_N / 16][4]:
2756     //   - d[0][*][*]: M 方向第 1 个 WGMMMA atom (rows 0~63)
2757     //   - d[1][*][*]: M 方向第 2 个 WGMMMA atom (rows 64~127)
2758     //   - d[*][g][*]: N 方向第 g 组 16 列
2759     //   - d[*][*][0..3]: 4 条 uint32 寄存器, 共 8 个 half (覆盖 16x16 子块)
2760     // 每个线程总共 2 * 8 * 4 = 2 * 32 = 64 uint32 = 128 half (每次WGMMMA Atom 32 uint32 输出)
2761     // 128 线程 * 128 half = 16384 half = 128 * 128 = BM*BN (刚好覆盖整个 C tile)
2762     uint32_t d[B_WG_M / WGMMMA_M][WGMMMA_N / 16][4] = {};
2763
2764     int qidx = 0;
2765     // K 维外循环: 沿 K tile 迭代 (BK=64, 每个 K tile 做 4 次 WGMMMA 累加)
2766     for (int block_k_iter = 0; block_k_iter < num_blocks_k; ++block_k_iter, ++qidx) {
2767         if (qidx == K_STAGE)
2768             qidx = 0;
2769
2770         // Step C1: 等待 TMA 数据就绪 (full 信号)
2771         //

```



```

2772 // 模式 full[qidx].wait(full[qidx].arrive()):
2773 // - 128 个 Consumer 线程各调用 arrive(), 共 128 次到达。
2774 // - 当前 phase 累积: 128/129, phase 尚未翻转。
2775 // - wait(token) 阻塞, 直到 Producer 的 barrier_arrive_tx (Step P3)
2776 // 贡献第 129 次到达 + TMA 字节全部写完 → phase 翻转 → wait 返回。
2777 //
2778 // 语义: Consumer 说"我准备好读 stage qidx 了, 数据到了没有? "
2779 full[qidx].wait(full[qidx].arrive());
2780
2781 // Step C2: 发射 WGMMMA 指令序列
2782 //
2783 // WGMMMA 是异步指令 (fire-and-forget), 发射后立即返回, 不等待计算完成。
2784 // 标准流程: FENCE → 发射 WGMMMA → COMMIT → WAIT。
2785 //
2786 // WGMMMA_FENCE (wgmma.fence.sync.aligned) :
2787 // - 确保 TMA 写入 smem 的数据对 async proxy (WGMMMA) 可见
2788 // - 确保累加器寄存器 d[] 已准备好接收 WGMMMA 输出
2789 // - 本质是 proxy 间的内存序 (memory ordering), 不是传统 __syncthreads
2790 //
2791 // 每个 WGMMMA_M64N128K16_F16F16F16 指令:
2792 // D[64×128] += A[64×16] × B[16×128]
2793 // A/B 均为 K-major (imm-trans=0), 由 descA/descB 描述 smem 中的布局。
2794 // ScaleD=1: 累加。K 维有 BK/WGMMMA_K=4 次迭代, 每次都使用 ScaleD=1。
2795 WGMMMA_FENCE();
2796
2797 // M 维迭代: BM/WGMMMA_M = 128/64 = 2 个 WGMMMA atom
2798 // 每个 atom 处理 64 行 M 方向数据。
2799 #pragma unroll
2800 for (int m_it = 0; m_it < B_WG_M / WGMMMA_M; ++m_it) {
2801 // wgmma_sA 指向当前 stage 中 A tile 的第 m_it 个 64×BK 子块
2802 // s_a 布局: [K_STAGE][BM][BK] → qidx * BK*BM + BK * (m_it*64)
2803 half *wgmma_sA = s_a + qidx * BK * BM + BK * m_it * WGMMMA_M;
2804
2805 // K 维迭代: BK/WGMMMA_K = 64/16 = 4 次 WGMMMA (累加)
2806 // 每次处理 K=16 维的矩阵乘, 4 次累加后覆盖完整的 BK=64。
2807 #pragma unroll
2808 for (int k_it = 0; k_it < BK / WGMMMA_K; ++k_it) {
2809 // 第 k_it 次 WGMMMA:
2810 // A: wgmma_sA + k_it * WGMMMA_K (A 的 K 维起始位置)
2811 // B: s_b + qidx * BK * BN + k_it * WGMMMA_K (B 的 K 维起始位置)
2812 // 注意 B 的 smem 布局是 [BK, BN] row-major, K-major 读取时从
2813 // 第 k_it*16 行开始取 16 行, 每行 BN=128 列。
2814 // ScaleD=1: 累加 (K 维迭代需要累积结果)
2815 WGMMMA_M64N128K16_F16F16F16(d[m_it], wgmma_sA + k_it * WGMMMA_K,
2816 s_b + qidx * BK * BN + k_it * WGMMMA_K,
2817 1, // ScaleD=1: accumulate (不清零)
2818 1, 1, 0, 0);
2819 }
2820 }
2821
2822 // Step C3: 提交并等待 WGMMMA 完成
2823 //
2824 // WGMMMA_COMMIT_GROUP (wgmma.commit_group.sync.aligned):
2825 // 将自上一次 COMMIT 以来发射的所有 WGMMMA 归为一组, 提交到 async proxy 执行。
2826 // 类比 cp.async.commit_group, 但 scope 是 warpgroup 而非 per-thread。
2827 //
2828 // WGMMMA_WAIT_GROUP(0) (wgmma.wait_group.sync.aligned 0):
2829 // 阻塞直到最多 N 个 group 尚未完成 (pending ≤ N)。N=0 即等待所有 group。
2830 // ★ 语义与 cp.async.wait_group 完全一致 (PTX ISA §9.7.15.7.3 vs §9.7.9.25.3.3) :
2831 // 两者都是 "wait until only N or fewer of the most recent groups are pending"。
2832 // 此处用 0 是因为 Consumer 在本次迭代中只 commit 了 1 个 group,
2833 // 必须等它全部完成才能进入下一步 (读下一 stage 的数据)。
2834 WGMMMA_COMMIT_GROUP();

```

```

2835     WGMMMA_WAIT_GROUP(0);
2836
2837     // Step C4: 释放 stage qidx — 通知 Producer 可以覆盖写入
2838     //
2839     // 128 个 Consumer 线程在 empty[qidx] 上 arrive(), 为 **下一 phase** 积累到达次数。
2840     // 这 128 次 arrive 不会立即翻转 phase (还需 Producer 在下一轮的 1 次 arrive)。
2841     //
2842     // 语义: Consumer 说"stage qidx 我已经读完了, 你可以放心覆盖了。"
2843     [[maybe_unused]] auto token = empty[qidx].arrive();
2844 }
2845
2846 // =====
2847 // Epilogue: 将寄存器中的累加结果写回 global memory C
2848 // =====
2849 //
2850 // WGMMMA m64n128k16.f16.f16.f16 的输出寄存器映射:
2851 //
2852 // 对 warpgroup 内每个线程, 输出 64 个 half = 32 个 uint32。
2853 // 这些 half 分布在 d[2][8][4] 数组中:
2854 //   d[m_it][g][0]: (row, col) 和 (row, col+2) 位置的 2 个 half
2855 //   d[m_it][g][1]: (row+8, col) 和 (row+8, col+2) 位置的 2 个 half
2856 //   d[m_it][g][2]: (row, col+8) 和 (row, col+10) 位置的 2 个 half
2857 //   d[m_it][g][3]: (row+8, col+8) 和 (row+8, col+10) 位置的 2 个 half
2858 //
2859 // 其中:
2860 //   row = warp * 16 + lane / 4    (0~63, 4 warps * 16 rows)
2861 //   col = g * 16 + 2 * (lane % 4) (0~126, step 2, 8 g's * 16 cols)
2862 //
2863 // 每个线程覆盖一个 16x16 子块中的 16 个 half:
2864 //   +-----+-----+
2865 //   | reg[0] | reg[2] | <- rows [row, row+8)
2866 //   |(col,+2)|(col+8)|
2867 //   +-----+-----+
2868 //   | reg[1] | reg[3] | <- rows [row+8, row+16)
2869 //   |(col,+2)|(col+8)|
2870 //   +-----+-----+
2871 //   每个 reg 包含 2 个连续 half (col 和 col+2), 用 uint32 存储。
2872 //
2873 // 三维遍历:
2874 //   m_it=0: rows 0~63, m_it=1: rows 64~127
2875 //   g=0..7: 每个覆盖 16 列, 8*16=128 列 ok
2876 //   每个线程写 4 uint32 * 2 half/uint32 * 8 g * 2 m_it = 128 half。
2877 //   128 线程 * 128 half = 16384 half = 128 * 128 ok
2878
2879 const int lane = tid % 32;
2880 const int warp = tid / 32;
2881 // row: 当前线程在当前 WGMMMA atom 中负责的起始行。
2882 // warp 0~3 各负责 16 行: rows [warp*16, warp*16+15]。
2883 // lane/4: 每 4 个连续 lane 负责同一行 (因为 WGMMMA fragment 中每行有 4 个 uint32,
2884 //         分别覆盖列对 {c0,c1}、{c2,c3}、{c8,c9}、{c10,c11}, 由 lane%4 区分)。
2885 //         因此 lane/4 给出该行在 16-row block 内的行号 (0~7 对应 rows 0~7,
2886 //         同一 warp 中 lane/4==0 的有 lane 0,1,2,3, 都负责 row 0)。
2887 const int row = warp * 16 + lane / 4;
2888 // block_C: 当前 C tile 在 global memory 中的起始地址
2889 // by*BM 是 M 方向偏移, bx*BN 是 N 方向偏移
2890 half *block_C = C + by * BM * N + bx * BN;
2891
2892 // 2 * (8 * 4) = 2 * 32 = 64 uint32 = 128 half
2893 #pragma unroll
2894 for (int m_it = 0; m_it < B_WG_M / WGMMMA_M; ++m_it) {
2895     int yo = m_it * WGMMMA_M; // M 方向行偏移 (0 或 64)
2896 #pragma unroll
2897     for (int g = 0; g < WGMMMA_N / 16; ++g) {

```

```

2898     int col = g * 16 + 2 * (lane % 4); // N 方向列偏移 (0,2,4,...,14 then 16,18,...)
2899     // 一次 uint32 store 写入 2 个 half (连续列 col 和 col+2)
2900     // 左上象限: (row, col)
2901     *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col]) =
2902         d[m_it][g][0];
2903     // 左下象限: (row+8, col)
2904     *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col]) =
2905         d[m_it][g][1];
2906     // 右上象限: (row, col+8)
2907     *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col + 8]) =
2908         d[m_it][g][2];
2909     // 右下象限: (row+8, col+8)
2910     *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col + 8]) =
2911         d[m_it][g][3];
2912 }
2913 }
2914 }
2915 }
2916
2917 #if (defined(NOTES_V2_ENABLE_WGMMMA))
2918 // ---- Host-side TMA Tensor Map helpers (WGMMMA test) ----
2919 // 面试要点 (TMA descriptor 创建 — cuTensorMapEncodeTiled):
2920 //   - TMA descriptor 描述 global memory 中矩形的 shape/stride/dtype,
2921 //     以及硬件搬运的 box (tile) 大小和 smem swizzle 模式
2922 //   - 关键易错点: TMA shape 参数写的是 (W,H) 而不是 (H,W)!
2923 //     对 row-major [M,K] 矩阵, TMA shape = (K,M), minor=K, major=M
2924 //   - cuTensorMapEncodeTiled 是 CUDA Driver API, 需 #include <cuda.h> 并链接 -lcuda
2925 //   - smem_box 维度顺序也是 (minor, major) = (BK, BM) 或 (BK, BN)
2926 //   - Swizzle 模式通常选 CU_TENSOR_MAP_SWIZZLE_128B 配合 WGMMMA 的 128B swizzle
2927 //
2928 // ★ gmem_prob_stride + 1 的含义:
2929 // 语法层面 — 数组名退化 + 指针算术:
2930 //   gmem_prob_stride 类型为 uint64_t[5], 在表达式中退化为 uint64_t*
2931 //   (指向首元素 gmem_prob_stride[0])。+ 1 偏移 1 个 uint64_t, 指向
2932 //   gmem_prob_stride[1], 等价于 &gmem_prob_stride[1]。
2933 // 语义层面 — 跳过隐式的最内维 stride:
2934 //   cuTensorMapEncodeTiled 的 globalStrides 参数只接收 tensorRank-1 个
2935 //   stride 值。最内维 (fastest-changing dimension) 的 stride 是隐式的,
2936 //   固定等于 sizeof(element_type), 不需要显式传入。
2937 //   对于 tensorRank=2 的 FP16 矩阵:
2938 //       dim 0 (内维, K): stride = sizeof(half)    ← 隐式, API 不需要
2939 //       dim 1 (外维, M): stride = sizeof(half)*K  ← 需要传给 API
2940 //   gmem_prob_stride 数组的布局是内维在前:
2941 //       [0] = sizeof(half)                        ← 内维, 不传
2942 //       [1] = sizeof(half) * BlockMinorSize * blocks_width ← 外维, 传给 API
2943 //   所以 + 1 的作用是跳过 [0], 让 API 从 [1] 开始读取, 正好得到外维 stride。
2944 //
2945 // ★ smem_box_stride 为什么不需要 +1?
2946 //   与 globalStrides 不同, smemBoxStrides 参数接收完整的 tensorRank 个 stride,
2947 //   最内维 stride 也必须显式传入 (不隐式)。
2948 //   smem_box_stride[5] = {1, 1, 1, 1, 1} 表示 smem tile 中所有维度元素
2949 //   都是连续存放的, 维度间没有 padding/stride。
2950 //   对于 tensorRank=2: strides[0]=1 (内维 stride), strides[1]=1 (外维 stride),
2951 //   即 smem 中第 (i,j) 元素的偏移 = i*1 + j*1 = i+j (元素连续排列)。
2952 //   所以这里直接传 smem_box_stride (即 &smem_box_stride[0]), API 会读到全部
2953 //   两个 stride 值, 不需要跳过任何一个。
2954 //
2955 // 参考: CUDA Programming Guide §TMA, PTX ISA §9.7.15.5, CUTLASS GmmaDescriptor
2956
2957 template <int BlockMajorSize, int BlockMinorSize>
2958 __host__ static inline void create_tensor_map(CUtensorMap *tma_map,
2959                                               half *gmem_ptr,
2960                                               int blocks_height,

```

```

2961         int blocks_width) {
2962     void *gmem_address = (void *)gmem_ptr;
2963     uint64_t gmem_prob_shape[5] = {(uint64_t)BlockMinorSize * blocks_width,
2964                                     (uint64_t)BlockMajorSize * blocks_height,
2965                                     1, 1, 1};
2966     uint64_t gmem_prob_stride[5] = {
2967         sizeof(half), sizeof(half) * BlockMinorSize * blocks_width, 0, 0, 0};
2968     uint32_t smem_box_shape[5] = {uint32_t(BlockMinorSize),
2969                                   uint32_t(BlockMajorSize), 1, 1, 1};
2970     uint32_t smem_box_stride[5] = {1, 1, 1, 1, 1};
2971     CUresult result = cuTensorMapEncodeTiled(
2972         tma_map, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, 2, gmem_address,
2973         gmem_prob_shape, gmem_prob_stride + 1, smem_box_shape, smem_box_stride,
2974         CU_TENSOR_MAP_INTERLEAVE_NONE, CU_TENSOR_MAP_SWIZZLE_128B,
2975         CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE);
2976     if (result != CUDA_SUCCESS)
2977         printf("cuTensorMapEncodeTiled failed: %d\n", (int)result);
2978 }
2979
2980 __host__ static inline CUTensorMap *allocate_and_create_tensor_map(
2981     half *src, int blocks_height, int blocks_width) {
2982     CUTensorMap *tma_map_d;
2983     cudaMalloc(&tma_map_d, sizeof(CUTensorMap));
2984     CUTensorMap tma_map_host;
2985     create_tensor_map<128, 64>(&tma_map_host, src, blocks_height, blocks_width);
2986     cudaMemcpy(tma_map_d, &tma_map_host, sizeof(CUTensorMap),
2987               cudaMemcpyHostToDevice);
2988     return tma_map_d;
2989 }
2990 #endif /* NOTES_V2_ENABLE_WGMMA */
2991
2992 // =====
2993 // Phase 8: FlashAttention-2 (Split-Q + MMA m16n8k16)
2994 // =====
2995 // 面试题点 (FlashAttention 算法) :
2996 // 1. 核心问题: 标准 Attention 的  $O(N^2)$  中间矩阵 ( $S=QK^T$ ) 必须写入 HBM,
2997 //    但 HBM 带宽是瓶颈 → FlashAttention 用 tiling + online softmax 避免写回
2998 // 2. FA 三板斧:
2999 //    a) Tiling: Q 分块 [Br,d], K/V 沿 seqlen 分块 [Bc,d]
3000 //    b) Online Softmax: 迭代更新 m(行max) 和 l(行sum), 无需全局同步
3001 //    c) Recomputation(反向): 反向传播时重新计算 S/P, 而非存储中间矩阵
3002 // 3. Split-Q 设计: 所有 warp 共享同一块 K, 各 warp 处理 Q 的不同行片段
3003 //    - warp_KV=0 (所有 warp 共享 K), warp_QP=warp_id (各 warp 不同 Q 行)
3004 //    - 优点: 减少 warp 间通信和 shuffle
3005 // 4. Online rescaling 公式 (FA 核心, arXiv:2307.08691) :
3006 //    for each K,V tile:
3007 //        S_cur = Q @ K^T // 未缩放, 存入 R_S
3008 //        m_new = max(m_old, row_max(S_cur * scale))
3009 //        P_cur = exp(S_cur * scale - m_new) // ← 写回 R_S 寄存器!
3010 //        l_new = exp(m_old - m_new) * l_old + row_sum(P_cur)
3011 //        O_new = diag(exp(m_old - m_new)) * O_old + P_cur @ V
3012 //        O_final = O_new / l_final
3013 // 5. 为什么 R_S 可以直接用作 P@V 的 A 矩阵?
3014 //    - R_S 经过 softmax 后, 存储的是  $P = \exp(S - m)$ , 数据仍然是 half 精度
3015 //    - 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局, 使 softmax
3016 //      后的 P 可以继续留在 R_S 中供后面的 P@V 直接消费
3017 //    - 这是此实现的寄存器布局复用技巧, 不要背成“所有 MMA A/C fragment
3018 //      都天然同构”的通用结论
3019 //
3020 // 本实现参考: FlashAttention-2 (Dao et al., arXiv:2307.08691)
3021 // 从 LeetCUDA flash-attn/mma/basic/flash_attn_mma_split_q.cu 提取
3022 // Grid: ((QKV_seqlen + 63) / 64, QKV_batch * QKV_head, 1), Br=64
3023 // Block: (128, 1, 1), kNumThreads=WARP_SIZE×kMmaTileSeqLenQ×kMmaTileSeqLenK=128

```

```

3024 // source: LeetCUDA/kernels/flash-attn/mma/basic/flash_attn_mma_split_q.cu
3025
3026 // ---- 寄存器填充辅助函数 ----
3027 template <typename T, int M, const int N, const int K = 2>
3028 __device__ inline void fill_3D_regs(T (&R)[M][N][K], T val) {
3029     #pragma unroll
3030     for (int i = 0; i < M; ++i)
3031     #pragma unroll
3032         for (int j = 0; j < N; ++j)
3033     #pragma unroll
3034         for (int k = 0; k < K; ++k)
3035             R[i][j][k] = val;
3036 }
3037
3038 template <typename T, int M, const int N = 2>
3039 __device__ inline void fill_2D_regs(T (&R)[M][N], T val) {
3040     #pragma unroll
3041     for (int i = 0; i < M; ++i)
3042     #pragma unroll
3043         for (int j = 0; j < N; ++j)
3044             R[i][j] = val;
3045 }
3046
3047 // =====
3048 // FlashAttention-2 Split-Q Kernel (完整实现)
3049 // =====
3050 // Q,K,V,0: [batch_size, num_heads, seq_len, head_dim], [B,H,N,d]
3051 //
3052 // Tile 设计 (以 kHeadDim=64 为例) :
3053 //   Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ = 16*4*1 = 64
3054 //   Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK = 8*1*8 = 64
3055 //   Warp 布局: 4 warps, warp_QP 0~3 各处理 16 行, warp_KV=0 共享 K
3056 //
3057 // 执行流程:
3058 //   1) 预加载 Q[Br,d] 到 smem (只加载一次, split-Q 的核心优势)
3059 //   2) 外循环: 沿 K seqlen 分块迭代 (Tc = seqlen/Bc)
3060 //   3)   3a: cp.async 加载当前 V + 预加载下一个 K (多 stage pipeline)
3061 //   4)   3b: Q@K^T — 沿 head dim 内循环 → ldmatrix Q/K → HMMA16816
3062 //   5)   3c: Online Safe Softmax — warp reduce max/row → exp(S*scale - max)
3063 //       → 关键: 将 P 写回 R_S 寄存器 (替换 S), R_S 现在存储 P matrix
3064 //   6)   3d: P@V — 沿 V_Bc 内循环 → ldmatrix V (transposed) → 直接用 R_S 做 A
3065 //       → 当前实现依赖同一组寄存器布局约定, 避免为 P@V 额外重组 P fragment
3066 //   7)   3e: Online rescaling — O_new = exp(m_old-m_new)*O_old + P@V
3067 //   8) 最终 rescale: O_final = (1/l_final)*O_final
3068 //   9) Epilogue: warp shuffle + 128-bit collective store
3069
3070 template <
3071     const int kHeadDim,           // head dim: 32, 64, 128
3072     const int kMmaAtomM,          // 16 (MMA instruction M dimension)
3073     const int kMmaAtomN,          // 8 (MMA instruction N dimension)
3074     const int kMmaAtomK,          // 16 (MMA instruction K dimension)
3075     const int kMmaTileSeqLenQ,    // MMA tiles along Q's M dim, 4 → Br=16*4=64
3076     const int kMmaTileSeqLenK,    // MMA tiles along K's N dim, 1 → Bc basis=8
3077     const int kMmaTileSeqLenP,    // MMA tiles for P@V M dim, must equal kMmaTileSeqLenQ
3078     const int kMmaTileHeadDimV,   // MMA tiles for P@V N dim (head dim direction)
3079     const int kValTileSeqLenQ,    // value tiles along Q's M, 1 → Br per warp=16
3080     const int kValTileSeqLenK,    // value tiles along K's N, 8 → Bc_warp=8*8=64
3081     const int kValTileSeqLenP,    // value tiles for P@V M dim, 1
3082     const int kValTileHeadDimV,   // value tiles for P@V N dim, kHeadDim/(8*kMmaTileHeadDimV)
3083     const int kStage,             // pipeline stages for K: 1 or 2
3084     const int kPad>
3085 __global__ void __launch_bounds__(WARP_SIZE * kMmaTileSeqLenQ * kMmaTileSeqLenK)
3086     flash_attn_mma_stages_split_q(half *Q, half *K, half *V, half *O,

```

```

3087         int QKV_seqlen, int QKV_head) {
3088     constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ; // 64
3089     constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK; // 64
3090     constexpr int kNumThreads = WARP_SIZE * kMmaTileSeqLenQ * kMmaTileSeqLenK; // 128
3091     const int Tc = (QKV_seqlen + Bc - 1) / Bc;
3092     // 原始实现默认 seqlen 与 Bc 对齐; 最后一个不完整 tile 需要额外 pad/边界处理。
3093     // 这里保留 ceil 写法是为了说明 tile 划分方式, 不等于当前实现已经完整处理了尾 tile。
3094     const float scale = 1.0f / sqrtf((float)kHeadDim);
3095
3096     // Block indexing
3097     const int QKV_batch_id = blockIdx.y / QKV_head;
3098     const int QKV_head_id = blockIdx.y % QKV_head;
3099     const int Q_tile_id = blockIdx.x;
3100     const int tid = threadIdx.x;
3101     const int warp_id = tid / WARP_SIZE;
3102     const int lane_id = tid % WARP_SIZE;
3103     const int warp_QP = warp_id; // Split-Q: 每个 warp 处理不同的 Q 行片段
3104     const int warp_KV = 0; // 所有 warp 共享 K (减少跨 warp 通信)
3105
3106     // Global memory base offsets for this (batch, head)
3107     // 这里默认 Q/K/V 共享同一 per-head 基址布局, 对应 self-attention 场景
3108     const int Q_gmem_offset =
3109         (QKV_batch_id * QKV_head * QKV_seqlen + QKV_head_id * QKV_seqlen) *
3110         kHeadDim;
3111     const int K_gmem_offset = Q_gmem_offset;
3112     const int V_gmem_offset = Q_gmem_offset;
3113     const int O_gmem_offset = Q_gmem_offset;
3114
3115     // Thread-to-smem mapping for cooperative load
3116     int load_smem_Q_Br = tid / (kNumThreads / Br);
3117     int load_smem_Q_d =
3118         (tid % (kNumThreads / Br)) * (kHeadDim / (kNumThreads / Br));
3119     int load_smem_K_Bc = tid / (kNumThreads / Bc);
3120     int load_smem_K_d =
3121         (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
3122     int load_smem_V_Bc = tid / (kNumThreads / Bc);
3123     int load_smem_V_d =
3124         (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
3125
3126     int load_gmem_Q_Br = Q_tile_id * Br + load_smem_Q_Br;
3127     if (load_gmem_Q_Br >= QKV_seqlen)
3128         return;
3129
3130     // ---- Shared memory layout ----
3131     extern __shared__ half smem[];
3132     constexpr int Q_tile_size = Br * (kHeadDim + kPad); // Q tile: [Br, d+kPad]
3133     constexpr int KV_tile_size = Bc * (kHeadDim + kPad); // K/V tile: [Bc, d+kPad]
3134     half *Q_tile_smem = smem;
3135     half *K_tile_smem = Q_tile_smem + Q_tile_size;
3136     half *V_tile_smem = K_tile_smem + kStage * KV_tile_size; // kStage copies of K
3137     // 原始 kernel 还留了一个优化点: 若 kStage=1, K 和 V 在时序上并不重叠,
3138     // 理论上可以复用同一块 KV shared memory 来进一步压缩 smem 占用。
3139
3140     uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_tile_smem);
3141     uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_tile_smem);
3142     uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_tile_smem);
3143
3144     // ---- Online Softmax persistent state ----
3145     // lane_block_row_max_old[i][r]: running max for row r of warp tile i
3146     // lane_block_row_sum_old[i][r]: running denominator l for row r of warp tile i
3147     float lane_block_row_max_old[kValTileSeqLenQ][2];
3148     float lane_block_row_sum_old[kValTileSeqLenQ][2];
3149     fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_max_old, -INFINITY);

```



```

3150 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_sum_old, 0.0f);
3151
3152 // ---- Register allocation ----
3153 uint32_t R_Q[kValTileSeqLenQ][4]; // Q regs
3154 uint32_t R_K[kValTileSeqLenK][2]; // K regs
3155 uint32_t R_V[kValTileHeadDimV][2]; // V regs
3156 // R_S / R_O / R_D 都按 mma.sync.aligned.m16n8k16 的 fragment 约定存储。
3157 // 对单个 m16n8k16 tile 而言：
3158 // - reg[0] 持有该 tile 前 8 行里的两个 half 值
3159 // - reg[1] 持有该 tile 后 8 行里的两个 half 值
3160 // 后续 softmax、P@V、online rescale 都直接围绕这组 fragment 布局做寄存器内变换。
3161 uint32_t R_S[kValTileSeqLenQ][kValTileSeqLenK][2]; // S=Q@K^T / P=softmax(S)
3162 uint32_t R_O[kValTileSeqLenP][kValTileHeadDimV][2]; // O for current tile
3163 uint32_t R_D[kValTileSeqLenP][kValTileHeadDimV]
3164         [2]; // O accumulator (final output)
3165
3166 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
3167 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_D, 0);
3168
3169 // =====
3170 // Step 1: 加载 Q[Br, d] 到 shared memory (整个外循环只加载一次)
3171 // =====
3172 {
3173     int load_gmem_Q_addr =
3174         Q_gmem_offset + load_gmem_Q_Br * kHeadDim + load_smem_Q_d;
3175     uint32_t load_smem_Q_ptr =
3176         smem_Q_base_ptr +
3177         (load_smem_Q_Br * (kHeadDim + kPad) + load_smem_Q_d) * sizeof(half);
3178 #pragma unroll
3179     for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {
3180         CP_ASYNC_CG(load_smem_Q_ptr + i * 2, &Q[load_gmem_Q_addr + i], 16);
3181     }
3182     CP_ASYNC_COMMIT_GROUP();
3183 }
3184
3185 // =====
3186 // Step 2: 预加载前 (kStage-1) 个 K tile (多 stage pipeline 预热)
3187 // 注意: Q 由 blockIdx.x 固定到当前 Q tile; 而 K/V 的 seqlen 遍历始终从 tile 0 开始,
3188 // 后续在外循环里不断递增至 tile 1/2/3/.../Tc-1。
3189 // =====
3190 if constexpr (kStage > 1) {
3191 #pragma unroll
3192     for (int stage = 0; stage < (kStage - 1); ++stage) {
3193         int load_gmem_K_Bc = stage * Bc + load_smem_K_Bc;
3194         int load_gmem_K_addr =
3195             K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
3196         uint32_t load_smem_K_ptr =
3197             smem_K_base_ptr +
3198             (stage * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
3199              load_smem_K_d) *
3200             sizeof(half);
3201 #pragma unroll
3202         for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
3203             CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
3204         }
3205         CP_ASYNC_COMMIT_GROUP();
3206     }
3207     CP_ASYNC_WAIT_GROUP(kStage - 2);
3208     __syncthreads();
3209 }
3210
3211 // =====
3212 // Step 3: 外循环 — 沿 K seqlen 迭代 (Tc = ceil(seqlen/Bc))

```



```

3213 // 每次迭代处理一个 K[Bc,d] + V[Bc,d] tile
3214 // =====
3215 #pragma unroll 1
3216 for (int tile_K_seqlen = 0; tile_K_seqlen < Tc; ++tile_K_seqlen) {
3217     int smem_sel = tile_K_seqlen % kStage;
3218     int smem_sel_next = (tile_K_seqlen + (kStage - 1)) % kStage;
3219
3220     // ---- 3a: 异步加载 K/V tile (多 stage pipeline) ----
3221     if constexpr (kStage > 1) {
3222         // 只有 kStage>1 才能真正做 K 的 pipeline:
3223         // smem_sel 负责 “当前正在计算” 的 K tile, smem_sel_next 负责 “下一轮预取” 的 K tile。
3224         // 若 kStage=1, 这两个槽位永远都等于 0, 当前 K 还没算完就无法安全覆盖同一块 smem。
3225         // Load current V tile (no pipeline for V — one stage is enough)
3226         {
3227             int load_gmem_V_Bc = tile_K_seqlen * Bc + load_smem_V_Bc;
3228             int load_gmem_V_addr =
3229                 V_gmem_offset + load_gmem_V_Bc * kHeadDim + load_smem_V_d;
3230             uint32_t load_smem_V_ptr =
3231                 smem_V_base_ptr +
3232                 (load_smem_V_Bc * (kHeadDim + kPad) + load_smem_V_d) * sizeof(half);
3233 #pragma unroll
3234 for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
3235     CP_ASYNC.CG(load_smem_V_ptr + i * 2, &V[load_gmem_V_addr + i], 16);
3236 }
3237 CP_ASYNC.COMMIT_GROUP();
3238 }
3239
3240 // Prefetch next K tile (pipelined)
3241 if ((tile_K_seqlen + 1) < Tc) {
3242     int load_gmem_K_Bc = (tile_K_seqlen + 1) * Bc + load_smem_K_Bc;
3243     int load_gmem_K_addr =
3244         K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
3245     uint32_t load_smem_K_ptr =
3246         smem_K_base_ptr +
3247         (smem_sel_next * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
3248          load_smem_K_d) *
3249         sizeof(half);
3250 #pragma unroll
3251 for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
3252     CP_ASYNC.CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
3253 }
3254 CP_ASYNC.COMMIT_GROUP();
3255 }
3256 }
3257
3258 // ---- 3b: Q@K^T = S[Br, Bc] — 沿 head dim (d/kMmaAtomK=16) 内循环 ----
3259 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
3260 #pragma unroll
3261 for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d) {
3262     // ldmatrix.x4: 加载 Q 的 m16k16 片段到 R_Q
3263     #pragma unroll
3264     for (int i = 0; i < kValTileSeqLenQ; ++i) {
3265         int warp_smem_Q_Br =
3266             warp_QP * (kMmaAtomM * kValTileSeqLenQ) + i * kMmaAtomM;
3267         int lane_smem_Q_Br =
3268             warp_smem_Q_Br + lane_id % 16; // ldmatrix uses 16 lanes
3269         int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8; // 0, 8
3270         uint32_t lane_smem_Q_ptr =
3271             smem_Q_base_ptr +
3272             (lane_smem_Q_Br * (kHeadDim + kPad) + lane_smem_Q_d) * sizeof(half);
3273         LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
3274                    lane_smem_Q_ptr);
3275     }

```

```

3276 // ldmatrix.x2: 加载 K 的 k16n8 片段到 R_K
3277 // K[Bc,d] row-major = K^T[d,Bc] col-major (NT 布局的 B 矩阵)
3278 #pragma unroll
3279 for (int j = 0; j < kValTileSeqLenK; ++j) {
3280     int warp_smem_K_Bc =
3281         warp_KV * (kMmaAtomN * kValTileSeqLenK) + j * kMmaAtomN;
3282     int lane_smem_K_Bc =
3283         warp_smem_K_Bc + lane_id % 8; // ldmatrix B uses 8 lanes
3284     int lane_smem_K_d =
3285         tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8; // 0, 8
3286     uint32_t lane_smem_K_ptr =
3287         smem_K_base_ptr +
3288         (smem_sel * KV_tile_size + lane_smem_K_Bc * (kHeadDim + kPad) +
3289          lane_smem_K_d) *
3290         sizeof(half);
3291     LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);
3292 }
3293
3294 // MMA: S[tile] += Q[tile] @ K^T[tile]
3295 #pragma unroll
3296 for (int i = 0; i < kValTileSeqLenQ; ++i) {
3297     #pragma unroll
3298     for (int j = 0; j < kValTileSeqLenK; ++j) {
3299         HMMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1], R_Q[i][2],
3300                  R_Q[i][3], R_K[j][0], R_K[j][1], R_S[i][j][0],
3301                  R_S[i][j][1]);
3302     }
3303 }
3304 } // end loop over d
3305 __syncthreads();
3306
3307 // =====
3308 // 3c: Online Safe Softmax — row-wise max + exp + sum, then store P back to R_S
3309 // =====
3310 // MMA C fragment layout for m16n8k16 (PTX ISA 对应的线程寄存器分布):
3311 // - R_S[i][j][0] 对应当前 16x8 tile 的 rows 0~7 里的两个 half 值 {c0,c1}
3312 // - R_S[i][j][1] 对应 rows 8~15 里的两个 half 值 {c2,c3}
3313 // - lane 0~3 持有 row 0 的片段, lane 4~7 持有 row 1, ..., lane 28~31 持有 row 7
3314 // - 对于 rows 8~15 也是同样的 lane 分组, 只是读取的是 reg[1]
3315 // 这就是为什么后面做 row max / row sum 时, warp 内真正参与同一行归约的是
3316 // {0,4,8,...,28} 这一类 4-lane 子组, 而不是整 warp 32 个线程。
3317 // Each (i, j) pair = one 16x8 MMA tile; there are kValTileSeqLenQ x
3318 // kValTileSeqLenK tiles.
3319
3320 float lane_row_max_new[kValTileSeqLenQ][2];
3321 float lane_row_sum_new[kValTileSeqLenQ][2];
3322 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_max_new, -INFINITY);
3323 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);
3324
3325 // Pass 1: Thread-level reduce max across all columns (kValTileSeqLenK tiles)
3326 #pragma unroll
3327 for (int i = 0; i < kValTileSeqLenQ; ++i) {
3328     #pragma unroll
3329     for (int j = 0; j < kValTileSeqLenK; ++j) {
3330         // Extract half values from R_S registers (C matrix fragment layout)
3331         float2 t_reg_S_0 =
3332             __half2float2(HALF2(R_S[i][j][0])); // rows 0~7: {c0, c1}
3333         float2 t_reg_S_1 =
3334             __half2float2(HALF2(R_S[i][j][1])); // rows 8~15: {c2, c3}
3335         // S = (Q@K^T) * scale
3336         float tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;
3337         float tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;

```

```

3339     lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
3340     lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
3341 }
3342 // Warp-level reduce max (warp_size = 4 for Q@K^T — only lanes
3343 // {0,4,8,...,28} hold valid data)
3344 lane_row_max_new[i][0] =
3345     warp_reduce_max<4, float>(lane_row_max_new[i][0]);
3346 lane_row_max_new[i][1] =
3347     warp_reduce_max<4, float>(lane_row_max_new[i][1]);
3348 }
3349
3350 // Pass 2: Compute P = exp(S*scale - m_new), store back to R_S
3351 // 面试关键点: 这里将 P 写回 R_S 寄存器!
3352 // 为什么可以? 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局,
3353 // 使 softmax 后的 P 能继续留在 R_S 中供后面的 P@V 直接消费, 无需额外重组。
3354 #pragma unroll
3355 for (int i = 0; i < kValTileSeqLenQ; ++i) {
3356     // m_new = max(m_old, m_cur)
3357     float block_row_max_new_0 =
3358         max(lane_block_row_max_old[i][0], lane_row_max_new[i][0]);
3359     float block_row_max_new_1 =
3360         max(lane_block_row_max_old[i][1], lane_row_max_new[i][1]);
3361
3362 #pragma unroll
3363 for (int j = 0; j < kValTileSeqLenK; ++j) {
3364     float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
3365     float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));
3366
3367     // P = exp(S * scale - m_new), 用 fma 保证精度
3368     t_reg_S_0.x =
3369         __expf(__fmaf_rn(t_reg_S_0.x, scale, -block_row_max_new_0));
3370     t_reg_S_0.y =
3371         __expf(__fmaf_rn(t_reg_S_0.y, scale, -block_row_max_new_0));
3372     t_reg_S_1.x =
3373         __expf(__fmaf_rn(t_reg_S_1.x, scale, -block_row_max_new_1));
3374     t_reg_S_1.y =
3375         __expf(__fmaf_rn(t_reg_S_1.y, scale, -block_row_max_new_1));
3376
3377     // Accumulate row sums
3378     lane_row_sum_new[i][0] += (t_reg_S_0.x + t_reg_S_0.y);
3379     lane_row_sum_new[i][1] += (t_reg_S_1.x + t_reg_S_1.y);
3380
3381     // 关键: 将 P 写回 R_S! R_S 现在存储的是 P = softmax(S), 不是 S
3382     HALF2(R_S[i][j][0]) = __float22half2_rn(t_reg_S_0);
3383     HALF2(R_S[i][j][1]) = __float22half2_rn(t_reg_S_1);
3384 }
3385
3386 // Warp-level reduce sum (warp_size = 4, same as max)
3387 lane_row_sum_new[i][0] =
3388     warp_reduce_sum<4, float>(lane_row_sum_new[i][0]);
3389 lane_row_sum_new[i][1] =
3390     warp_reduce_sum<4, float>(lane_row_sum_new[i][1]);
3391 }
3392 __syncthreads();
3393
3394 // =====
3395 // 3d: P@V — P[Br,Bc] @ V[Bc,d] = O[Br,d]
3396 // =====
3397 // Wait for V to be ready before computing P@V
3398 if constexpr (kStage > 1) {
3399     if ((tile_K_seqLen + 1) < Tc) {
3400         CP_ASYNC_WAIT_GROUP(1);
3401     } else {

```

```

3402     CP_ASYNC_WAIT_GROUP(0);
3403 }
3404 } else {
3405     CP_ASYNC_WAIT_GROUP(0);
3406 }
3407 __syncthreads();
3408
3409 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_0, 0);
3410
3411 // tile_V_Bc: iterate over chunks of Bc/K=16 columns in P matrix
3412 // Bc=kMmaAtomK=16 → 1 iteration for kHeadDim≤64 configurations
3413 #pragma unroll
3414 for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
3415     // ldmatrix.x2.trans: load V[Bc,d] with transposition
3416     // V is row-major [Bc,d], but NN matmul needs B matrix in col-major → use
3417     // transposed ldmatrix
3418 #pragma unroll
3419 for (int j = 0; j < kValTileHeadDimV; ++j) {
3420     int warp_smem_V_d =
3421         warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
3422     int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
3423     int lane_smem_V_d = warp_smem_V_d;
3424     uint32_t lane_smem_V_ptr =
3425         smem_V_base_ptr +
3426         (lane_smem_V_Bc * (kHeadDim + kPad) + lane_smem_V_d) * sizeof(half);
3427     LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
3428 }
3429
3430 // P matrix layout in R_S[i][j][2]:
3431 // MMA = m16n8k16, Br=16x4=64, Bc=8x8=64, layout: 4 warps
3432 // | 64x64 | warp_KV 0 |
3433 // | warp_QP 0 | MMA 0 ... MMA 0 (x8) |
3434 // | warp_QP 1 | MMA 1 ... MMA 1 (x8) |
3435 // | warp_QP 2 | MMA 2 ... MMA 2 (x8) |
3436 // | warp_QP 3 | MMA 3 ... MMA 3 (x8) |
3437 // tile_V_Bc selects which 16-column slice of P to use:
3438 // tile_V_Bc=0 → cols 0:16 → MMA indices 0,1 → w=0
3439 // tile_V_Bc=1 → cols 16:32 → MMA indices 2,3 → w=2
3440 // tile_V_Bc=2 → cols 32:48 → MMA indices 4,5 → w=4
3441 // tile_V_Bc=3 → cols 48:64 → MMA indices 6,7 → w=6
3442 // 对应的 MMA A fragment 布局可以这样记:
3443 // - rows 0~7: lane 0~3 → {a0,a1} 与 {a4,a5}, lane 4~7 → 下一行, 依次类推
3444 // - rows 8~15: lane 0~3 → {a2,a3} 与 {a6,a7}, lane 4~7 → 下一行, 依次类推
3445 // 当前实现正是利用这一路径下 A fragment 与前面生成的 P fragment 可以直接对接,
3446 // 才能把 R_S 中的 P 直接喂给 HMMA16816 做 P@V; 复习时不要把它背成对所有 MMA
3447 // fragment 都无条件成立的通用结论。layout转换逻辑:
3448 // C layout: 8 x (16 x 8) layout → A layout: 4 x (16 x 16) layout
3449 int w = tile_V_Bc * 2;
3450 #pragma unroll
3451 for (int i = 0; i < kValTileSeqLenP; ++i) {
3452 #pragma unroll
3453 for (int j = 0; j < kValTileHeadDimV; ++j) {
3454     HMMA16816(R_0[i][j][0], R_0[i][j][1], // C fragment output
3455         R_S[i][w][0], R_S[i][w][1], R_S[i][w + 1][0], R_S[i][w + 1][1], // A fragment = P
3456         R_V[j][0], R_V[j][1], // B fragment = V
3457         R_0[i][j][0], R_0[i][j][1]); // C fragment output
3458     }
3459 }
3460 } // end for tile_V_Bc
3461 __syncthreads();
3462
3463 // =====
3464 // 3e: Online rescaling — 0_new = exp(m_old - m_new) * 0_old + P@V

```

```

3465 // =====
3466 // 公式来源: FA2 paper Eq.(7-8), 使用  $\exp(m_{old} - m_{new})$  做 0 与 l 的 rescale
3467 #pragma unroll
3468 for (int i = 0; i < kValTileSeqLenP; ++i) {
3469     float block_row_max_new_0 = lane_row_max_new[i][0];
3470     float block_row_max_new_1 = lane_row_max_new[i][1];
3471     float block_row_sum_new_0 = lane_row_sum_new[i][0];
3472     float block_row_sum_new_1 = lane_row_sum_new[i][1];
3473
3474     float block_row_max_old_0 = lane_block_row_max_old[i][0];
3475     float block_row_max_old_1 = lane_block_row_max_old[i][1];
3476
3477     block_row_max_new_0 = max(block_row_max_old_0, block_row_max_new_0);
3478     block_row_max_new_1 = max(block_row_max_old_1, block_row_max_new_1);
3479
3480     // Handle first iteration: m_old = -inf, need to use m_new directly
3481     block_row_max_old_0 =
3482         (tile_K_seqlen > 0 ? block_row_max_old_0 : block_row_max_new_0);
3483     block_row_max_old_1 =
3484         (tile_K_seqlen > 0 ? block_row_max_old_1 : block_row_max_new_1);
3485
3486     float rescale_o_factor_0 =
3487         __expf(block_row_max_old_0 - block_row_max_new_0);
3488     float rescale_o_factor_1 =
3489         __expf(block_row_max_old_1 - block_row_max_new_1);
3490
3491     // Rescale O_old + Add P@V in one fused step
3492 #pragma unroll
3493     for (int j = 0; j < kValTileHeadDimV; ++j) {
3494         // R_0 / R_D 与前面的 R_S 一样, 都按 MMA C fragment 布局解释:
3495         //   reg[0] -> rows 0~7 的 {c0,c1}
3496         //   reg[1] -> rows 8~15 的 {c2,c3}
3497         float2 t_reg_0_0 = __half22float2(HALF2(R_0[i][j][0]));
3498         float2 t_reg_0_1 = __half22float2(HALF2(R_0[i][j][1]));
3499         float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
3500         float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
3501
3502         // O_new =  $\exp(m_{old} - m_{new}) * O_{old} + P@V$  (fused multiply-add)
3503         t_reg_D_0.x = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.x, t_reg_0_0.x);
3504         t_reg_D_0.y = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.y, t_reg_0_0.y);
3505         t_reg_D_1.x = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.x, t_reg_0_1.x);
3506         t_reg_D_1.y = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.y, t_reg_0_1.y);
3507
3508         HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
3509         HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
3510     }
3511
3512     // Update l:  $l_{new} = \exp(m_{old} - m_{new}) * l_{old} + row\_sum(P)$ 
3513     float block_row_sum_old_0 = lane_block_row_sum_old[i][0];
3514     float block_row_sum_old_1 = lane_block_row_sum_old[i][1];
3515     lane_block_row_sum_old[i][0] = __fmaf_rn(
3516         rescale_o_factor_0, block_row_sum_old_0, block_row_sum_new_0);
3517     lane_block_row_sum_old[i][1] = __fmaf_rn(
3518         rescale_o_factor_1, block_row_sum_old_1, block_row_sum_new_1);
3519
3520     // Update m
3521     lane_block_row_max_old[i][0] = block_row_max_new_0;
3522     lane_block_row_max_old[i][1] = block_row_max_new_1;
3523 }
3524
3525 // Wait for next K tile to be ready in smem before next iteration
3526 if constexpr (kStage > 1) {
3527     if ((tile_K_seqlen + 1) < Tc) {

```

```

3528     CP_ASYNC_WAIT_GROUP(0);
3529 }
3530 __syncthreads();
3531 }
3532 } // end outer loop over K seqLen
3533 __syncthreads();
3534
3535 // =====
3536 // Step 4: 最终 rescale — 0_final = (1/l_final) * 0_final
3537 // =====
3538 #pragma unroll
3539 for (int i = 0; i < kValTileSeqLenP; ++i) {
3540     float rescale_factor_0 = __frcp_rn(lane_block_row_sum_old[i][0]);
3541     float rescale_factor_1 = __frcp_rn(lane_block_row_sum_old[i][1]);
3542 #pragma unroll
3543     for (int j = 0; j < kValTileHeadDimV; ++j) {
3544         float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
3545         float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
3546         t_reg_D_0.x = rescale_factor_0 * t_reg_D_0.x;
3547         t_reg_D_0.y = rescale_factor_0 * t_reg_D_0.y;
3548         t_reg_D_1.x = rescale_factor_1 * t_reg_D_1.x;
3549         t_reg_D_1.y = rescale_factor_1 * t_reg_D_1.y;
3550         HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
3551         HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
3552     }
3553 }
3554
3555 // =====
3556 // Step 5: Epilogue — Collective store via warp shuffle + 128-bit store
3557 // =====
3558 // 利用 warp shuffle 将分散在各 lane 的寄存器数据收集到 lane 0~3,
3559 // 然后用 LDST128BITS (st.global.v4.f32) 一次性写入 16 bytes
3560 #pragma unroll
3561 for (int i = 0; i < kValTileSeqLenP; ++i) {
3562 #pragma unroll
3563     for (int j = 0; j < kValTileHeadDimV; ++j) {
3564         uint32_t R_Z[2][4];
3565         R_Z[0][0] = R_D[i][j][0];
3566         R_Z[1][0] = R_D[i][j][1];
3567         R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
3568         R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
3569         R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
3570         R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
3571         R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);
3572         R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);
3573
3574         // st.global.v4.f32: 128-bit store, 4 lanes × 32-bit
3575         if (lane_id % 4 == 0) {
3576             int store_warp_regs_0_Br =
3577                 warp_QP * (kMmaAtomM * kValTileSeqLenP) + i * kMmaAtomM;
3578             int store_lane_gmem_0_Br =
3579                 Q_tile_id * Br + store_warp_regs_0_Br + lane_id / 4;
3580             int store_warp_regs_0_d =
3581                 warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
3582             int store_lane_gmem_0_d = store_warp_regs_0_d;
3583             int store_gmem_0_addr_0 = 0_gmem_offset +
3584                 (store_lane_gmem_0_Br + 0) * kHeadDim +
3585                 store_lane_gmem_0_d;
3586             int store_gmem_0_addr_1 = 0_gmem_offset +
3587                 (store_lane_gmem_0_Br + 8) * kHeadDim +
3588                 store_lane_gmem_0_d;
3589             // LDST128BITS = reinterpret_cast<float4*>
3590             *reinterpret_cast<float4*>(&0[store_gmem_0_addr_0]) =

```



```

3591     *reinterpret_cast<float4 *>(&R_Z[0][0]);
3592     *reinterpret_cast<float4 *>(&0[store_gmem_0_addr_1]) =
3593     *reinterpret_cast<float4 *>(&R_Z[1][0]);
3594 }
3595 }
3596 }
3597 }
3598
3599 // =====
3600 // 以下是测试代码, 验证 Phase 1 - Phase 8 的kernel的正确性, 不评估性能。
3601 // =====
3602
3603 static inline void check(cudaError_t err, const char *msg) {
3604     if (err != cudaSuccess) {
3605         fprintf(stderr, "[ERROR] %s: %s\n", msg, cudaGetErrorString(err));
3606         exit(EXIT_FAILURE);
3607     }
3608 }
3609
3610 static void test_block_reduce(int N) {
3611
3612     srand(42);
3613     float *h_a = (float *)malloc((size_t)N * sizeof(float));
3614     for (int i = 0; i < N; i++)
3615         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3616
3617     // CPU reference: sum of all elements
3618     double ref = 0.0;
3619     for (int i = 0; i < N; i++) ref += (double)h_a[i];
3620
3621     float *d_a, *d_y;
3622     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "blockreduce alloc A");
3623     check(cudaMalloc(&d_y, sizeof(float)), "blockreduce alloc Y");
3624
3625     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "blockreduce H2D A");
3626     check(cudaMemset(d_y, 0, sizeof(float)), "blockreduce zero Y");
3627
3628     dim3 block(128);
3629     dim3 grid((N + 127) / 128);
3630     block_reduce_all<128><<<grid, block>>>(d_a, d_y, N);
3631     check(cudaGetLastError(), "blockreduce launch");
3632     check(cudaDeviceSynchronize(), "blockreduce sync");
3633
3634     float result;
3635     check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "blockreduce D2H");
3636
3637     float err = fabsf(result - (float)ref);
3638     printf("| %-35s | %.6e | %-4s |\n", "BlockReduce", err,
3639         err < 1e-2f ? "PASS" : "FAIL");
3640
3641     free(h_a);
3642     cudaFree(d_a); cudaFree(d_y);
3643 }
3644
3645 static void test_dot(int N) {
3646
3647     srand(42);
3648     float *h_a = (float *)malloc((size_t)N * sizeof(float));
3649     float *h_b = (float *)malloc((size_t)N * sizeof(float));
3650     for (int i = 0; i < N; i++) {
3651         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;

```

```

3654     h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3655 }
3656
3657 // CPU reference
3658 double ref = 0.0;
3659 for (int i = 0; i < N; i++) ref += (double)h_a[i] * (double)h_b[i];
3660
3661 float *d_a, *d_b, *d_y;
3662 check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "dot alloc A");
3663 check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "dot alloc B");
3664 check(cudaMalloc(&d_y, sizeof(float)), "dot alloc Y");
3665
3666 check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D A");
3667 check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D B");
3668 check(cudaMemset(d_y, 0, sizeof(float)), "dot zero Y");
3669
3670 dim3 block(128);
3671 dim3 grid((N + 127) / 128);
3672 dot<128><<<grid, block>>>(d_a, d_b, d_y, N);
3673 check(cudaGetLastError(), "dot launch");
3674 check(cudaDeviceSynchronize(), "dot sync");
3675
3676 float result;
3677 check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot D2H");
3678
3679 float err = fabsf(result - (float)ref);
3680 printf("| %-35s | %.6e | %-4s |\n", "Dot", err,
3681     err < 1e-2f ? "PASS" : "FAIL");
3682
3683 // ---- Dot Vec4 ----
3684 check(cudaMemset(d_y, 0, sizeof(float)), "dot_vec4 zero Y");
3685 dim3 block_v4(32);
3686 dot_vec4<32><<<grid, block_v4>>>(d_a, d_b, d_y, N);
3687 check(cudaGetLastError(), "dot_vec4 launch");
3688 check(cudaDeviceSynchronize(), "dot_vec4 sync");
3689
3690 check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot_vec4 D2H");
3691 float err_v4 = fabsf(result - (float)ref);
3692 printf("| %-35s | %.6e | %-4s |\n", "Dot-Vec4", err_v4, err_v4 < 1e-2f ? "PASS" : "FAIL");
3693
3694 free(h_a); free(h_b);
3695 cudaFree(d_a); cudaFree(d_b); cudaFree(d_y);
3696 }
3697
3698
3699 static void test_relu(int N) {
3700     srand(42);
3701     float *h_x = (float *)malloc((size_t)N * sizeof(float));
3702     float *h_y = (float *)malloc((size_t)N * sizeof(float));
3703     for (int i = 0; i < N; i++)
3704         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3705
3706     float *d_x, *d_y;
3707     check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "relu alloc X");
3708     check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "relu alloc Y");
3709     check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "relu H2D");
3710
3711     for (int i = 0; i < N; i++) h_y[i] = fmaxf(0.0f, h_x[i]);
3712     dim3 block256(256);
3713     dim3 grid256((N + 255) / 256);
3714     relu<<<grid256, block256>>>(d_x, d_y, N);
3715     check(cudaGetLastError(), "relu launch");
3716     check(cudaDeviceSynchronize(), "relu sync");

```

```

3717 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu D2H");
3718 float max_err = 0.0f;
3719 for (int i = 0; i < N; i++) {
3720     float expected = fmaxf(0.0f, h_x[i]);
3721     float err = fabsf(h_y[i] - expected);
3722     if (err > max_err) max_err = err;
3723 }
3724 printf("| %-35s | %.6e | %-4s |\n", "ReLU", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3725
3726 dim3 block64(64);
3727 relu_vec4<<<grid256, block64>>>(d_x, d_y, N);
3728 check(cudaGetLastError(), "relu_vec4 launch");
3729 check(cudaDeviceSynchronize(), "relu_vec4 sync");
3730 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu_vec4 D2H");
3731 max_err = 0.0f;
3732 for (int i = 0; i < N; i++) {
3733     float expected = fmaxf(0.0f, h_x[i]);
3734     float err = fabsf(h_y[i] - expected);
3735     if (err > max_err) max_err = err;
3736 }
3737 printf("| %-35s | %.6e | %-4s |\n", "ReLU-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3738
3739 free(h_x); free(h_y);
3740 cudaFree(d_x); cudaFree(d_y);
3741 }
3742
3743
3744 static void test_elementwise(int N) {
3745     srand(42);
3746     float *h_a = (float *)malloc((size_t)N * sizeof(float));
3747     float *h_b = (float *)malloc((size_t)N * sizeof(float));
3748     for (int i = 0; i < N; i++) {
3749         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3750         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3751     }
3752
3753     float *d_a, *d_b, *d_c;
3754     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "eadd alloc A");
3755     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "eadd alloc B");
3756     check(cudaMalloc(&d_c, (size_t)N * sizeof(float)), "eadd alloc C");
3757     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D A");
3758     check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D B");
3759
3760     dim3 block256(256);
3761     dim3 grid256((N + 255) / 256);
3762     elementwise_add<<<grid256, block256>>>(d_a, d_b, d_c, N);
3763     check(cudaGetLastError(), "eadd launch");
3764     check(cudaDeviceSynchronize(), "eadd sync");
3765     float *h_c = (float *)malloc((size_t)N * sizeof(float));
3766     check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd D2H");
3767     float max_err = 0.0f;
3768     for (int i = 0; i < N; i++) {
3769         float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
3770         if (err > max_err) max_err = err;
3771     }
3772     printf("| %-35s | %.6e | %-4s |\n", "ElemwiseAdd", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3773
3774     dim3 block64(64);
3775     check(cudaMemset(d_c, 0, (size_t)N * sizeof(float)), "eadd_vec4 zero C");
3776     elementwise_add_vec4<<<grid256, block64>>>(d_a, d_b, d_c, N);
3777     check(cudaGetLastError(), "eadd_vec4 launch");
3778     check(cudaDeviceSynchronize(), "eadd_vec4 sync");
3779     check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd_vec4 D2H");

```

```

3780 max_err = 0.0f;
3781 for (int i = 0; i < N; i++) {
3782     float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
3783     if (err > max_err) max_err = err;
3784 }
3785 printf("| %-35s | %.6e | %-4s |\n", "ElemwiseAdd-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3786
3787 free(h_a); free(h_b); free(h_c);
3788 cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
3789 }
3790
3791
3792 static void test_histogram(int N) {
3793     srand(42);
3794     int BINS = 16;
3795     int *h_hist = (int *)calloc(BINS, sizeof(int));
3796     int *h_hist_ref = (int *)calloc(BINS, sizeof(int));
3797     int *h_idx = (int *)malloc((size_t)N * sizeof(int));
3798     for (int i = 0; i < N; i++) h_idx[i] = rand() % BINS;
3799     for (int i = 0; i < N; i++) h_hist_ref[h_idx[i]]++;
3800
3801     int *d_idx, *d_hist;
3802     check(cudaMalloc(&d_idx, (size_t)N * sizeof(int)), "hist alloc idx");
3803     check(cudaMalloc(&d_hist, BINS * sizeof(int)), "hist alloc hist");
3804     check(cudaMemcpy(d_idx, h_idx, (size_t)N * sizeof(int), cudaMemcpyHostToDevice), "hist H2D idx");
3805     check(cudaMemset(d_hist, 0, BINS * sizeof(int)), "hist zero");
3806
3807     dim3 block256(256);
3808     dim3 grid256((N + 255) / 256);
3809     histogram<<<grid256, block256>>>(d_idx, d_hist, N);
3810     check(cudaGetLastError(), "histogram launch");
3811     check(cudaDeviceSynchronize(), "histogram sync");
3812     check(cudaMemcpy(h_hist, d_hist, BINS * sizeof(int), cudaMemcpyDeviceToHost), "hist D2H");
3813
3814     float max_err = 0.0f;
3815     for (int i = 0; i < BINS; i++) {
3816         float err = fabsf((float)(h_hist[i] - h_hist_ref[i]));
3817         if (err > max_err) max_err = err;
3818     }
3819     printf("| %-35s | %.6e | %-4s |\n", "Histogram", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3820
3821     free(h_hist); free(h_hist_ref); free(h_idx);
3822     cudaFree(d_idx); cudaFree(d_hist);
3823 }
3824
3825
3826 static void test_merge_attn_states(int num_tokens, int num_heads,
3827                                   int head_size) {
3828
3829     size_t out_size = (size_t)num_tokens * num_heads * head_size * sizeof(float);
3830     size_t lse_size = (size_t)num_heads * num_tokens * sizeof(float);
3831
3832     float *h_prefix_out = (float *)malloc(out_size);
3833     float *h_suffix_out = (float *)malloc(out_size);
3834     float *h_prefix_lse = (float *)malloc(lse_size);
3835     float *h_suffix_lse = (float *)malloc(lse_size);
3836     float *h_output_ref = (float *)malloc(out_size);
3837
3838     srand(42);
3839     for (int i = 0; i < num_tokens * num_heads * head_size; i++) {
3840         h_prefix_out[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3841         h_suffix_out[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3842     }

```

```

3843 for (int i = 0; i < num_heads * num_tokens; i++) {
3844     h_prefix_lse[i] = ((float)rand() / RAND_MAX) * 20.0f - 10.0f;
3845     h_suffix_lse[i] = ((float)rand() / RAND_MAX) * 20.0f - 10.0f;
3846 }
3847
3848 // CPU reference: 与 kernel 完全相同的浮点计算
3849 for (int t = 0; t < num_tokens; t++) {
3850     for (int h = 0; h < num_heads; h++) {
3851         float p_lse = h_prefix_lse[h * num_tokens + t];
3852         float s_lse = h_suffix_lse[h * num_tokens + t];
3853         p_lse = isinf(p_lse) ? -INFINITY : p_lse;
3854         s_lse = isinf(s_lse) ? -INFINITY : s_lse;
3855
3856         float max_lse = fmaxf(p_lse, s_lse);
3857         p_lse -= max_lse;
3858         s_lse -= max_lse;
3859         float p_se = expf(p_lse);
3860         float s_se = expf(s_lse);
3861         float p_scale = p_se / (p_se + s_se);
3862         float s_scale = s_se / (p_se + s_se);
3863
3864         int head_off = t * num_heads * head_size + h * head_size;
3865         for (int d = 0; d < head_size; d++) {
3866             h_output_ref[head_off + d] =
3867                 h_prefix_out[head_off + d] * p_scale +
3868                 h_suffix_out[head_off + d] * s_scale;
3869         }
3870     }
3871 }
3872
3873 float *d_prefix_out, *d_suffix_out, *d_prefix_lse, *d_suffix_lse, *d_output;
3874 check(cudaMalloc(&d_prefix_out, out_size), "merge_attn alloc p_out");
3875 check(cudaMalloc(&d_suffix_out, out_size), "merge_attn alloc s_out");
3876 check(cudaMalloc(&d_prefix_lse, lse_size), "merge_attn alloc p_lse");
3877 check(cudaMalloc(&d_suffix_lse, lse_size), "merge_attn alloc s_lse");
3878 check(cudaMalloc(&d_output, out_size), "merge_attn alloc output");
3879
3880 check(cudaMemcpy(d_prefix_out, h_prefix_out, out_size,
3881                 cudaMemcpyHostToDevice),
3882        "merge_attn H2D p_out");
3883 check(cudaMemcpy(d_suffix_out, h_suffix_out, out_size,
3884                 cudaMemcpyHostToDevice),
3885        "merge_attn H2D s_out");
3886 check(cudaMemcpy(d_prefix_lse, h_prefix_lse, lse_size,
3887                 cudaMemcpyHostToDevice),
3888        "merge_attn H2D p_lse");
3889 check(cudaMemcpy(d_suffix_lse, h_suffix_lse, lse_size,
3890                 cudaMemcpyHostToDevice),
3891        "merge_attn H2D s_lse");
3892
3893 int threads_per_head = head_size / 4;
3894 int total_threads = num_tokens * num_heads * threads_per_head;
3895 dim3 block(128);
3896 dim3 grid((total_threads + 127) / 128);
3897 merge_attn_states<<<grid, block>>>(
3898     d_output, d_prefix_out, d_prefix_lse, d_suffix_out, d_suffix_lse,
3899     num_tokens, num_heads, head_size);
3900 check(cudaGetLastError(), "merge_attn launch");
3901 check(cudaDeviceSynchronize(), "merge_attn sync");
3902
3903 float *h_output = (float *)malloc(out_size);
3904 check(cudaMemcpy(h_output, d_output, out_size, cudaMemcpyDeviceToHost),
3905        "merge_attn D2H");

```

```

3906 float max_err = 0.0f;
3907 for (int i = 0; i < num_tokens * num_heads * head_size; i++) {
3908     float err = fabsf(h_output[i] - h_output_ref[i]);
3909     if (err > max_err) max_err = err;
3910 }
3911 printf("| %-35s | %.6e | %-4s |\n", "MergeAttnStates", max_err,
3912        max_err < 1e-4f ? "PASS" : "FAIL");
3913
3914 // 边界测试: +inf LSE → 权重退化为 0 (空 attention 段)
3915 h_prefix_lse[0] = INFINITY; // head 0, token 0 的 LSE = +inf
3916 h_suffix_lse[0] = 0.0f;
3917 check(cudaMemcpy(d_prefix_lse, h_prefix_lse, lse_size,
3918                 cudaMemcpyHostToDevice),
3919        "merge_attn H2D inf lse");
3920 merge_attn_states<<<grid, block>>>(
3921     d_output, d_prefix_out, d_prefix_lse, d_suffix_out, d_suffix_lse,
3922     num_tokens, num_heads, head_size);
3923 check(cudaGetLastError(), "merge_attn inf launch");
3924 check(cudaDeviceSynchronize(), "merge_attn inf sync");
3925 check(cudaMemcpy(h_output, d_output, out_size, cudaMemcpyDeviceToHost),
3926        "merge_attn inf D2H");
3927
3928 // token 0, head 0 的所有元素应等于 suffix_output (prefix 权重 α=0)
3929 float inf_err = 0.0f;
3930 for (int d = 0; d < head_size; d++) {
3931     float err = fabsf(h_output[d] - h_suffix_out[d]);
3932     if (err > inf_err) inf_err = err;
3933 }
3934 printf("| %-35s | %.6e | %-4s |\n", "MergeAttnStates-inf", inf_err,
3935        inf_err < 1e-4f ? "PASS" : "FAIL");
3936
3937 free(h_prefix_out);
3938 free(h_suffix_out);
3939 free(h_prefix_lse);
3940 free(h_suffix_lse);
3941 free(h_output_ref);
3942 free(h_output);
3943 cudaFree(d_prefix_out);
3944 cudaFree(d_suffix_out);
3945 cudaFree(d_prefix_lse);
3946 cudaFree(d_suffix_lse);
3947 cudaFree(d_output);
3948 }
3949
3950
3951 static void test_softmax(int N) {
3952     // Use N = blockDim.x so one block processes all elements as one token.
3953     constexpr int NUM_THREADS = 256;
3954     if (N != NUM_THREADS) {
3955         printf("  OnlineSafeSoftmax: N must be %d (got %d), skipping.\n", NUM_THREADS, N);
3956         return;
3957     }
3958
3959     srand(42);
3960     float *h_x = (float *)malloc((size_t)N * sizeof(float));
3961     float *h_y_ref = (float *)malloc((size_t)N * sizeof(float));
3962     for (int i = 0; i < N; i++)
3963         h_x[i] = ((float)rand() / RAND_MAX) * 10.0f - 5.0f; // [-5, 5]
3964
3965     // CPU reference: softmax(x_i) = exp(x_i - max) / sum(exp(x_j - max))
3966     float max_val = -FLT_MAX;
3967     for (int i = 0; i < N; i++) if (h_x[i] > max_val) max_val = h_x[i];

```



```

3969 double sum_exp = 0.0;
3970 for (int i = 0; i < N; i++) sum_exp += (double)expf(h_x[i] - max_val);
3971 for (int i = 0; i < N; i++)
3972     h_y_ref[i] = expf(h_x[i] - max_val) / (float)sum_exp;
3973
3974 float *d_x, *d_y;
3975 check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "softmax alloc X");
3976 check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "softmax alloc Y");
3977
3978 check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "softmax H2D X");
3979
3980 dim3 block(256);
3981 dim3 grid(1); // one token covering all N elements
3982 online_safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
3983 check(cudaGetLastError(), "softmax launch");
3984 check(cudaDeviceSynchronize(), "softmax sync");
3985
3986 float *h_y = (float *)malloc((size_t)N * sizeof(float));
3987 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "softmax D2H");
3988
3989 float max_err = 0.0f;
3990 for (int i = 0; i < N; i++) {
3991     float err = fabsf(h_y[i] - h_y_ref[i]);
3992     if (err > max_err) max_err = err;
3993 }
3994 printf("| %-35s | %.6e | %-4s |\n", "OnlineSafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3995
3996 // ---- Safe Softmax ----
3997 safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
3998 check(cudaGetLastError(), "safe_softmax launch");
3999 check(cudaDeviceSynchronize(), "safe_softmax sync");
4000 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "safe_softmax D2H");
4001 max_err = 0.0f;
4002 for (int i = 0; i < N; i++) {
4003     float err = fabsf(h_y[i] - h_y_ref[i]);
4004     if (err > max_err) max_err = err;
4005 }
4006 printf("| %-35s | %.6e | %-4s |\n", "SafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4007
4008 // ---- Naive Softmax ----
4009 softmax_per_token<<<grid, block>>>(d_x, d_y, N);
4010 check(cudaGetLastError(), "naive_softmax launch");
4011 check(cudaDeviceSynchronize(), "naive_softmax sync");
4012 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "naive_softmax D2H");
4013 max_err = 0.0f;
4014 for (int i = 0; i < N; i++) {
4015     float err = fabsf(h_y[i] - h_y_ref[i]);
4016     if (err > max_err) max_err = err;
4017 }
4018 printf("| %-35s | %.6e | %-4s |\n", "NaiveSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4019
4020 free(h_x); free(h_y); free(h_y_ref);
4021 cudaFree(d_x); cudaFree(d_y);
4022 }
4023
4024
4025 static void test_rms_norm(int N, int K) {
4026
4027     srand(42);
4028     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
4029     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
4030     for (int i = 0; i < N * K; i++)
4031         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;

```

```

4032 float g = 1.5f; // gain
4033
4034 // CPU reference: y = (x / rms(x)) * g
4035 float epsilon = 1e-5f;
4036 for (int n = 0; n < N; n++) {
4037     double sum_sq = 0.0;
4038     for (int k = 0; k < K; k++) sum_sq += (double)h_x[n * K + k] * (double)h_x[n * K + k];
4039     float rms = sqrtf((float)sum_sq / (float)K + epsilon);
4040     for (int k = 0; k < K; k++)
4041         h_y_ref[n * K + k] = (h_x[n * K + k] / rms) * g;
4042 }
4043
4044 float *d_x, *d_y;
4045 check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "rmsnorm alloc X");
4046 check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "rmsnorm alloc Y");
4047 check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "rmsnorm H2D X");
4048
4049 dim3 block(128);
4050 dim3 grid(N);
4051 rms_norm<<<grid, block>>>(d_x, d_y, g, N, K);
4052 check(cudaGetLastError(), "rmsnorm launch");
4053 check(cudaDeviceSynchronize(), "rmsnorm sync");
4054
4055 float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
4056 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm D2H");
4057
4058 float max_err = 0.0f;
4059 for (int i = 0; i < N * K; i++) {
4060     float err = fabsf(h_y[i] - h_y_ref[i]);
4061     if (err > max_err) max_err = err;
4062 }
4063 printf("| %-35s | %.6e | %-4s |\n", "RMSNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4064
4065 // ---- RMS Norm Vec4 ----
4066 dim3 block_rv4(32);
4067 rms_norm_vec4<<<grid, block_rv4>>>(d_x, d_y, g, N, K);
4068 check(cudaGetLastError(), "rmsnorm_vec4 launch");
4069 check(cudaDeviceSynchronize(), "rmsnorm_vec4 sync");
4070 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm_vec4 D2H");
4071 max_err = 0.0f;
4072 for (int i = 0; i < N * K; i++) {
4073     float err = fabsf(h_y[i] - h_y_ref[i]);
4074     if (err > max_err) max_err = err;
4075 }
4076 printf("| %-35s | %.6e | %-4s |\n", "RMSNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4077
4078 free(h_x); free(h_y); free(h_y_ref);
4079 cudaFree(d_x); cudaFree(d_y);
4080 }
4081
4082
4083 static void test_layer_norm(int N, int K) {
4084
4085     srand(42);
4086     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
4087     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
4088     for (int i = 0; i < N * K; i++)
4089         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4090     float g = 1.5f, b = 0.3f; // gain and bias
4091
4092     // CPU reference: y = ((x - mean) / std) * g + b
4093     float epsilon = 1e-5f;
4094     for (int n = 0; n < N; n++) {

```

```

4095 double sum = 0.0;
4096 for (int k = 0; k < K; k++) sum += (double)h_x[n * K + k];
4097 float mean = (float)sum / (float)K;
4098 double sum_sq = 0.0;
4099 for (int k = 0; k < K; k++) {
4100     float diff = h_x[n * K + k] - mean;
4101     sum_sq += (double)diff * (double)diff;
4102 }
4103 float std = sqrtf((float)sum_sq / (float)K + epsilon);
4104 for (int k = 0; k < K; k++)
4105     h_y_ref[n * K + k] = ((h_x[n * K + k] - mean) / std) * g + b;
4106 }
4107
4108 float *d_x, *d_y;
4109 check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "layernorm alloc X");
4110 check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "layernorm alloc Y");
4111 check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "layernorm H2D X");
4112
4113 dim3 block(128);
4114 dim3 grid(N);
4115 layer_norm<<<grid, block>>>(d_x, d_y, g, b, N, K);
4116 check(cudaGetLastError(), "layernorm launch");
4117 check(cudaDeviceSynchronize(), "layernorm sync");
4118
4119 float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
4120 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm D2H");
4121
4122 float max_err = 0.0f;
4123 for (int i = 0; i < N * K; i++) {
4124     float err = fabsf(h_y[i] - h_y_ref[i]);
4125     if (err > max_err) max_err = err;
4126 }
4127 printf("| %-35s | %.6e | %-4s |\n", "LayerNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4128
4129 // ---- Layer Norm Vec4 ----
4130 dim3 block_lv4(32);
4131 layer_norm_vec4<<<grid, block_lv4>>>(d_x, d_y, g, b, N, K);
4132 check(cudaGetLastError(), "layernorm_vec4 launch");
4133 check(cudaDeviceSynchronize(), "layernorm_vec4 sync");
4134 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm_vec4 D2H");
4135 max_err = 0.0f;
4136 for (int i = 0; i < N * K; i++) {
4137     float err = fabsf(h_y[i] - h_y_ref[i]);
4138     if (err > max_err) max_err = err;
4139 }
4140 printf("| %-35s | %.6e | %-4s |\n", "LayerNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4141
4142 free(h_x); free(h_y); free(h_y_ref);
4143 cudaFree(d_x); cudaFree(d_y);
4144 }
4145
4146
4147 static void test_rope(int seq_len, int N) {
4148
4149     int total_pairs = seq_len * N;
4150     int total_elems = total_pairs * 2;
4151     size_t size = (size_t)total_elems * sizeof(float);
4152
4153     float *h_x = (float *)malloc(size);
4154     float *h_y_ref = (float *)malloc(size);
4155
4156     srand(42);
4157     for (int i = 0; i < total_elems; i++)

```

```

4158     h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4159
4160     // CPU reference: 2D rotation for each pair
4161     for (int idx = 0; idx < total_pairs; idx++) {
4162         int token_pos = idx / N;
4163         int token_idx = idx % N;
4164         float x1 = h_x[idx * 2];
4165         float x2 = h_x[idx * 2 + 1];
4166         float theta = 1.0f / powf(10000.0f, 2.0f * token_idx / (N * 2.0f));
4167         float angle = (float)token_pos * theta;
4168         float cos_v = cosf(angle);
4169         float sin_v = sinf(angle);
4170         h_y_ref[idx * 2] = x1 * cos_v - x2 * sin_v;
4171         h_y_ref[idx * 2 + 1] = x1 * sin_v + x2 * cos_v;
4172     }
4173
4174     float *d_x, *d_y;
4175     check(cudaMalloc(&d_x, size), "rope alloc X");
4176     check(cudaMalloc(&d_y, size), "rope alloc Y");
4177     check(cudaMemcpy(d_x, h_x, size, cudaMemcpyHostToDevice), "rope H2D");
4178
4179     dim3 block(256);
4180     dim3 grid((total_pairs + 255) / 256);
4181     rope<<<grid, block>>>(d_x, d_y, seq_len, N);
4182     check(cudaGetLastError(), "rope launch");
4183     check(cudaDeviceSynchronize(), "rope sync");
4184
4185     float *h_y = (float *)malloc(size);
4186     check(cudaMemcpy(h_y, d_y, size, cudaMemcpyDeviceToHost), "rope D2H");
4187
4188     float max_err = 0.0f;
4189     for (int i = 0; i < total_elems; i++) {
4190         float err = fabsf(h_y[i] - h_y_ref[i]);
4191         if (err > max_err) max_err = err;
4192     }
4193     printf("| %-35s | %.6e | %-4s |\n", "RoPE", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4194
4195     free(h_x);
4196     free(h_y);
4197     free(h_y_ref);
4198     cudaFree(d_x);
4199     cudaFree(d_y);
4200 }
4201
4202
4203 static void test_mat_transpose(int row, int col) {
4204
4205     size_t size_in = (size_t)row * col * sizeof(float);
4206     size_t size_out = (size_t)col * row * sizeof(float);
4207
4208     float *h_x = (float *)malloc(size_in);
4209     float *h_y_ref = (float *)malloc(size_out);
4210
4211     srand(42);
4212     for (int i = 0; i < row * col; i++)
4213         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4214
4215     // CPU reference: y[j][i] = x[i][j] (row-major)
4216     for (int i = 0; i < row; i++)
4217         for (int j = 0; j < col; j++)
4218             h_y_ref[j * row + i] = h_x[i * col + j];
4219
4220     float *d_x, *d_y;

```

```

4221 check(cudaMalloc(&d_x, size_in), "mattrans alloc X");
4222 check(cudaMalloc(&d_y, size_out), "mattrans alloc Y");
4223 check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans H2D");
4224
4225 dim3 block(16, 16);
4226 dim3 grid((col + 15) / 16, (row + 15) / 16);
4227 mat_transpose<<<grid, block>>>(d_x, d_y, row, col);
4228 check(cudaGetLastError(), "mattrans launch");
4229 check(cudaDeviceSynchronize(), "mattrans sync");
4230
4231 float *h_y = (float *)malloc(size_out);
4232 check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans D2H");
4233
4234 float max_err = 0.0f;
4235 for (int i = 0; i < col * row; i++) {
4236     float err = fabsf(h_y[i] - h_y_ref[i]);
4237     if (err > max_err) max_err = err;
4238 }
4239 printf("| %-35s | %.6e | %-4s |\n", "MatTranspose", max_err, max_err < 1e-6f ? "PASS" : "FAIL");
4240
4241 free(h_x);
4242 free(h_y);
4243 free(h_y_ref);
4244 cudaFree(d_x);
4245 cudaFree(d_y);
4246 }
4247
4248
4249 static void test_mat_transpose_padded(int row, int col) {
4250
4251     size_t size_in = (size_t)row * col * sizeof(float);
4252     size_t size_out = (size_t)col * row * sizeof(float);
4253
4254     float *h_x = (float *)malloc(size_in);
4255     float *h_y_ref = (float *)malloc(size_out);
4256
4257     srand(42);
4258     for (int i = 0; i < row * col; i++)
4259         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4260
4261     // CPU reference: y[j][i] = x[i][j] (row-major)
4262     for (int i = 0; i < row; i++)
4263         for (int j = 0; j < col; j++)
4264             h_y_ref[j * row + i] = h_x[i * col + j];
4265
4266     float *d_x, *d_y;
4267     check(cudaMalloc(&d_x, size_in), "mattrans_padded alloc X");
4268     check(cudaMalloc(&d_y, size_out), "mattrans_padded alloc Y");
4269     check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans_padded H2D");
4270
4271     dim3 block(16, 16);
4272     dim3 grid((col + 15) / 16, (row + 63) / 64);
4273     mat_transpose_padded<<<grid, block>>>(d_x, d_y, row, col);
4274     check(cudaGetLastError(), "mattrans_padded launch");
4275     check(cudaDeviceSynchronize(), "mattrans_padded sync");
4276
4277     float *h_y = (float *)malloc(size_out);
4278     check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans_padded D2H");
4279
4280     float max_err = 0.0f;
4281     for (int i = 0; i < col * row; i++) {
4282         float err = fabsf(h_y[i] - h_y_ref[i]);
4283         if (err > max_err) max_err = err;

```

```

4284 }
4285 printf("| %-35s | %.6e | %-4s |\n", "MatTransposePadded", max_err, max_err < 1e-6f ? "PASS" : "FAIL");
4286
4287 free(h_x);
4288 free(h_y);
4289 free(h_y_ref);
4290 cudaFree(d_x);
4291 cudaFree(d_y);
4292 }
4293
4294
4295 static void test_sgemv(int M, int K) {
4296
4297     srand(42);
4298     float *h_a = (float *)malloc((size_t)M * K * sizeof(float));
4299     float *h_x = (float *)malloc((size_t)K * sizeof(float));
4300     float *h_y_ref = (float *)malloc((size_t)M * sizeof(float));
4301     for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4302     for (int i = 0; i < K; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4303
4304     // CPU reference: y[m] = sum_k A[m,k] * x[k]
4305     for (int m = 0; m < M; m++) {
4306         double sum = 0.0;
4307         for (int k = 0; k < K; k++) sum += (double)h_a[m * K + k] * (double)h_x[k];
4308         h_y_ref[m] = (float)sum;
4309     }
4310
4311     float *d_a, *d_x, *d_y;
4312     check(cudaMalloc(&d_a, (size_t)M * K * sizeof(float)), "sgemv alloc A");
4313     check(cudaMalloc(&d_x, (size_t)K * sizeof(float)), "sgemv alloc X");
4314     check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv alloc Y");
4315
4316     check(cudaMemcpy(d_a, h_a, (size_t)M * K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D A");
4317     check(cudaMemcpy(d_x, h_x, (size_t)K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D X");
4318
4319     dim3 block(32, 4);
4320     dim3 grid((M + 3) / 4);
4321     sgemv_k128<<<grid, block>>>(d_a, d_x, d_y, M, K);
4322     check(cudaGetLastError(), "sgemv launch");
4323     check(cudaDeviceSynchronize(), "sgemv sync");
4324
4325     float *h_y = (float *)malloc((size_t)M * sizeof(float));
4326     check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv D2H");
4327
4328     float max_err = 0.0f;
4329     for (int m = 0; m < M; m++) {
4330         float err = fabsf(h_y[m] - h_y_ref[m]);
4331         if (err > max_err) max_err = err;
4332     }
4333     printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K128", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4334
4335     // ---- SGEMV K32 ----
4336     check(cudaMemset(d_y, 0, M * sizeof(float)), "sgemv_k32 zero Y");
4337     sgemv_k32<<<grid, block>>>(d_a, d_x, d_y, M, K);
4338     check(cudaGetLastError(), "sgemv_k32 launch");
4339     check(cudaDeviceSynchronize(), "sgemv_k32 sync");
4340
4341     check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k32 D2H");
4342
4343     max_err = 0.0f;
4344     for (int m = 0; m < M; m++) {
4345         float err = fabsf(h_y[m] - h_y_ref[m]);
4346         if (err > max_err) max_err = err;

```

```

4347 }
4348 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K32", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4349
4350 // ---- SGEMV K16 ----
4351 free(h_a); free(h_x); free(h_y); free(h_y_ref);
4352 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
4353
4354 int K16 = 16;
4355 h_a = (float *)malloc((size_t)M * K16 * sizeof(float));
4356 h_x = (float *)malloc((size_t)K16 * sizeof(float));
4357 h_y_ref = (float *)malloc((size_t)M * sizeof(float));
4358 for (int i = 0; i < M * K16; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4359 for (int i = 0; i < K16; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4360
4361 for (int m = 0; m < M; m++) {
4362     double sum = 0.0;
4363     for (int k = 0; k < K16; k++) sum += (double)h_a[m * K16 + k] * (double)h_x[k];
4364     h_y_ref[m] = (float)sum;
4365 }
4366
4367 check(cudaMalloc(&d_a, (size_t)M * K16 * sizeof(float)), "sgemv_k16 alloc A");
4368 check(cudaMalloc(&d_x, (size_t)K16 * sizeof(float)), "sgemv_k16 alloc X");
4369 check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv_k16 alloc Y");
4370
4371 check(cudaMemcpy(d_a, h_a, (size_t)M * K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D A");
4372 check(cudaMemcpy(d_x, h_x, (size_t)K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D X");
4373
4374 dim3 grid_k16((M + 7) / 8);
4375 sgemv_k16<2><<<grid_k16, block>>>(d_a, d_x, d_y, M, K16);
4376 check(cudaGetLastError(), "sgemv_k16 launch");
4377 check(cudaDeviceSynchronize(), "sgemv_k16 sync");
4378
4379 h_y = (float *)malloc((size_t)M * sizeof(float));
4380 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k16 D2H");
4381
4382 max_err = 0.0f;
4383 for (int m = 0; m < M; m++) {
4384     float err = fabsf(h_y[m] - h_y_ref[m]);
4385     if (err > max_err) max_err = err;
4386 }
4387 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K16", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4388
4389 free(h_a); free(h_x); free(h_y); free(h_y_ref);
4390 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
4391 }
4392
4393
4394 static void test_sgemm(int M, int N, int K) {
4395
4396     size_t size_a = (size_t)M * K * sizeof(float);
4397     size_t size_b = (size_t)K * N * sizeof(float);
4398     size_t size_c = (size_t)M * N * sizeof(float);
4399
4400     float *h_a = (float *)malloc(size_a);
4401     float *h_b = (float *)malloc(size_b);
4402     float *h_c_ref = (float *)malloc(size_c);
4403
4404     srand(42);
4405     for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4406     for (int i = 0; i < K * N; i++) h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4407
4408     float *d_a, *d_b, *d_c;
4409     check(cudaMalloc(&d_a, size_a), "sgemm alloc A");

```



```

4410 check(cudaMalloc(&d_b, size_b), "sgemm alloc B");
4411 check(cudaMalloc(&d_c, size_c), "sgemm alloc C");
4412
4413 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "sgemm H2D A");
4414 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "sgemm H2D B");
4415
4416 // cuBLAS reference (row-major idiom: swap M/N, swap A/B)
4417 cublasHandle_t handle;
4418 cublasCreate(&handle);
4419 float alpha = 1.0f, beta = 0.0f;
4420 cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
4421             &alpha, d_b, N, d_a, K, &beta, d_c, N);
4422 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H ref");
4423
4424 // Kernel
4425 cudaEvent_t start, stop;
4426 cudaEventCreate(&start);
4427 cudaEventCreate(&stop);
4428
4429 dim3 block(32, 32);
4430 dim3 grid((N + 31) / 32, (M + 31) / 32);
4431 sgemm<<<grid, block>>>(d_a, d_b, d_c, M, N, K);
4432 check(cudaGetLastError(), "sgemm launch");
4433 check(cudaDeviceSynchronize(), "sgemm sync");
4434
4435 float *h_c = (float *)malloc(size_c);
4436 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H");
4437
4438 // Verify
4439 float max_err = 0.0f;
4440 for (int i = 0; i < M * N; i++) {
4441     float err = fabsf(h_c[i] - h_c_ref[i]);
4442     if (err > max_err) max_err = err;
4443 }
4444 printf("| %-35s | %.6e | %-4s |\n", "SGEMM", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4445
4446 // ---- SGEMM Vec4 (128x128 tile, 4x4 thread tile) ----
4447
4448 dim3 grid_vec4((N + 127) / 128, (M + 127) / 128);
4449 check(cudaMemset(d_c, 0, size_c), "sgemm_vec4 zero C");
4450 sgemm_vec4<<<grid_vec4, block>>>(d_a, d_b, d_c, M, N, K);
4451 check(cudaGetLastError(), "sgemm_vec4 launch");
4452 check(cudaDeviceSynchronize(), "sgemm_vec4 sync");
4453
4454 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm_vec4 D2H");
4455
4456 max_err = 0.0f;
4457 for (int i = 0; i < M * N; i++) {
4458     float err = fabsf(h_c[i] - h_c_ref[i]);
4459     if (err > max_err) max_err = err;
4460 }
4461 printf("| %-35s | %.6e | %-4s |\n", "SGEMM-Vec4", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4462
4463 free(h_a); free(h_b); free(h_c); free(h_c_ref);
4464 cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
4465 cublasDestroy(handle);
4466 cudaEventDestroy(start);
4467 cudaEventDestroy(stop);
4468 }
4469
4470
4471 static void test_hgemm_mma(int M, int N, int K) {
4472

```

```

4473 size_t size_a = (size_t)M * K * sizeof(half);
4474 size_t size_b = (size_t)K * N * sizeof(half);
4475 size_t size_c = (size_t)M * N * sizeof(half);
4476
4477 half *h_a = (half *)malloc(size_a);
4478 half *h_b = (half *)malloc(size_b);
4479 half *h_c_ref = (half *)malloc(size_c);
4480
4481 srand(42);
4482 for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4483 for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4484
4485 // Kernel expects B^T [N×K] row-major layout (TN layout convention).
4486 // We store h_b as B [K×N] row-major for cuBLAS, then create B^T for kernel.
4487 size_t size_b_t = (size_t)N * K * sizeof(half);
4488 half *h_b_t = (half *)malloc(size_b_t);
4489 for (int n = 0; n < N; n++)
4490     for (int k = 0; k < K; k++)
4491         h_b_t[n * K + k] = h_b[k * N + n];
4492
4493 half *d_a, *d_b, *d_b_t, *d_c;
4494 check(cudaMalloc(&d_a, size_a), "hgemm alloc A");
4495 check(cudaMalloc(&d_b, size_b), "hgemm alloc B (cuBLAS)");
4496 check(cudaMalloc(&d_b_t, size_b_t), "hgemm alloc B_t (kernel)");
4497 check(cudaMalloc(&d_c, size_c), "hgemm alloc C");
4498
4499 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm H2D A");
4500 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm H2D B (cuBLAS)");
4501 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm H2D B_t (kernel)");
4502
4503 // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
4504 // Note: use CUBLAS_COMPUTE_16F to match the kernel's f16.f16.f16.f16 accumulation.
4505 cublasHandle_t handle;
4506 cublasCreate(&handle);
4507 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
4508 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
4509              &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
4510              &beta_h, d_c, CUDA_R_16F, N,
4511              CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
4512 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H ref");
4513
4514 // MMA kernel (TN layout: A row-major, B^T row-major = B col-major)
4515 cudaEvent_t start, stop;
4516 cudaEventCreate(&start);
4517 cudaEventCreate(&stop);
4518
4519 constexpr int BM = 128, BN = 128, BK = 16, K_STAGE = 3;
4520 size_t smem_bytes = K_STAGE * (BM * BK + BN * BK) * sizeof(half); // 24576
4521 dim3 block(256);
4522 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
4523 hgemm_mma_stages_tn<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
4524 check(cudaGetLastError(), "hgemm launch");
4525 check(cudaDeviceSynchronize(), "hgemm sync");
4526
4527 half *h_c = (half *)malloc(size_c);
4528 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H");
4529
4530 // Verify
4531 float max_err = 0.0f;
4532 for (int i = 0; i < M * N; i++) {
4533     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
4534     if (err > max_err) max_err = err;
4535 }

```

```

4536 printf("| %-35s | %.6e | %-4s |\n", "HGEMM MMA", max_err, max_err < 1.0f ? "PASS" : "FAIL");
4537
4538 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
4539 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
4540 cublasDestroy(handle);
4541 cudaEventDestroy(start);
4542 cudaEventDestroy(stop);
4543 }
4544
4545
4546 static void test_hgemm_swizzle(int M, int N, int K) {
4547     // HGEMM MMA Swizzle — m16n8k16 + multistage pipeline + TN 布局 + XOR swizzle
4548     // TN layout: C[M×N] = A[M×K] × BT[N×K]
4549     // Kernel: hgemm_mma_stages_tn_swizzle with default template params
4550
4551     size_t size_a = (size_t)M * K * sizeof(half);
4552     size_t size_b = (size_t)K * N * sizeof(half);
4553     size_t size_c = (size_t)M * N * sizeof(half);
4554
4555     half *h_a = (half *)malloc(size_a);
4556     half *h_b = (half *)malloc(size_b);
4557     half *h_c_ref = (half *)malloc(size_c);
4558
4559     srand(42);
4560     for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4561     for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4562
4563     // Kernel expects BT [N×K] row-major layout (TN layout convention).
4564     size_t size_b_t = (size_t)N * K * sizeof(half);
4565     half *h_b_t = (half *)malloc(size_b_t);
4566     for (int n = 0; n < N; n++)
4567         for (int k = 0; k < K; k++)
4568             h_b_t[n * K + k] = h_b[k * N + n];
4569
4570     half *d_a, *d_b, *d_b_t, *d_c;
4571     check(cudaMalloc(&d_a, size_a), "hgemm_swizzle alloc A");
4572     check(cudaMalloc(&d_b, size_b), "hgemm_swizzle alloc B (cuBLAS)");
4573     check(cudaMalloc(&d_b_t, size_b_t), "hgemm_swizzle alloc B_t (kernel)");
4574     check(cudaMalloc(&d_c, size_c), "hgemm_swizzle alloc C");
4575
4576     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm_swizzle H2D A");
4577     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm_swizzle H2D B (cuBLAS)");
4578     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm_swizzle H2D B_t (kernel)");
4579
4580     // cuBLAS FP16 reference
4581     cublasHandle_t handle;
4582     cublasCreate(&handle);
4583     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
4584     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
4585                 &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
4586                 &beta_h, d_c, CUDA_R_16F, N,
4587                 CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
4588     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_swizzle D2H ref");
4589
4590     // MMA swizzle kernel (default params: K_STAGE=3)
4591     constexpr int BM = 128, BN = 128, BK = 16, K_STAGE_S = 3;
4592     size_t smem_bytes = K_STAGE_S * (BM * BK + BN * BK) * sizeof(half);
4593     dim3 block(256);
4594     dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
4595     hgemm_mma_stages_tn_swizzle<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
4596     check(cudaGetLastError(), "hgemm_swizzle launch");
4597     check(cudaDeviceSynchronize(), "hgemm_swizzle sync");
4598

```

```

4599     half *h_c = (half *)malloc(size_c);
4600     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_swizzle D2H");
4601
4602     // Verify
4603     float max_err = 0.0f;
4604     for (int i = 0; i < M * N; i++) {
4605         float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
4606         if (err > max_err) max_err = err;
4607     }
4608     printf("| %-35s | %.6e | %-4s |\n", "HGEMM MMA Swizzle", max_err, max_err < 1.0f ? "PASS" : "FAIL");
4609
4610     free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
4611     cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
4612     cublasDestroy(handle);
4613 }
4614
4615
4616 #if defined(NOTES_V2_ENABLE_CUTE)
4617 static void test_hgemm_cute(int M, int N, int K) {
4618     // HGEMM CuTe — SM80_16x8x16_F16F16F16F16_TN + Swizzle<3,3> + kStage=2
4619     // TN layout: C[MxN] = A[MxK] × B^T[NxK]
4620     // Kernel: hgemm_mma_stages_tn_cute via launch_hgemm_mma_stages_tn_cute
4621     // Tile: BM=128, BN=256, BK=32, 128 threads/block
4622
4623     size_t size_a = (size_t)M * K * sizeof(half);
4624     size_t size_b = (size_t)K * N * sizeof(half);
4625     size_t size_c = (size_t)M * N * sizeof(half);
4626
4627     half *h_a = (half *)malloc(size_a);
4628     half *h_b = (half *)malloc(size_b);
4629     half *h_c_ref = (half *)malloc(size_c);
4630
4631     srand(42);
4632     for (int i = 0; i < M * K; i++)
4633         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4634     for (int i = 0; i < K * N; i++)
4635         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4636
4637     // CuTe kernel expects B^T [NxK] row-major (TN layout).
4638     size_t size_b_t = (size_t)N * K * sizeof(half);
4639     half *h_b_t = (half *)malloc(size_b_t);
4640     for (int n = 0; n < N; n++)
4641         for (int k = 0; k < K; k++)
4642             h_b_t[n * K + k] = h_b[k * N + n];
4643
4644     half *d_a, *d_b, *d_b_t, *d_c;
4645     check(cudaMalloc(&d_a, size_a), "hgemm_cute alloc A");
4646     check(cudaMalloc(&d_b, size_b), "hgemm_cute alloc B (cuBLAS)");
4647     check(cudaMalloc(&d_b_t, size_b_t), "hgemm_cute alloc B_t (kernel)");
4648     check(cudaMalloc(&d_c, size_c), "hgemm_cute alloc C");
4649
4650     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm_cute H2D A");
4651     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm_cute H2D B (cuBLAS)");
4652     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm_cute H2D B_t (kernel)");
4653
4654     // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
4655     cublasHandle_t handle;
4656     cublasCreate(&handle);
4657     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
4658     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
4659                 CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
4660                 CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
4661     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_cute D2H ref");

```

```

4662 // CuTe kernel (Stages=2, TN layout)
4663 launch_hgemm_mma_stages_tn_cute<half, 2>(d_a, d_b_t, d_c, M, N, K);
4664 check(cudaGetLastError(), "hgemm_cute launch");
4665 check(cudaDeviceSynchronize(), "hgemm_cute sync");
4666
4667 half *h_c = (half *)malloc(size_c);
4668 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_cute D2H");
4669
4670 // Verify
4671 float max_err = 0.0f;
4672 for (int i = 0; i < M * N; i++) {
4673     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
4674     if (err > max_err) max_err = err;
4675 }
4676 printf("| %-35s | %.6e | %-4s |\n", "HGEMM CuTe", max_err, max_err < 1.0f ? "PASS" : "FAIL");
4677
4678 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
4679 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
4680 cublasDestroy(handle);
4681 }
4682 #endif /* NOTES_V2_ENABLE_CUTE */
4683
4684
4685
4686 #if defined(NOTES_V2_ENABLE_WGMMA)
4687 static void test_hgemm_wgmma(int M, int N, int K) {
4688     // HGEMM WGMMA — m64n128k16 + TMA + Warp Specialization (Hopper SM90+)
4689     // TN layout: C[M×N] = A[M×K] × BT[N×K]
4690     // Kernel: hgemm_wgmma_stages_tn with default template params
4691
4692     constexpr int BM = 128, BN = 128, BK = 64, K_STAGE = 3, NUM_THREADS = 256;
4693
4694     // M, K must be divisible by tile dims
4695     if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
4696         printf("| %-35s | %-12s | %-4s |\n",
4697             "HGEMM WGMMA", "SKIP", "SKIP");
4698         return;
4699     }
4700
4701     size_t size_a = (size_t)M * K * sizeof(half);
4702     size_t size_b = (size_t)K * N * sizeof(half);
4703     size_t size_c = (size_t)M * N * sizeof(half);
4704
4705     half *h_a = (half *)malloc(size_a);
4706     half *h_b = (half *)malloc(size_b);
4707     half *h_c_ref = (half *)malloc(size_c);
4708
4709     srand(42);
4710     for (int i = 0; i < M * K; i++)
4711         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4712     for (int i = 0; i < K * N; i++)
4713         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4714
4715     // BT [N×K] row-major for TN layout (same as hgemm_mma kernel)
4716     size_t size_b_t = (size_t)N * K * sizeof(half);
4717     half *h_b_t = (half *)malloc(size_b_t);
4718     for (int n = 0; n < N; n++)
4719         for (int k = 0; k < K; k++)
4720             h_b_t[n * K + k] = h_b[k * N + n];
4721
4722     half *d_a, *d_b, *d_b_t, *d_c;
4723     check(cudaMalloc(&d_a, size_a), "wgmma alloc A");
4724     check(cudaMalloc(&d_b, size_b), "wgmma alloc B (cuBLAS)");

```

```

4725 check(cudaMalloc(&d_b_t, size_b_t), "wgmma alloc B_t (kernel)");
4726 check(cudaMalloc(&d_c, size_c), "wgmma alloc C");
4727
4728 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "wgmma H2D A");
4729 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice),
4730        "wgmma H2D B (cuBLAS)");
4731 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
4732        "wgmma H2D B_t (kernel)");
4733
4734 // cuBLAS FP16 reference (row-major idiom, same as hgemm_mma)
4735 cublasHandle_t handle;
4736 cublasCreate(&handle);
4737 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
4738 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
4739              CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
4740              CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
4741 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
4742        "wgmma D2H ref");
4743
4744 // Create TMA tensor maps for A and B^T
4745 // A[MxK] row-major: TMA box=(BK=64, BM=128), global shape=(K, M)
4746 // B^T[NxK] row-major: TMA box=(BK=64, BN=128), global shape=(K, N)
4747 CUtensorMap *tma_a =
4748     allocate_and_create_tensor_map(d_a, M / BM, K / BK);
4749 CUtensorMap *tma_b =
4750     allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
4751
4752 // Launch WGMMMA kernel
4753 // BLOCK_SWIZZLE=false → 2D grid, no swizzle
4754 size_t smem_bytes =
4755     K_STAGE * (BM * BK + BN * BK) * sizeof(half); // 3*16384*2 = 96KB
4756 cudaFuncSetAttribute(
4757     hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, NUM_THREADS, K_STAGE,
4758     false>,
4759     cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
4760
4761 dim3 block(NUM_THREADS);
4762 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
4763 hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, NUM_THREADS, K_STAGE, false>
4764     <<<grid, block, smem_bytes>>>(M, N, K, d_c, tma_a, tma_b);
4765 check(cudaGetLastError(), "wgmma launch");
4766 check(cudaDeviceSynchronize(), "wgmma sync");
4767
4768 half *h_c = (half *)malloc(size_c);
4769 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "wgmma D2H");
4770
4771 // Verify
4772 float max_err = 0.0f;
4773 for (int i = 0; i < M * N; i++) {
4774     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
4775     if (err > max_err)
4776         max_err = err;
4777 }
4778 printf("| %-35s | %.6e | %-4s |\n", "HGEMM WGMMMA", max_err,
4779        max_err < 1.0f ? "PASS" : "FAIL");
4780
4781 free(h_a);
4782 free(h_b);
4783 free(h_b_t);
4784 free(h_c);
4785 free(h_c_ref);
4786 cudaFree(d_a);
4787 cudaFree(d_b);

```

```

4788     cudaFree(d_b_t);
4789     cudaFree(d_c);
4790     cudaFree(tma_a);
4791     cudaFree(tma_b);
4792     cublasDestroy(handle);
4793 }
4794 #endif /* NOTES_V2_ENABLE_WGMMA */
4795
4796
4797 static void test_flash_attn(int seqlen, int head_dim) {
4798     // FlashAttention-2 with split-Q, MMA m16n8k16
4799     int B = 1, H = 8;
4800
4801     size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
4802
4803     srand(42);
4804     half *h_q = (half *)malloc(sz);
4805     half *h_k = (half *)malloc(sz);
4806     half *h_v = (half *)malloc(sz);
4807     for (int i = 0; i < B * H * seqlen * head_dim; i++) {
4808         h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4809         h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4810         h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4811     }
4812
4813     // CPU reference in FP32: 0 = softmax(Q @ K^T / sqrt(d)) @ V
4814     float *ref_q = (float *)malloc(sz * 4 / sizeof(half)); // 4x for float
4815     float *ref_k = (float *)malloc(sz * 4 / sizeof(half));
4816     float *ref_v = (float *)malloc(sz * 4 / sizeof(half));
4817     float *ref_o = (float *)malloc(sz * 4 / sizeof(half));
4818     int count = B * H * seqlen * head_dim;
4819     for (int i = 0; i < count; i++) {
4820         ref_q[i] = __half2float(h_q[i]);
4821         ref_k[i] = __half2float(h_k[i]);
4822         ref_v[i] = __half2float(h_v[i]);
4823     }
4824
4825     float scale = 1.0f / sqrtf((float)head_dim);
4826     for (int bi = 0; bi < B * H; bi++) {
4827         for (int qi = 0; qi < seqlen; qi++) {
4828             // S[qi, kj] = Q[qi, :] @ K[kj, :]^T * scale
4829             float smax = -INFINITY;
4830             float *S = (float *)malloc((size_t)seqlen * sizeof(float));
4831             for (int kj = 0; kj < seqlen; kj++) {
4832                 float s = 0.0f;
4833                 for (int d = 0; d < head_dim; d++)
4834                     s += ref_q[bi * seqlen * head_dim + qi * head_dim + d] *
4835                        ref_k[bi * seqlen * head_dim + kj * head_dim + d];
4836                 S[kj] = s * scale;
4837                 if (S[kj] > smax) smax = S[kj];
4838             }
4839             // softmax
4840             double sum_exp = 0.0;
4841             for (int kj = 0; kj < seqlen; kj++) sum_exp += (double)expf(S[kj] - smax);
4842             float inv_sum = 1.0f / (float)sum_exp;
4843             // O[qi, :] = sum_kj P[qi, kj] * V[kj, :]
4844             for (int d = 0; d < head_dim; d++) {
4845                 double o_acc = 0.0;
4846                 for (int kj = 0; kj < seqlen; kj++)
4847                     o_acc += (double)(expf(S[kj] - smax) * inv_sum) *
4848                        ref_v[bi * seqlen * head_dim + kj * head_dim + d];
4849                 ref_o[bi * seqlen * head_dim + qi * head_dim + d] = (float)o_acc;
4850             }

```



```

4851     free(S);
4852 }
4853 }
4854
4855 half *d_q, *d_k, *d_v, *d_o;
4856 check(cudaMalloc(&d_q, sz), "fa alloc Q");
4857 check(cudaMalloc(&d_k, sz), "fa alloc K");
4858 check(cudaMalloc(&d_v, sz), "fa alloc V");
4859 check(cudaMalloc(&d_o, sz), "fa alloc O");
4860 check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "fa H2D Q");
4861 check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "fa H2D K");
4862 check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "fa H2D V");
4863
4864 // Template params for kHeadDim=64, kStage=2
4865 constexpr int kHeadDim = 64, kStageV = 2, kPadV = 8;
4866 constexpr int kMmaAtomM = 16, kMmaAtomN = 8, kMmaAtomK = 16;
4867 constexpr int kMmaTileSeqLenQ = 4;
4868 constexpr int kMmaTileSeqLenK = 1;
4869 constexpr int kMmaTileSeqLenP = 4;
4870 constexpr int kMmaTileHeadDimV = 1;
4871 constexpr int kValTileSeqLenQ = 1;
4872 constexpr int kValTileSeqLenK = 8;
4873 constexpr int kValTileSeqLenP = 1;
4874 constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
4875
4876 constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;
4877 constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;
4878 size_t smem_bytes = (Br * (kHeadDim + kPadV) +
4879                     kStageV * Bc * (kHeadDim + kPadV) +
4880                     Bc * (kHeadDim + kPadV)) * sizeof(half);
4881
4882 dim3 block(128);
4883 dim3 grid((seqlen + Br - 1) / Br, B * H);
4884
4885 flash_attn_mma_stages_split_q<kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK,
4886     kMmaTileSeqLenQ, kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV,
4887     kValTileSeqLenQ, kValTileSeqLenK, kValTileSeqLenP, kValTileHeadDimV,
4888     kStageV, kPadV>
4889     <<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, head_dim);
4890 check(cudaGetLastError(), "fa launch");
4891 check(cudaDeviceSynchronize(), "fa sync");
4892
4893 half *h_o = (half *)malloc(sz);
4894 check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa D2H");
4895
4896 float max_err = 0.0f;
4897 for (int i = 0; i < count; i++) {
4898     float err = fabsf(__half2float(h_o[i]) - ref_o[i]);
4899     if (err > max_err) max_err = err;
4900 }
4901 printf("| %-35s | %.6e | %-4s |\n", "FlashAttn-SplitQ", max_err, max_err < 1e-1f ? "PASS" : "FAIL");
4902
4903 free(h_q); free(h_k); free(h_v); free(h_o);
4904 free(ref_q); free(ref_k); free(ref_v); free(ref_o);
4905 cudaFree(d_q); cudaFree(d_k); cudaFree(d_v); cudaFree(d_o);
4906 }
4907
4908
4909 int main(int argc, char *argv[]) {
4910 #if defined(NOTES_V2_ENABLE_WGMMA)
4911     cuInit(0); // Driver API init required for cuTensorMapEncodeTiled (TMA, sm_90a+)
4912 #endif
4913     int M = 1024, N = 1024, K = 1024;

```

```

4914 if (argc > 3) { M = atoi(argv[1]); N = atoi(argv[2]); K = atoi(argv[3]); }
4915
4916 printf("=== notes-v2.cu verification harness ===\n");
4917 printf("| %-35s | %-12s | %-4s |\n", "Kernel", "Max Err", "Pass");
4918 printf("|-----|-----|-----|\n");
4919
4920 test_block_reduce(N);
4921 test_dot(N);
4922 test_relu(1024);
4923 test_elementwise(1024);
4924 test_histogram(1024);
4925 test_merge_attn_states(512, 16, 128);
4926 test_softmax(256);
4927 test_rms_norm(8, 128);
4928 test_layer_norm(8, 128);
4929 test_rope(8, 128);
4930 test_mat_transpose(256, 256);
4931 test_mat_transpose_padded(256, 256);
4932 test_sgemv(256, 128);
4933 test_sgemm(M, N, K);
4934 test_hgemm_mma(M, N, K);
4935 test_hgemm_swizzle(M, N, K);
4936 #if (defined(NOTES_V2_ENABLE_CUTE))
4937     test_hgemm_cute(M, N, K);
4938 #endif
4939 #if (defined(NOTES_V2_ENABLE_WGMMA))
4940     test_hgemm_wgmma(M, N, K);
4941 #endif
4942 test_flash_attn(1024, 64);
4943
4944 printf("=== All tests done ===\n");
4945 return 0;
4946 }
4947
4948 // =====
4949 // Quick build & run reference
4950 // =====
4951 // # sm_89 单独编译 + 运行:
4952 // nvcc -std=c++20 -O2 -arch=sm_89 -lcublas -lcuda notes-v2.cu -o notes_v2_sm89.bin
4953 //
4954 // # sm_89 + CuTe (需要 CUTLASS include 路径, 编译 CUDA_HOME 指向的 CUTLASS 或
4955 // 项目内置 third-party/cutlass):
4956 // nvcc -std=c++20 -O2 -arch=sm_89 -DNOTES_V2_ENABLE_CUTE \
4957 // -I ../../third-party/cutlass/include \
4958 // -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm89.bin
4959 //
4960 // # sm_90a 单独编译 + 运行 (需要 Hopper GPU, H100/H200 均可):
4961 // nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a \
4962 // -DNOTES_V2_ENABLE_WGMMA -lcublas -lcuda notes-v2.cu -o notes_v2_sm90.bin
4963 //
4964 // # sm_90a + CuTe + WGMMA:
4965 // nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a \
4966 // -DNOTES_V2_ENABLE_CUTE -DNOTES_V2_ENABLE_WGMMA \
4967 // -I ../../third-party/cutlass/include \
4968 // -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm90.bin

```